

LAYER: 1_Core

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: **/obj/**, **/bin/**, **/*.png, **/*.jpg, **/node_modules/**, **/*.Designer.cs
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

Configuration.Web/Components/_Imports.razor
Configuration.Web/Components/Layout/MainLayout.razor
Configuration.Web/Components/Layout/ReconnectModal.razor
Configuration.Web/Components/NavMenu.razor
Configuration.Web/Components/NotFound.razor
Configuration.Web/Components/Routes.razor
Configuration.Web/Configurator/IWebAppConfigurator.cs
Configuration.Web/Configurator/WebAppConfigurator.cs
Configuration.Web/Configurator/WebScanningExtensions.cs
Configuration.Web/Promatis.Net.Configuration.Web.csproj
Configuration.Web/Properties/launchSettings.json
Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs
Configuration.Web/Services/TabNavigationService/TabItem.cs
Configuration.Web/Services/TabNavigationService/TabNavigationService.cs
Configuration.Web/Services/UiModuleService.cs
Configuration.Web/UiCommandBus/IUiCommandBus.cs
Configuration.Web/UiCommandBus/UiCommandBus.cs
Configuration.Web/WebAppBootstrapper.cs
Configuration/AppBootstrapper.cs
Configuration/AppConfigurator.cs
Configuration/AppInfrastructure.cs
Configuration/IAppConfigurator.cs
Configuration/Promatis.Net.Configuration.csproj
Configuration/ServiceScanningExtensions.cs
Data/Configurations/AuditLogConfiguration.cs
Data/Configurations/ModelBuilderConventions.cs
Data/DbContext/ApplicationDbContext.cs

Data/DBContext/ModelBuilderExtensions.cs
Data/Interceptors/ChangeEntryModel.cs
Data/Interceptors/DatabaseTriggerInterceptor.cs
Data/Interceptors/IUserProvider.cs
Data/Promatis.Net.Data.csproj
Data/Triggers/Castom/AuditTrigger.cs
Data/Triggers/Global/FluentValidationTrigger.cs
Data/Triggers/Global/ReferenceTreeParentTrigger.cs
Data/TriggerService/DatabaseTriggerService.cs
Data/TriggerService/EntityStateChangeEnum.cs
Data/TriggerService/IDatabaseTriggerService.cs
Data/TriggerService/PropertyChangeInfo.cs
Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs
Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgsBase.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs
Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs
Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs
Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs
Data/Validation/GlobalPolymorphicValidator.cs
Domain.Interface/Enums/EnumExtensions.cs
Domain.Interface/Interfaces/IAudit.cs
Domain.Interface/Interfaces/IDomainObject.cs
Domain.Interface/Interfaces/IDomainObjectHasKey.cs
Domain.Interface/Interfaces/IEntityCloner.cs
Domain.Interface/Interfaces/ISoftDeletable.cs
Domain.Interface/Interfaces/ITreeNode.cs
Domain.Interface/Promatis.Net.Domain.Interface.csproj
Domain/AuditLog/AuditLog.cs
Domain/DomainObject/DomainObject.cs
Domain/DomainObject/DomainObjectBase.cs
Domain/DomainObject/DomainObjectValidator.cs
Domain/EntityCloner/JsonEntityCloner.cs
Domain/Promatis.Net.Domain.csproj
Domain/ReferenceBase/ReferenceBase.cs
Domain/ReferenceBase/ReferenceBaseValidator.cs
Domain/ReferenceTreeBase/ReferenceTreeBase.cs
Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs
Promatis.Net.UI/_Imports.razor
Promatis.Net.UI/Components/ActionContext/DataContext.cs
Promatis.Net.UI/Components/ActionContext/EntityContext.cs
Promatis.Net.UI/Components/ActionContext/GridContext.cs
Promatis.Net.UI/Components/ActionContext/IWorkspaceContext.cs
Promatis.Net.UI/Components/ActionContext/ReferenceContext.cs
Promatis.Net.UI/Components/ActionContext/ReferenceTreeContext.cs
Promatis.Net.UI/Components/ActionContext/TreeContext.cs
Promatis.Net.UI/Components/ActionContext/WorkspaceContext.cs
Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor
Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor.cs
Promatis.Net.UI/Components/Dialogs/DialogTabConfig.cs
Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor
Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor.cs
Promatis.Net.UI/Components/UIControls/BaseUiControl.cs
Promatis.Net.UI/Components/UIControls/IHasOptions.cs
Promatis.Net.UI/Components/UIControls/IHasValue.cs
Promatis.Net.UI/Components/UIControls/IUiControl.cs
Promatis.Net.UI/Components/UIControls/UiControlAlignment.cs
Promatis.Net.UI/Components/Workspaces/GridPage.razor
Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs
Promatis.Net.UI/Components/Workspaces/ReferencePage.razor
Promatis.Net.UI/Components/Workspaces/ReferencePage.razor.cs
Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor
Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor.cs
Promatis.Net.UI/Components/Workspaces/TreePage.razor
Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs

```

Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor
Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs
Promatis.Net.UI/Controls/CreateChildButton.cs
Promatis.Net.UI/Controls/CreateEntityButton.cs
Promatis.Net.UI/Controls/DeleteEntityButton.cs
Promatis.Net.UI/Controls/EditEntityButton.cs
Promatis.Net.UI/Controls/ToolbarDivider.cs
Promatis.Net.UI/Data/Cache/FlatOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/HierarchicalOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/IOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/IUiOzuCache.cs
Promatis.Net.UI/Data/Cache/UiOzuCache.cs
Promatis.Net.UI/Data/IUiDataBroker.cs
Promatis.Net.UI/Data/UiDataBroker.cs
Promatis.Net.UI/Extensions/MudColorExtensions.cs
Promatis.Net.UI/Interfaces/IHasSelectedData.cs
Promatis.Net.UI/IUiModule.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogContext.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditActionSelect.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditEntitySelect.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditExportButton.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditPeriodPicker.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditToolbarDivider.cs
Promatis.Net.UI/Pages/Dashboard.razor
Promatis.Net.UI/Pages/Dashboard.razor.css
Promatis.Net.UI/Promatis.Net.UI.csproj
Promatis.Net.UI/Theme/PromatisTheme.cs
Promatis.Net.UI/UiModule.cs
Promatis.Net.UI/wwwroot/App.css
Service/Contracts/AuditLog/AuditLogSearchRequest.cs
Service/Contracts/AuditLog/PagedResult.cs
Service/Promatis.Net.Service.csproj
Service/Services/AuditLogService/AuditLogService.cs
Service/Services/AuditLogService/IAuditLogService.cs
Service/Services/BaseService/BaseService.cs
Service/Services/BaseService/IBaseService.cs
Service/Services/ReferenceService/IReferenceService.cs
Service/Services/ReferenceService/ReferenceService.cs
Service/Services/ReferenceTreeService/IReferenceTreeService.cs
Service/Services/ReferenceTreeService/ReferenceTreeService.cs
Service/Services/TreeService/ITreeService.cs
Service/Services/TreeService/TreeService.cs
Tests/AssemblyInfo.cs
Tests/Domain/DomainLogicTests.cs
Tests/Domain/DomainObjectBaseLogicTests.cs
Tests/Domain/DomainObjectBaseTests.cs
Tests/Domain/DomainObjectExtendedTests.cs
Tests/Domain/ReferenceBaseTests.cs
Tests/Domain/ReferenceTreeTests.cs
Tests/Domain/SoftDeletableTests.cs
Tests/Interceptor/DatabaseTriggerInterceptorTests.cs
Tests/Promatis.Net.ApplicationModel.Tests.csproj
Tests/Service/BaseServiceTests_Crud.cs
Tests/Service/BaseServiceTests.cs
Tests/Service/ReferenceServiceTests_Search.cs
Tests/Service/ReferenceTreeServiceTests.cs
Tests/Service/ServiceIntegrationTests.cs
Tests/Trigger/AuditTriggerTests.cs
Tests/Trigger/DatabaseTriggerServiceTests.cs
Tests/Trigger/FluentValidationTriggerTests.cs
Tests/Trigger/FullTriggerChainTests.cs
Tests/Trigger/ReferenceTreeParentTriggerTests.cs
Tests/Trigger/TreeTriggerChainTests.cs
Tests/Validator/DomainObjectValidatorTests.cs
Tests/Validator/ReferenceBaseValidatorTests.cs
Tests/Validator/ReferenceTreeBaseValidatorTests.cs
</directory_structure>

```

<files>

This section contains the contents of the repository's files.

```

<file path="Configuration.Web/Components/_Imports.razor">
@using System.Net.Http
@using System.Net.Http.Json

```

```

@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.JSInterop
@using MudBlazor
@using Promatis.Net.Ui
@using Promatis.Net.Configuration.Web
@using Promatis.Net.Configuration.Web.Components
@using Promatis.Net.Configuration.Web.Components.Layout
@using static Microsoft.AspNetCore.Components.Web.RenderMode
</file>

<file path="Configuration.Web/Components/Layout/MainLayout.razor">
@inherits LayoutComponentBase
@using Promatis.Net.Ui.Theme
@
@inject UiModuleService UiService
@inject TabNavigationService TabService
@implements IDisposable
@
@* A,,!Aô A A´ Aô : B 2Dô7D´2C 5CÂ ?D >C$0C"4CT@ D$5CÄK D DC´0C4>CÂ BE <Cô>C4> D 5Cd8CÄ0 *@
<MudThemeProvider Theme="PromatisTheme.DefaultTheme" @bind-IsDarkMode="_isDarkMode" />
<MudPopoverProvider />
@
<MudDialogProvider FullWidth="false"
    MaxWidth="MaxWidth.False"
    CloseButton="false"
    BackdropClick="false"
    NoHeader="false"
    Position="DialogPosition.Center"
    CloseOnEscapeKey="true" />
@
<MudSnackbarProvider />
@
<CascadingValue Value="this">
    <MudLayout>
        @* A$5D ECô0Dò ?C =CT;DÂ ?D 8C´>Cd5Cô8Dò ¢
        <MudAppBar Color="Color.Default" Elevation="1">
            @* A,,!Aô A A´ Aô : At0CÄ5Cô5Cô> Color.Primary C00 Color.Default *@
            <MudIconButton Icon="@Icons.Material.Filled.Menu"
                Color="Color.Inherit"
                Edge="Edge.Start"
                Size="Size.Large"
                OnClick="ToggleDrawer" />
            <MudStack Spacing="-1">
                @* AD5DD>C´BCô>CR =C 7C$0Cô8CR ¢
                <MudText Typo="Typo.h6" Class="ml-2">
                    B$5D BCä2D´9 C0@Cä5C=B Promatis Net
                </MudText>
            </MudStack>
        @
        <MudSpacer />
        @
        <MudIconButton Icon="@(_isDarkMode ? Icons.Material.Filled.LightMode :
Icons.Material.Filled.DarkMode)"
            Color="Color.Inherit"
            OnClick="ToggleDarkMode" />
    </MudAppBar>
    @
    @* Aô0Cô5C´L CÄ5Côn *@
    <MudDrawer @bind-Open="_drawerOpen" ClipMode="DrawerClipMode.Always"
Variant="DrawerVariant.Temporary" Overlay="true" OverlayAutoClose="true" Width="220px"
Elevation="1">
        <NavMenu />
    </MudDrawer>
    @
    <MudMainContent Class="pt-16 px-4 pb-4 d-flex flex-column"
        Style="height: calc(100vh - 8px); min-height: calc(100vh - 8px); margin-
top: 10px; display: flex; flex-direction: column; flex-grow: 1; overflow: hidden;">
        @
        @* A,,!Aô A A´ Aô : A4;Cä1C ;DÄ=D´9 ErrorBoundary D41D 0Cò A C$5D ECô5C4> D4@Cä2C00. B BC,,;C, fÄW
        @if (TabService.OpenTabs.Count == 0)
        {
            <div class="flex-grow-1 overflow-auto">
                <ErrorBoundary>
                    <ChildContent>

```

```

        @Body
    </ChildContent>
    <ErrorContent Context="exception">
        <MudAlert Severity="Severity.Error" Class="my-4">
            A@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1C=D$5D DCT9D 0: @exception.Message
        </MudAlert>
    </ErrorContent>
    </ErrorBoundary>
</div>
}
else
{
    <MudTabs @bind-ActivePanelIndex="TabService.ActiveTabIndex"
        Position="Position.Top"
        Color="Color.Default"
        SliderColor="Color.Primary"
        Class="mdi-tabs-container w-100 flex-grow-1 d-flex flex-column"
        Style="height: 100%; max-height: 100%; min-height: 0; display: flex; flex-
direction: column; flex-grow: 1;"
        TabPanelsClass="pa-0 flex-grow-1 d-flex flex-column overflow-hidden"
        ApplyEffectsToContainer="true"
        TabButtonsClass="rounded-t-lg shadow-sm"
        Outlined="true">

        @for (int i = 0; i < TabService.OpenTabs.Count; i++)
        {
            int localIndex = i;
            TabItem tab = TabService.OpenTabs[i];

            @* A,,!Aô A A´ Aô : AD>C 0C$;CT=C fÆW,ôAD$@D4:D$CD 0 CD;Dò AC <Cä3Cä BC 1-Cô0Cô5C´0,
            <MudTabPanel @key="tab" ID="@localIndex" Class="h-100 d-flex flex-column">
                <TabContent>
                    <div class="d-flex align-center gap-2" style="cursor: pointer;">
                        @if (!string.IsNullOrEmpty(tab.Icon))
                        {
                            <MudIcon Icon="@tab.Icon" Size="Size.Small" />
                        }
                        <MudText Typo="Typo.body2" Class="font-weight-
medium">@tab.Title</MudText>
                        <MudIconButton Icon="@Icons.Material.Filled.Close"
                            Size="Size.Small"
                            Color="Color.Default"
                            Class="pa-0 ms-1"
                            Style="width: 16px; height: 16px; min-width:
16px;"
                            OnClick="@((e) =>
TabService.CloseTabByIndex(localIndex))" />
                    </div>
                </TabContent>
                <ChildContent>
                    @* BôBCäB Cæ>CôBCT9Cô5D 6CTAD$:Cä 7C =C,,<C 5D" R 2D´ACäBD² 2Cæ;C 4Cæ8 C,
                    <div class="pa-2 rounded-b-lg w-100 d-flex flex-column flex-grow-1"
                        Style="height: 100%; max-height: 100%; min-height: 0;
overflow: hidden;">
                    <ErrorBoundary>
                        <ChildContent>
                            @* A,,!Aô A A´ Aô : AÄK Cä1Cä@C GC,,2C 5CÄ G-æ Ö-46ö× öæVçB 2 DD8C
                            @* Cæ>D$>D KC´ AC,,;Cä9 C$KD$0C48C$0CTB D BD 0C08DdC C00D,,5C4> CD5
                            <div class="d-flex flex-column flex-grow-1 w-100"
                                <DynamicComponent Type="@tab.ComponentType" />
                            </div>
                        </ChildContent>
                        <ErrorContent Context="exception">
                            <MudAlert Severity="Severity.Error" Class="my-4">
                                Aô@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1C=D$5D DCT9D 0 C$:
                            </MudAlert>
                        </ErrorContent>
                    </ErrorBoundary>
                </div>
            </ChildContent>
        </MudTabPanel>
    }
}

```

```

        </MudTabs>@
    }@
</MudMainContent>@
</MudLayout>@
</CascadingValue>@
@
@code {@
    private bool _drawerOpen = true;@
    private bool _isDarkMode; // Aô> D4<Cä;Dt0C08Dâ ?D 8C´>Cd5C08CR 7C ?D4AC=0CTBD 0 C" AC$5D$;Cä9 D$5CÄ5D
@
    protected override void OnInitialized()@
    {@
        base.OnInitialized();@
        TabService.OnTabsChanged += HandleTabsChanged;@
        _drawerOpen = false;@
    }@
@
    private void ToggleDrawer() => _drawerOpen = !_drawerOpen;@
@
    /// <summary>@
    /// Aô5D 5C@;DäGC 5D" DC´0C2 BE <C0>C4> D 5Cd8CÄ0 C, 7C AD$0C$;Dô5D" xVEF†VÖU &÷f-FW" <C4=Cä2CT=C0> C05D
    /// </summary>@
    private void ToggleDarkMode() => _isDarkMode = !_isDarkMode;@
@
    private void HandleTabsChanged() => InvokeAsync(StateHasChanged);@
@
    public void CloseDrawer()@
    {@
        if (_drawerOpen)@
        {@
            _drawerOpen = false;@
            StateHasChanged();@
        }@
    }@
@
    public void Dispose() => TabService.OnTabsChanged -= HandleTabsChanged;@
}
</file>

<file path="Configuration.Web/Components/Layout/ReconnectModal.razor">
<script type="module" src="@Assets["Components/Layout/ReconnectModal.razor.js"]"></script>@
@
<dialog id="components-reconnect-modal" data-nosnippet>@
    <div class="components-reconnect-container">@
        <div class="components-rejoining-animation" aria-hidden="true">@
            <div></div>@
            <div></div>@
        </div>@
        <p class="components-reconnect-first-attempt-visible">@
            Rejoining the server...@
        </p>@
        <p class="components-reconnect-repeated-attempt-visible">@
            Rejoin failed... trying again in <span id="components-seconds-to-next-attempt"></span>
seconds.@
        </p>@
        <p class="components-reconnect-failed-visible">@
            Failed to rejoin.<br />Please retry or reload the page.@
        </p>@
        <button id="components-reconnect-button" class="components-reconnect-failed-visible">@
            Retry@
        </button>@
        <p class="components-pause-visible">@
            The session has been paused by the server.@
        </p>@
        <p class="components-resume-failed-visible">@
            Failed to resume the session.<br />Please retry or reload the page.@
        </p>@
        <button id="components-resume-button" class="components-pause-visible components-resume-
failed-visible">@
            Resume@
        </button>@
    </div>@
</dialog>
</file>

<file path="Configuration.Web/Components/NavMenu.razor">

```

```

@using System.Reflection
@inject UiModuleService UiService
@inject TabNavigationService TabService
@inject ComponentRegistry Registry
@
<MudNavMenu>
    @{
        List<(string Title, string Href, string Icon, string? Group)> allItems = UiService.Modules
            .SelectMany(m => m.GetMenuItems())
            .ToList();
    }

    // AÄBD 8D >C$KC$0CT< DÔ;CT<CT=D$K C$5D EC05C4> D4@Cä2C00
    IEnumerable<(string Title, string Href, string Icon, string? Group)> topLevelItems =
allItems.Where(i => string.IsNullOrEmpty(i.Group));
    foreach ((string Title, string Href, string Icon, string? Group) item in topLevelItems)
    {
        <!-- A,,!Aô A A´ Aô : A$<CTAD$> Href C,,ACô>C´LctCCT< OnClick CD;Dò >D$:D KD$8Dò 2C;C 4Cä: C 5Cr
        <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title, item.Href,
item.Icon))">
            @item.Title
        </MudNavLink>
    }

    // A4@D4?C08D CCT< DÔ;CT<CT=D$K Cö> C@0D$5C4>D 8Dô<
    IOrderedEnumerable<IGrouping<string, (string Title, string Href, string Icon, string?
Group)>> pooledGroups = allItems
        .Where(i => !string.IsNullOrEmpty(i.Group))
        .GroupBy(i => i.Group!)
        .OrderBy(g => g.Key);

    foreach (IGrouping<string, (string Title, string Href, string Icon, string? Group)> group
in pooledGroups)
    {
        <MudNavGroup Title="@group.Key" Icon="@Icons.Material.Filled.Folder" Expanded="false">
            @foreach ((string Title, string Href, string Icon, string? Group) item in group)
            {
                <!-- A,,!Aô A A´ Aô : B41D 0C0 ‡&VbÂ 4Cä1C 2C´5C0 öä6Æ-6² 00í
                <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title,
item.Href, item.Icon))">
                    @item.Title
                </MudNavLink>
            }
        </MudNavGroup>
    }
}
</MudNavMenu>
@
@code {
    /// <summary>
    /// Að5D 5DT2C BD´2C 5CÂ AC²2Cä7C0>C´ MC²7CT<Cö;Dô@ D >CD8D$5C´LD :Cä3Cä <C :CTBC
    /// </summary>
    [CascadingParameter]
    protected MainLayout? MainLayoutInstance { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();
        IEnumerable<Assembly> assemblies = UiService.Modules.Select(m =>
m.GetType().Assembly).Distinct();
        Registry.RegisterModules(assemblies);
    }

    private void OpenMenuAsTab(string title, string href, string icon)
    {
        Type? componentType = Registry.GetComponentTypeByRoute(href);

        if (componentType != null)
        {
            // AäBC@D´2C 5CÂ 2C;C 4C²C
            TabService.OpenTab(title, href, icon, componentType);
        }

        // A 2D$>CÄ0D$8Dt5D :C, 7C :D KC$0CT< C >C²>C$>CR <CT=Dä ?Cä2CT@DR >C²=C ?CäAC´5 C²;C,,:C
        MainLayoutInstance?.CloseDrawer();
    }
    else
    {
    }
}

```

```

        System.Diagnostics.Debug.WriteLine($"AöCD$L {href} C05 Ct0D 5C48D BD 8D >C$0C0 2 Cα>CÄ?Cä=CT=D$0D
    }D
}D
}
</file>

<file path="Configuration.Web/Components/NotFound.razor">
@page "/not-found"
@layout MainLayout
D
<PageTitle>404 - B BD 0C08Dd0 C05 C00C"4CT=C Âö vUF-FÆSí
D
<MudContainer MaxWidth="MaxWidth.Small" Class="d-flex align-center justify-center" Style="height:
70vh;">D
    <MudPaper Elevation="3" Class="pa-8 text-center" Style="width: 100%;">D
        <MudIcon Icon="@Icons.Material.Filled.SearchOff" Color="Color.Error" Size="Size.Large"
Class="mb-4" />D
        <MudText Typo="Typo.h5" Class="mb-2">AäHC,,1Cα0 404</MudText>D
        <MudText Typo="Typo.body2" Class="mud-text-secondary mb-6">D
            A,,7C$8C08D$5, Ct0C0@C HC,,2C 5CÄKC' 0CD@CTA C05 D CD"5D BC$CCTB, C'8C > D >CäBC$5D$AD$2D4ND"8C' <C
        </MudText>D
        <MudButton Variant="Variant.Filled" Color="Color.Primary" href="/">D
            A$5D =D4BDÄADò =C 3C'0C$=D4ND
        </MudButton>D
    </MudPaper>D
</MudContainer>
</file>

<file path="Configuration.Web/Components/Routes.razor">
@using System.Reflection
@inject UiModuleService UiService
D
<Router AppAssembly="@typeof(Routes).Assembly"
    AdditionalAssemblies="@_moduleAssemblies"
    NotFoundPage="@typeof(NotFound)">D
    <Found Context="routeData">D
        <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />D
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />D
    </Found>D
</Router>D
D
@code {D
    private Assembly[] _moduleAssemblies = [];D
D
    protected override void OnInitialized()D
    {D
        _moduleAssemblies = UiService.Modules
            .Select(m => m.GetType().Assembly)
            .Distinct()
            .ToArray();D
        D
        base.OnInitialized();D
    }D
}
</file>

<file path="Configuration.Web/Configurator/IWebAppConfigurator.cs">
namespace Promatis.Net.Configuration.Web;
D
public interface IWebAppConfigurator : IAppConfigurator
{D
    void ConfigureMiddleware(WebApplication app);D
}
</file>

<file path="Configuration.Web/Configurator/WebAppConfigurator.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor.Services;
using Promatis.Net.UI;
using System.Reflection;
D
namespace Promatis.Net.Configuration.Web;
D
/// <summary>D
/// A 0Ct>C$KC' :Cä=DD8C4CD 0D$>D 4C'0 Web-C0@C,,;Cä6CT=C,,9 C00 C 0Ct5 Blazor C, xVD&Æ |÷"í
/// </summary>D

```



```

public class WebAppConfigurator<TRootComponent> : AppConfigurator, IWebAppConfigurator
{
    where TRootComponent : IComponent
    {
        public override void ConfigureServices(IServiceCollection services, IConfiguration
configuration)
        {
            // 1. A 0Ct>C$0Dò 8CÔDD 0D BD CC=BD4@C „ ABÂ !CT@C$8D K, A$0C´8CD0D$>D K, B$@C„3C45D K, B :C =CT@ DL
            base.ConfigureServices(services, configuration);
        }

        // 2. UI C„=DD@C AD$@D4:D$CD 0 (Aô>C„AC¢ •V”ÖöGVÆR 2 D 1Cä@C=0DRÂ =C 2C„3C FC„0)
        services.AddWebInfrastructure(projectPrefix: "Promatis.");

        // B B "AT A A A Aä (MDI): B 5C48D BD 8D CCT< C„=DD@C AD$@D4:D$CD C CÄ=Cä3Cä2C¢;C 4CäGCÔ>C4> C„
        services.AddSingleton<ComponentRegistry>();
        services.AddScoped<TabNavigationService>();

        // 3. B ?CTFC„DC„:C &Æ |÷" „-çFW& 7F-fR 6W'fW"•
        services.AddRazorComponents()
            .AddInteractiveServerComponents();

        // 4. B 5C48D BD 0Dd8Dò 8 CÔ0D BD >C":C xVD&Æ |÷-
        services.AddMudServices(config =>
        {
            config.SnackbarConfiguration.PositionClass =
MudBlazor.Defaults.Classes.Position.BottomRight;
            config.SnackbarConfiguration.PreventDuplicates = false;
            config.SnackbarConfiguration.NewestOnTop = true;
            config.SnackbarConfiguration.ShowCloseIcon = true;
            config.SnackbarConfiguration.VisibleStateDuration = 5000;
        });

        // --- B A4 B "B Bd Bò Aô$B B "B #A= "B4 Aô A4 Bô B Aô A "BD B B² $ôô D•2 T´ òòÝ

        // 1. B 5C48D BD 0Dd8Dò At#-C=MD„0 C, 4CTDCä;D$=Cä9 Cò;CäAC= >C´ AD$@C BCT3C„8 CÄCD$0Dd8C•
        services.AddTransient(typeof(IUiOzuCache<>), typeof(UiOzuCache<>));
        services.AddTransient(typeof(IOzuMutationStrategy<>), typeof(FlatOzuMutationStrategy<>));

        // 2. B 5C48D BD 0Dd8Dò CCô8C$5D AC ;DÄ=Cä3Cä D >C=5D 0 CD0CÔ=D´E (3 C45Cô5D 8C¢0?C @C <CTBD 0: TENT
        services.AddTransient(typeof(IUiDataBroker<,,>), typeof(UiDataBroker<,,>));

        // 3. B„8CÔ0 UI-D >C KD$8C•
        services.AddScoped<IUiCommandBus, UiCommandBus>();

        // B 5C48D BD 0Dd8Dò AC„AD$5CÄ=Cä3Cä 0C=ACTAD >D 0 CD;Dò ?Cä4CD5D 6C=8 C´5Cô8C$>C4> D 0Ct@CTHCT=C„0 S
        services.AddHttpContextAccessor();
    }

    /// <summary>
    /// Aô0D BD >C":C :Cä=C$5C"5D 0 Cä1D 0C >D$:C, ...EE ô7C ?D >D >C" „Ö-FFÆWv &R´ı
    /// </summary>
    public virtual void ConfigureMiddleware(WebApplication app)
    {
        // A 0Ct>C$KCR Ö-FFÆWv &W2 4C´0 D 0C >D$K Web-Cô@C„;Cä6CT=C„0D
        app.UseStaticFiles();
        app.UseAntiforgery();

        // 1. Aô>C´CDt0CT< Ct0D 5C48D BD 8D >C$0CÔ=D´5 UI-D 1Cä@C=8 C„7 DI-C= >CÔBCT9CÔ5D 0 DT>D BC
        using IServiceScope scope = app.Services.CreateScope();
        var uiService = scope.ServiceProvider.GetRequiredService<UiModuleService>();

        Assembly[] moduleAssemblies = uiService.Modules
            .Select(m => m.GetType().Assembly)
            .Distinct()
            .ToArray();

        // A„ A„&A„ A´ At Bd Bò A B$+ A$ A´ AD AÆ¢ C ?Cä;CÔ0CT< D 5CTAD$@ D >D4BCä2 C= >CÄ?Cä=CT=D$0CÄ8 C„7
        var componentRegistry = scope.ServiceProvider.GetRequiredService<ComponentRegistry>();
        componentRegistry.RegisterModules(moduleAssemblies);

        // 2. B 5C48D BD 8D CCT< C= >D =CT2Cä9 C= >CÄ?Cä=CT=D" 8 D :C =C„@D45CÂ AD$@C =C„FD² <Cä4D4;CT9 CÔ0 D 5
        app.MapRazorComponents<TRootComponent>()
            .AddInteractiveServerRenderMode()
            .AddAdditionalAssemblies(moduleAssemblies);
    }

    public override void ConfigureApp(IHost app)

```

```

{
    // A$Kct>C" 1C 7Cä2Cä9 C,,=C,,FC,,0C`8Ct0Dd8C, ,,0C$BCä@CT3C,,AD$@C FC,,O D$@C,,3C45D >C" AB GCT@CT7 D :C =
    base.ConfigureApp(app);
}
// At4CTADÂ <Cä6C0> CD>C 0C$8D$L C0@Cä2CT@C=C Ct4Cä@Cä2DÄ0 A C,,;C, =C GC ;DÄ=D´9 Seed CD0C0=D´E CD;
}
}
</file>

<file path="Configuration.Web/Configurator/WebScanningExtensions.cs">
using Promatis.Net.UI;
using Promatis.Net.UI.Components;
using System.Reflection;
namespace Promatis.Net.Configuration.Web;
public static class WebInfrastructureExtensions
{
    public static void AddWebInfrastructure(this IServiceCollection services, string projectPrefix
= "Promatis.")
    {
        Console.WriteLine("[SCANNER] At0C0CD : C0@Cä3D 5C$0 D 1Cä@Cä: CÄ>C0>C´8D$0 (B$>C´LC> UI-CÄ>CDCC´8)..
        string binPath = AppContext.BaseDirectory;
        // A,,!A0 A A´ A0 : A00 D4@Cä2C05 DD0C";Cä2Cä9 D 8D BCT<D² 8D"5CÄ BCä;DÄ:Cä BCR FÆÄÄ :CäBCä@D´5 C00Dt
        string[] assemblyFiles = Directory.GetFiles(binPath, $"{projectPrefix}*.UI.dll",
SearchOption.TopDirectoryOnly);
        foreach (string file in assemblyFiles)
        {
            try
            {
                // A0@C,,=D44C,,BCT;DÄ=Cä 7C 3D CCD0CT< C" 76VÖ&Ç"Æ0 D6öçFW†BäFVf VÇM
                Assembly.LoadFrom(file);
            }
            catch (Exception ex)
            {
                Console.WriteLine($"[SCANNER] A05 D44C ;CäADÄ 7C 3D CCT8D$L DD0C"; D 1Cä@C08 {Path.GetFileName
}
}
// A,,!A0 A A´ A0 : BD8C´LD$@D45CÄ F00 -âÄ 1CT@D0 AC >D :C, AD$@Cä3Cä A C0@CTDC,,:D >CÄ 8 Cä:Cä=Dt0
Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()
    .Where(a => a.FullName != null
        && a.FullName.StartsWith(projectPrefix)
        && a.GetName().Name!.EndsWith(".UI"))
    .Distinct()
    .ToArray();
    Console.WriteLine($"[SCANNER] B :C =C,,@Cä2C =C,,5 Ct0C$5D HCT=Cää C 9CD5C0> {assemblies.Length} Dd5C´
    Console.WriteLine();
    /*
    // A B$ AÄ B$ Bt B A / B A4 B "B Bd B0 T´0 Aä B$ A0!B$ A" A´ B$$Aä AÄ+D
    Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8D0 220:Cä=D$5C0AD$>C" AD$@C =C,,F C0> CÄ0D :CT@D2 •v÷&·7
    // A,,!A0 A A´ A0 : AÄ0D :CT@ C,,7CÄ5C05C0 =C 0C0BD40C´LC0KC´ •v÷&·7 6T6öçFW†B „1CT7 D ;Cä2C 7F-öâ
    var contextTypes = assemblies
        .SelectMany(s => s.GetTypes())
        .Where(t => typeof(IWorkspaceContext).IsAssignableFrom(t)
            && !t.IsAbstract
            && !t.IsInterface
            && !t.IsGenericTypeDefinition);
    int registeredContextsCount = 0;
    foreach (Type type in contextTypes)
    {
        // 1. B 5C48D BD 8D CCT< D 0CÄ :Cä=C0@CTBC0KC´ ?D 8C0;C 4C0>C´ :Cä=D$5C0AD" „=C ?D 8CÄ5D Ä Væ-Dö0
        services.AddTransient(type);
        registeredContextsCount++;
        Console.WriteLine($" % % [A0 A0"AT B "] {type.Name}");
        // 2. A 5D 5CÄ AD$@Cä3Cä 1C´8Cd0C´HC,,9 C 0Ct>C$KC´ BC,,? (C00C0@C,,<CT@, ReferenceContext<UnitOfMea
        Type? baseType = type.BaseType;
    }
}

```

```

        if (baseType != null && baseType.IsGenericType)
        {
            services.AddTransient(baseType, type);
        }

        string genericArgs = string.Join(", ", baseType.GetGenericArguments().Select(a =>
a.Name));

        string baseTypeName = baseType.Name.Split('`')[0];
        Console.WriteLine($"          % % %  AÄ0D :C ¢ ¶& 6UG- Tæ ÖWÓÇ¶vVæW&-4 &w7Óâ""½
    })
}
Console.WriteLine($"[SCANNER] B4AC05D,,=Câ @C 7C$5D =D4BCâ ·&Vv-7FW&VD6óçFW†G46÷VçG0 :Cä=D$5C=AD$>C" C
Console.WriteLine();*/
Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8D0 T'Ô:Cä<C0>C05C0BCä2 C, <Cä4D4;CT9 C00C$8C40Dd8C, GCT@C
// A 2D$>CÄ0D$8Dt5D :Cä5 D :C =C,,@Cä2C =C,,5 C, @CT3C,,AD$@C FC,,O C,,=D$5D DCT9D >C" •V"ÖöGVÆ]
services.Scan(scan => scan
    .FromAssemblies(assemblies)
    .AddClasses(c => c.AssignableTo<IUiModule>().Where(t => !t.IsAbstract))
    .AsImplementedInterfaces()
    .WithScopedLifetime()
);

// B 5C48D BD 0Dd8D0 2C HCT3Câ 0C4@CT3C BCä@C <Cä4D4;CT9
services.AddScoped<UiModuleService>();

// A´>C48D >C$0C08CR >C =C @D46CT=C0KDR ?D4=C=BCä2 CÄ5C0N
IEnumerable<Type> uiModuleTypes = assemblies
    .SelectMany(a => a.GetTypes())
    .Where(t => typeof(IUiModule).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract);

foreach (Type type in uiModuleTypes)
{
    try
    {
        if (Activator.CreateInstance(type) is IUiModule module)
        {
            Console.WriteLine($"[SCANNER] UI AÄ>CDCC´L: {module.Name}");
            List<(string Title, string Href, string Icon, string? Group)> menuItems =
module.GetMenuItems()?.ToList() ?? new();

            if (!menuItems.Any())
            {
                Console.WriteLine($"          % % [AôCD BCä5 CÄ5C0N]");
                continue;
            }

            IEnumerable<(string Title, string Href, string Icon, string? Group)> rootItems
= menuItems.Where(i => string.IsNullOrEmpty(i.Group));
            IEnumerable<IGrouping<string?, (string Title, string Href, string Icon, string?
Group)>> groupedItems = menuItems.Where(i => !string.IsNullOrEmpty(i.Group)).GroupBy(i => i.Group);

            foreach ((string Title, string Href, string Icon, string? Group) item in
rootItems)
            {
                Console.WriteLine($"          % % {item.Title} -> {item.Href}");
            }
            foreach (IGrouping<string?, (string Title, string Href, string Icon, string?
Group)> group in groupedItems)
            {
                Console.WriteLine($"          % % [{group.Key}]");
                List<(string Title, string Href, string Icon, string? Group)> itemsInGroup
= group.ToList();

                for (int i = 0; i < itemsInGroup.Count; i++)
                {
                    string prefix = (i == itemsInGroup.Count - 1) ? "          % % % " :
"          % % % ";
                    Console.WriteLine($"{prefix} {itemsInGroup[i].Title} ->
{itemsInGroup[i].Href}");
                }
            }
        }
    }
    catch (Exception ex)
    {

```

```

        Console.WriteLine($"[SCANNER] AäHC,,1Cð0 DtBCT=C,,0 CÄ5CÔN CD;Dò BC,,?C .G- Rää ÖWÓ¢ ¶W,äÖW76 v
    }ð
}ð
ð
    Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T'Ô:Cä<Cö>CÔ5CÔBCä2 D4ACô5D,,=Cä 7C 2CT@D,,5CÔ0");ð
    Console.WriteLine();ð
}ð
}
</file>

<file path="Configuration.Web/Promatis.Net.Configuration.Web.csproj">
<Project Sdk="Microsoft.NET.Sdk.Web">ð
ð
    <PropertyGroup>ð
        <TargetFramework>net10.0</TargetFramework>ð
        <ImplicitUsings>enable</ImplicitUsings>ð
        <Nullable>enable</Nullable>ð
"Ä÷WG WEG- SäÆ-'& '"Âô÷WG WEG- Sí
    </PropertyGroup>ð
ð
    <ItemGroup>ð
        <PackageReference Include="MudBlazor" Version="9.4.0" />ð
    </ItemGroup>ð
ð
    <ItemGroup>ð
        <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />ð
        <ProjectReference Include="..\Promatis.Net.UI\Promatis.Net.UI.csproj" />ð
    </ItemGroup>ð
ð
</Project>
</file>

<file path="Configuration.Web/Properties/launchSettings.json">
{ð
    "profiles": {ð
        "Promatis.Net.Configuration.Web": {ð
            "commandName": "Project",ð
            "launchBrowser": true,ð
            "environmentVariables": {ð
                "ASPNETCORE_ENVIRONMENT": "Development"ð
            },ð
            "applicationUrl": "https://localhost:56819;http://localhost:56820"ð
        },ð
    }ð
}ð
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs">
using System.Reflection;ð
using Microsoft.AspNetCore.Components;ð
ð
namespace Promatis.Net.Configuration.Web;ð
ð
public class ComponentRegistryð
{ð
    private readonly Dictionary<string, Type> _routeMap = new(StringComparer.OrdinalIgnoreCase);ð
ð
    /// <summary>ð
    /// B :C =C,,@D45D" AC >D :C, <Cä4D4;CT9 C, AD$@Cä8D" :C @D$C: URL -> B Æ 72 "C,,? Cð>CÄ?Cä=CT=D$0ð
    /// </summary>ð
    public void RegisterModules(IEnumerable<Assembly> assemblies)ð
    {ð
        foreach (Assembly assembly in assemblies)ð
        {ð
            IEnumerable<Type> componentTypes = assembly.GetTypes()ð
                .Where(t => typeof(IComponent).IsAssignableFrom(t) && !t.IsAbstract);ð
ð
            foreach (Type type in componentTypes)ð
            {ð
                // A,,ICT< C BD 8C CD" µ&÷WFT GG&-'WFUÔÂ 2 Cð>D$>D KC' :Cä<Cô8C'8D CCTBD 0 CD8D 5CðBC,,2C v
                IEnumerable<RouteAttribute> routeAttributes =
                type.GetCustomAttributes<RouteAttribute>();ð
                foreach (RouteAttribute attr in routeAttributes)ð
                {ð
                    _routeMap[attr.Template] = type;ð
                }ð
            }ð
        }ð
    }ð
}ð

```

```

        }
    }
}

public Type? GetComponentTypeByRoute(string route)
{
    return _routeMap.TryGetValue(route, out Type? type) ? type : null;
}
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabItem.cs">
namespace Promatis.Net.Configuration.Web;
{
    public class TabItem
    {
        public string Id { get; init; } = string.Empty; // A00D, #&Vm
        public string Title { get; init; } = string.Empty;
        public string Icon { get; init; } = string.Empty;
        public Type ComponentType { get; init; } = null!;
    }
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabNavigationService.cs">
namespace Promatis.Net.Configuration.Web;
{
    public class TabNavigationService
    {
        public List<TabItem> OpenTabs { get; } = new();
        public int ActiveTabIndex { get; set; }
        public event Action? OnTabsChanged;

        public void OpenTab(string title, string href, string icon, Type componentType)
        {
            TabItem? existingTab = OpenTabs.FirstOrDefault(t => t.Id.Equals(href,
                StringComparison.OrdinalIgnoreCase));

            if (existingTab != null)
            {
                ActiveTabIndex = OpenTabs.IndexOf(existingTab);
            }
            else
            {
                var newTab = new TabItem
                {
                    Id = href,
                    Title = title,
                    Icon = icon,
                    ComponentType = componentType
                };

                OpenTabs.Add(newTab);
                ActiveTabIndex = OpenTabs.Count - 1;
            }

            NotifyStateChanged();
        }

        /// <summary>
        /// A 5Ct>C00D =Cä5 Ct0C@D'BC,,5 C$:C'0CD:C, ?Cä 5E GC,,AC'>C$>CÄC C,,=CD5C@AD=
        /// </summary>
        public void CloseTabByIndex(int index)
        {
            if (index < 0 || index >= OpenTabs.Count) return;

            OpenTabs.RemoveAt(index);

            // A,,=D$5C';CT:D$CC ;DÄ=D'9 D 0D GCTB DD>C@CD 0 C00 CäAD$0C$HC,,5D 0 C$:C'0CD:C•
            if (ActiveTabIndex >= OpenTabs.Count)
            {
                ActiveTabIndex = OpenTabs.Count - 1;
            }

            if (ActiveTabIndex < 0 && OpenTabs.Count > 0)
            {
                ActiveTabIndex = 0;
            }
        }
    }
}

```

```

        }
    }
    NotifyStateChanged();
}

private void NotifyStateChanged() => OnTabsChanged?.Invoke();
}
</file>

<file path="Configuration.Web/Services/UiModuleService.cs">
using Promatis.Net.UI;
namespace Promatis.Net.Configuration.Web;
public class UiModuleService(IEnumerable<IUiModule> modules)
{
    public IEnumerable<IUiModule> Modules => modules;
}
</file>

<file path="Configuration.Web/UiCommandBus/IUiCommandBus.cs">
using MudBlazor;
namespace Promatis.Net.Configuration.Web;
/// <summary>
/// A,,=D$5D DCT9D HC,,=D² T'ÔACă1D'BC,,9D
/// </summary>
public interface IUiCommandBus
{
    /// <summary>
    /// B 5C48D BD 0Dd8Dò >C @C 1CăBDt8C=D C$Kct>C$0 DD>D <D² ,,2D'?Că;CÔ0CTBD O Cô@C,,=C,,<C ND"8CÂ <Că4D4;CT<,
    /// </summary>
    void RegisterHandler(string commandName, Func<Dictionary<string, object>, Task<DialogResult?>> handler);
}

/// <summary>
/// A$7C 8CĂ>CD5C"AD$2C,,5 CĂ5Cd4D2 <Că4D4;Dô<C, 2D ;CT?D4N (C$KctKC$0CTBD O C,,7 MDM)D
/// </summary>
Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object> parameters);
}
</file>

<file path="Configuration.Web/UiCommandBus/UiCommandBus.cs">
using MudBlazor;
namespace Promatis.Net.Configuration.Web;
/// <summary>
/// B,,8CÔ0 UI-D >C KD$8C•
/// </summary>
public class UiCommandBus : IUiCommandBus
{
    // Aô>D$>C=D>C 5Ct>Cô0D =D'9 D ;Că2C @DÂ 4C' O D 5C48D BD 0Dd8C, :Că<C =CB 2 D 0CÔBC 9CĂ5 SignalRD
    private readonly System.Collections.Concurrent.ConcurrentDictionary<string,
    Func<Dictionary<string, object>, Task<DialogResult?>>> _handlers = new();

    public void RegisterHandler(string commandName, Func<Dictionary<string, object>,
    Task<DialogResult?>> handler)
    {
        _handlers[commandName] = handler ?? throw new ArgumentNullException(nameof(handler));
    }

    public async Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object>
    parameters)
    {
        if (_handlers.TryGetValue(commandName, out Func<Dictionary<string, object>,
        Task<DialogResult?>>? handler))
        {
            return await handler(parameters);
        }
    }

    // ATAC'8 CĂ>CDCC'L (CÔ0Cô@C,,<CT@, DCA) CÔ5 Cô>CD:C'NDt5CÔ 2 C'0D4=Dt5D 5, D 8D BCT<C =CR CCô0CD5D"Â
    return null;
}
}

```

```

}
</file>

<file path="Configuration.Web/WebAppBootstrapper.cs">
using Microsoft.AspNetCore.Components;
{
namespace Promatis.Net.Configuration.Web;
{
// A00D ;CT4C08Cç &ö÷G7G& W"Â :CäBCä@D´9 D4<CT5D" 7C ?D4AC=0D$L Web-D 5D 2CT@D
public abstract class WebAppBootstrapper<TRootComponent> : AppBootstrapper
{
    where TRootComponent : IComponent
    {
        public override void Run(string[] args)
        {
            // B =Cä2C 7C 3D CCd0CT< DLL (C00 D ;D4GC 9 C0@D0<Cä3Cä 7C ?D4AC=0)D
            // LoadProjectAssemblies ("Promatis.") D46CR 2D´7C$0C00 C" & 6RÄ'VâÄ
            // C0> Ct4CTADÄ <D² 8D ?Cä;DÄ7D45CÄ vV$ Æ-6 F-öä'V-ÆFW"i

            // Aä B0 A "AT BÄ Aäç CT7 D0BCä3Cä F0Ö -â =CR CC$8CD8D" <Cä4D4;C, ?D 8 Type.GetTypeD
            LoadProjectAssemblies("Promatis.");

            WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
            List<IAppConfigurator> configs = GetConfigurators(builder.Configuration).ToList();

            foreach (IAppConfigurator cfg in configs)
                cfg.ConfigureServices(builder.Services, builder.Configuration);

            WebApplication app = builder.Build();

            // A00D BD >C":C Ö-FFÆWv &R <Cä4D4;CT9D
            foreach (IWebAppConfigurator cfg in configs.OfType<IWebAppConfigurator>())
            {
                cfg.ConfigureMiddleware(app);
            }

            foreach (IAppConfigurator cfg in configs)
                cfg.ConfigureApp(app);

            app.Run();
        }
    }
}
</file>

<file path="Configuration/AppBootstrapper.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Hosting;
using System.Reflection;
using System.Runtime.Loader;
{
namespace Promatis.Net.Configuration;
{
public abstract class AppBootstrapper
{
    public virtual void Run(string[] args)
    {
        // 1. A0 A,, B4 A,, "AT BÄ A / At A4 B4 A= DLL (DtBCä1D² G- RävWEG- R 8DR 2C,,4CT;)D
        LoadProjectAssemblies("Promatis.");

        HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

        // 2. B4 A0+A' Aä B A= A0$A,, B4 A "Aä Aä A,, JSOND
        List<IAppConfigurator> configurators = GetConfigurators(builder.Configuration).ToList();

        foreach (IAppConfigurator config in configurators)
        {
            config.ConfigureServices(builder.Services, builder.Configuration);
        }

        IHost host = builder.Build();

        foreach (IAppConfigurator config in configurators)
        {
            config.ConfigureApp(host);
        }

        host.Run();
    }
}
}

```

```

    }
}

protected virtual IEnumerable<IAppConfigurator> GetConfigurators(IConfiguration configuration)
{
    string[] typeNames = configuration.GetSection("LauncherSettings:Modules").Get<string[]>()
    ?? [];

    foreach (string name in typeNames)
    {
        // A,,ICT< D$8Cò 2Câ 2D 5DR 7C 3D CCd5CÔ=D´E D 1Cä@C¤0D]
        Type? type = AppDomain.CurrentDomain.GetAssemblies()
            .Select(a => a.GetType(name))
            .FirstOrDefault(t => t != null);

        if (type != null && typeof(IAppConfigurator).IsAssignableFrom(type))
        {
            if (Activator.CreateInstance(type) is IAppConfigurator instance)
            {
                yield return instance;
            }
        }
        else
        {
            Console.WriteLine($"[BOOTSTRAPPER][WARN] B$8Cò =CR =C 9CD5CÒ 8C´8 CÔ5 C$0C´8CD5CÓ¢ ¶æ ÖWò""½
        }
    }
}

protected void LoadProjectAssemblies(string prefix)
{
    string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
    if (path == null) return;

    foreach (string file in Directory.GetFiles(path, $"{prefix}*.dll"))
    {
        try
        {
            AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
        }
        catch { /* A,,3CÔ>D 8D CCT< CäHC,,1C¤8 Ct0C4@D47C¤8 */ }
    }
}
}
</file>

```

```

<file path="Configuration/AppConfigurator.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using System.Text.Json.Serialization;

namespace Promatis.Net.Configuration;

public class AppConfigurator : IAppConfigurator
{
    public virtual void ConfigureServices(IServiceCollection services, IConfiguration configuration)
    {
        // B :C =C,,@D45CÂ 2D Q (C$:C´NDt0Dò VF-EG&-vvW" 2 Cò@Cä5C¤BCR F F •
        services.AddDomainInfrastructure(projectPrefix: "Promatis.");

        // B 5C48D BD 8D CCT< C 0Ct>C$KCR ACT@C$8D KÐ
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
            sp.GetRequiredService<DatabaseTriggerService>());
        services.AddScoped<DatabaseTriggerInterceptor>();

        // Aä Bd Af¢ ¥40â =C AD$@Cä9C¤8Ð
        services.AddSingleton(new JsonSerializerOptions
        {
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
            DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
            WriteIndented = false
        });
    }
}

```



```

    });
}

// B A4 B "B Bd Bò A´ B$Aä AÄ AÔ Aä Aâ A´ AÔ B B #B" Aä!B$ A•
services.AddSingleton<IEntityCloner, JsonEntityCloner>();
}

public virtual void ConfigureApp(IHost app)
{
    if (app == null) throw new ArgumentNullException(nameof(app));

    // A,,=C,,FC,,0C´8Ct8D CCT< C4;Cä1C ;DÄ=D´9 Service Locator Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dý
    AppInfrastructure.Initialize(app.Services);

    using IServiceScope scope = app.Services.CreateScope();
    scope.ServiceProvider.AutoRegisterTriggers();
}
}
</file>

<file path="Configuration/AppInfrastructure.cs">
namespace Promatis.Net.Configuration;
{
    /// <summary>
    /// A4;Cä1C ;DÄ=C O D 8D BCT<C00Dò BCäGC=0 CD>D BD4?C : C,,=DD@C AD$@D4:D$CD =D´< D 5D 2C,,AC < Cò;C BDD>D <D²
    /// A,,=C,,FC,,0C´8Ct8D CCTBD O Cä4C0>C=0C BC0> Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dò 8 C,,AC=;DäGC 5D" @C 7CDCC$0C08CR
    /// </summary>
    public static class AppInfrastructure
    {
        private static IServiceProvider? _provider;
        private static readonly object _lock = new();

        /// <summary>
        /// Read-only CD>D BD4? C 3C´>C 0C´LC0>CÄC C=0C0BCT9C05D C Ct0C$8D 8CÄ>D BCT9.D
        /// </summary>
        public static IServiceProvider Provider
        {
            get
            {
                if (_provider == null)
                {
                    throw new InvalidOperationException(
                        "A=C,,BC,,GCTAC=0Dò >D,,8C :C  -æg& 7G'V7GW&R =CR 8C08Dd8C ;C,,7C,,@Cä2C =. " +
                        "B41CT4C,,BCTADÄÄ GD$> CÄ5D$>CB -æg& 7G'V7GW&Rä-æ-F- Ä-!R,' 2D´7C$0C0 2 Program.cs C,,;C
                    );
                }
                return _provider;
            }
        }

        /// <summary>
        /// Aò>D$>C=0C 5Ct>Cò0D =C O C,,=C,,FC,,0C´8Ct0Dd8Dò ;Cä:C BCä@C ä D´7D´2C 5D$ADò AD$@Cä3Cä >CD8Cò @C 7 Cò@
        /// </summary>
        public static void Initialize(IServiceProvider provider)
        {
            if (provider == null) throw new ArgumentNullException(nameof(provider));
            if (_provider != null) return;

            lock (_lock)
            {
                _provider ??= provider;
            }
        }
    }
}
</file>

<file path="Configuration/IAppConfigurator.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
{
    namespace Promatis.Net.Configuration;
    {
        public interface IAppConfigurator
        {
            void ConfigureServices(IServiceCollection services, IConfiguration configuration);
            void ConfigureApp(IHost app);
        }
    }
}

```

```

</file>

<file path="Configuration/Promatis.Net.Configuration.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="FluentValidation" Version="10.0.8" />
        <PackageReference Include="Microsoft.Extensions.DependencyInjection" Version="10.0.0" />
        <PackageReference Include="Promatis.Net.Data" Version="1.0.0" />
        <PackageReference Include="Promatis.Net.Domain" Version="1.0.0" />
        <PackageReference Include="Promatis.Net.Service" Version="1.0.0" />
        <PackageReference Include="System.Reflection" Version="4.7.0" />
        <PackageReference Include="System.Runtime.Loader" Version="4.7.0" />
    </ItemGroup>
    <Namespace>Promatis.Net.Configuration</Namespace>
    <PublicClass>
        public static class ServiceScanningExtensions
        {
            public static void AddDomainInfrastructure(this IServiceCollection services, string projectPrefix = "Promatis.")
            {
                Console.WriteLine();
                Console.WriteLine("[SCANNER] At 0C0D : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, 2 DI...");

                // 1. A0 A,, B4 A,, "AT BÄ A / At A4 B4 A° B Aä Aä D
                string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
                if (path != null)
                {
                    foreach (string file in Directory.GetFiles(path, $"{projectPrefix}*.dll"))
                    {
                        try
                        {
                            AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
                        }
                        catch (Exception ex)
                        {
                            string fileName = Path.GetFileName(file);
                            Console.ForegroundColor = ConsoleColor.Red;
                            Console.WriteLine($"[SCANNER][ERROR] A05 D44C ;CäADÂ 7C 3D CCT8D$L D 1Cä@C°C {fileName}");
                            Console.ResetColor();
                        }
                    }
                }
            }
        }

        // A 5D 5CÂ 2D 5 Ct0C4@D46CT=C0KCR AC >D :C, A C$0D,,8CÂ AC,,AD$5CÄ=D´< C0@CTDC,,:D >Cí
        Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies();
        assemblies = assemblies.Where(a => a.FullName != null && a.FullName.StartsWith(projectPrefix)).Distinct();
    </PublicClass>
</Project>
</file>

<file path="Configuration/ServiceScanningExtensions.cs">
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using System.Reflection;
using System.Runtime.Loader;

namespace Promatis.Net.Configuration;

public static class ServiceScanningExtensions
{
    public static void AddDomainInfrastructure(this IServiceCollection services, string projectPrefix = "Promatis.")
    {
        Console.WriteLine();
        Console.WriteLine("[SCANNER] At 0C0D : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, 2 DI...");

        // 1. A0 A,, B4 A,, "AT BÄ A / At A4 B4 A° B Aä Aä D
        string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
        if (path != null)
        {
            foreach (string file in Directory.GetFiles(path, $"{projectPrefix}*.dll"))
            {
                try
                {
                    AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
                }
                catch (Exception ex)
                {
                    string fileName = Path.GetFileName(file);
                    Console.ForegroundColor = ConsoleColor.Red;
                    Console.WriteLine($"[SCANNER][ERROR] A05 D44C ;CäADÂ 7C 3D CCT8D$L D 1Cä@C°C {fileName}");
                    Console.ResetColor();
                }
            }
        }

        // A 5D 5CÂ 2D 5 Ct0C4@D46CT=C0KCR AC >D :C, A C$0D,,8CÂ AC,,AD$5CÄ=D´< C0@CTDC,,:D >Cí
        Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies();
        assemblies = assemblies.Where(a => a.FullName != null && a.FullName.StartsWith(projectPrefix)).Distinct();
    }
}

```

```

        .ToArray();
    }

    int countBefore = services.Count;

    // 2. A B$ AÄ B$ Bt B Aä B A A,, Aä A A,, A, AT A,,!B$ A &A,,/ Bt B Ar 45%UDö-
    services.Scan(scan =>
    {
        // A`>C=0C`LC00D0 DD4=C=FC,,0 CD;D0 @CT:D4@D 8C$=Cä9 C0@Cä2CT@C=8 C$ACT9 Dd5C0>Dt:C, =C AC`5CD>C$0
        bool IsSubclassOfRawGeneric(Type generic, Type? toCheck)
        {
            while (toCheck != null && toCheck != typeof(object))
            {
                Type cur = toCheck.IsGenericType ? toCheck.GetGenericTypeDefinition() : toCheck;
                if (generic == cur) return true;
                toCheck = toCheck.BaseType;
            }
            return false;
        }

        scan.FromAssemblies(assemblies)
            // A,,!A0 A A` A0 : B 5C48D BD 0Dd8D0 !AT A$ B A" A C0>CD4CT@Cd:Cä9 C4;D41Cä:Cä3Cä =C AC`5CD
            .AddClasses(classes => classes.Where(type =>
                !type.IsAbstract &&
                !type.IsGenericTypeDefinition &&
                IsSubclassOfRawGeneric(typeof(BaseService<,,>), type)))
            .AsSelf()
            .AsImplementedInterfaces()
            .WithScopedLifetime()

            // B 5C48D BD 0Dd8D0 A A,, A "Aä Aä (FluentValidation)
            .AddClasses(c => c.AssignableTo(typeof(IValidator<>))
                .Where(t => !t.IsAbstract && !t.IsGenericTypeDefinition))
            .AsImplementedInterfaces()
            .WithTransientLifetime()

            // B 5C48D BD 0Dd8D0 "B A4 AT Aä (Before/After Save)
            .AddClasses(c => c.AssignableToAny(typeof(IBeforeSaveTrigger<>),
                typeof(IAfterSaveTrigger<>)).Where(t => !t.IsAbstract))
            .AsSelfWithInterfaces()
            .WithScopedLifetime();
    });

    // --- B A4 B "B Bd B0 A` A A`,A0 A4 A0 A` AÄ B $A0 A4 A$ A` AD B$ B B0 B ---
    // B 5C48D BD 8D CCT< C=0C$ >D$:D KD$K` 4Cd5C05D 8C$ä "CT?CT@DÄ ?D 8 Ct0C0@CäACR •f Æ-F F÷#Ä Dä1Cä9A
    // DI-C=0C0BCT9C05D ääUB 3C @C =D$8D >C$0C0=Cä >D$4C AD" =C H C0>C`8CÄ>D DC0KC` 4C,,AC05D$GCT@D
    services.AddScoped(typeof(IValidator<>), typeof(GlobalPolymorphicValidator<>));

    // 3. A,,!A0 A A` A0 Aä A, A AT A0 AR Aä A,, Aä A A,, B At#A`,B$ B$ A-
    List<ServiceDescriptor> newRegistrations = services.Skip(countBefore).ToList();
    var loggedTypes = new HashSet<string>();

    foreach (ServiceDescriptor reg in newRegistrations)
    {
        // A 5D 5CÄ @CT0C`LC0KC` BC,,? D 5C ;C,,7C FC,,8 (C" ?D 8Cä@C,,BCTBCR 8Cr -x ÆVÖVçF F-öâG- RÄ 8C00Dt5
        Type? implType = reg.ImplementationType ?? reg.ImplementationInstance?.GetType();

        if (implType == null) continue;

        // --- A A A,, A$ A` AD B$ B A" 00Ý
        bool isValidator = reg.ServiceType.IsGenericType &&
            reg.ServiceType.GetGenericTypeDefinition() == typeof(IValidator<>);

        if (isValidator && loggedTypes.Add($"Val:{implType.FullName}"))
        {
            var inheritanceChain = new List<string>();
            Type? currentType = implType;

            while (currentType != null && currentType != typeof(object))
            {
                string typeName = currentType.Name.Contains('.') ? currentType.Name.Split('.')
[0] : currentType.Name;
                inheritanceChain.Add(typeName);
                currentType = currentType.BaseType;
                if (typeName == "AbstractValidator") break;
            }
        }
    }

```

```

        string chainDisplay = string.Join(" -> ", inheritanceChain);
        Console.WriteLine($"[SCANNER] A$0C'8CD0D$>D $ ¶6† -äF-7 Æ -Ö $ •f Æ-F F÷""½
        continue; // A'>C2 4C'0 D0BCä9 Ct0C08D 8 C$KC$5CD5C0Â 8CD5CÂ : D ;CT4D4ND"5C•
    }
}

// --- A A A,, B B A,,!Aä (A00D ;CT4C08Cα>C" & 6U6W'f-6R' 00Ý
// A,,!A0 A A' A0 : B41D 0C0> Cd5D BCα>CR CD ;Cä2C,,5 reg.ServiceType == reg.ImplementationType, D
// D$0C$ :C : D 5D 2C,,AD² BCT?CT@DÂ @CT3C,,AD$@C,,@D4ND$AD0 ?C @C <C, C'0D A + A,,=D$5D DCT9D
bool isService = false;
var serviceChain = new List<string>();
Type? currentServiceType = implType;

while (currentServiceType != null && currentServiceType != typeof(object))
{
    string typeName = currentServiceType.Name.Contains('.') ?
currentServiceType.Name.Split('.')[0] : currentServiceType.Name;
    serviceChain.Add(typeName);

    if (currentServiceType.IsGenericType &&
currentServiceType.GetGenericTypeDefinition() == typeof(BaseService<,,>))
    {
        isService = true;
        break;
    }
    currentServiceType = currentServiceType.BaseType;
}

if (isService && loggedTypes.Add($"Srv:{implType.FullName}"))
{
    string chainDisplay = string.Join(" -> ", serviceChain);
    Console.WriteLine($"[SCANNER] B 5D 2C,,A: {chainDisplay}");
    continue;
}

// --- A A A,, B$ A,, A4 B A" 00Ý
bool isTrigger = implType.GetInterfaces().Any(i => i.IsGenericType &&
(i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));

if (isTrigger && loggedTypes.Add($"Tri:{implType.FullName}"))
{
    Console.WriteLine($"[SCANNER] B$@C,,3C45D $ ¶-× ÅG- Ræ ÖW0""½
}

Console.WriteLine("[SCANNER] A 2D$>CÄ0D$8Dt5D :C 0 D 5C48D BD 0Dd8D0 CD ?CTHC0> Ct0C$5D HCT=C ä""½
Console.WriteLine();
}

public static void AutoRegisterTriggers(this IServiceProvider serviceProvider)
{
    Console.WriteLine("[AUTOREG] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, BD 8C43CT@Cä2 C" F F &

    var triggerService = serviceProvider.GetRequiredService<IDatabaseTriggerService>();

    // 1. A0>C,,AC$ 2D 5DR :C'0D ACä2 D$@C,,3C45D >C" 2 D 1Cä@CαDR &öÖ F-2ä-
    IEnumerable<Type> triggerTypes = AppDomain.CurrentDomain.GetAssemblies()
        .Where(a => a.FullName?.StartsWith("Promatis.") == true)
        .SelectMany(s => s.GetTypes())
        .Where(t => t is { IsClass: true, IsAbstract: false } &&
            t.GetInterfaces().Any(i => i.IsGenericType &&
                (i.GetGenericTypeDefinition() ==
typeof(IBeforeSaveTrigger<>) ||
typeof(IAfterSaveTrigger<>))))
        .ToList();

    MethodInfo? registerMethod =
typeof(IDatabaseTriggerService).GetMethod(nameof(IDatabaseTriggerService.Register));

    int totalBindings = 0;

    foreach (Type triggerType in triggerTypes)
    {
        // A00DT>CD8CÂ 2D 5 C,,=D$5D DCT9D K D$@C,,3C45D >C"Â :CäBCä@D'5 D 5C ;C,,7D45D" 4C =C0KC' :C'0D A0
        IEnumerable<Type> interfaces = triggerType.GetInterfaces()

```

```

        .Where(i => i.IsGenericType &&
            (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
             i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));
    }

    foreach (Type @interface in interfaces)
    {
        // A0>C`CDt0CT< D$8C0 AD4IC0>D BC, ...B 8Cr "&Vf÷&U6 fUG&-vvW#ÂCâ•
        Type entityType = @interface.GetGenericArguments()[0];

        // Aâ?D 5CD5C`OCT< D$8C0 BD 8C43CT@C 4C`O C@C AC,,2Câ3Câ ;Câ3C
        string triggerKind = @interface.GetGenericTypeDefinition() ==
            typeof(IBeforeSaveTrigger<>)
                ? "Before"
                : "After ";

        try
        {
            // A$KctKC$0CT< Register<TEntity, TTrigger>()
            MethodInfo genericRegister = registerMethod!.MakeGenericMethod(entityType,
                triggerType);

            genericRegister.Invoke(triggerService, null);

            // A` A4 B A$ A0 AR #B AT(A0 A` B A$/At A•
            Console.WriteLine($"[AUTOREG] [{triggerKind}] {entityType.Name} <---
{triggerType.Name}");
            totalBindings++;
        }
        catch (Exception ex)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"[AUTOREG][ERROR] AâHC,,1C@0 C0@C,,2D07C@8 {triggerType.Name} C$ ¶VçF–G•
{ex.InnerException?.Message ?? ex.Message}");
            Console.ResetColor();
        }
    }

    Console.WriteLine($"[AUTOREG] At0C$5D HCT=Cââ !Câ7CD0C0> C0@C,,2D07Câ:: {totalBindings}");
    Console.WriteLine();
}
</file>

<file path="Data/Configurations/AuditLogConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
namespace Promatis.Net.Data;
public class AuditLogConfiguration : IEntityTypeConfiguration<AuditLog>
{
    public void Configure(EntityTypeBuilder<AuditLog> builder)
    {
        builder.Property(e => e.EntityName).HasMaxLength(100);
        builder.Property(e => e.Action).HasMaxLength(50);
        builder.Property(e => e.ChangedBy).HasMaxLength(100);

        // ChangesJson CâAD$0C$;D05CÂ 1CT7 C`8CÂ8D$0 (text/jsonb), D$0C$ :C : D ?C,,ACâ: C,,7CÂ5C05C08C' <Câ6CT
        builder.Property(e => e.ChangesJson).HasColumnType("jsonb");
    }
}
</file>

<file path="Data/Configurations/ModelBuilderConventions.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata;
using System.Linq.Expressions;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Data;
public static class ModelBuilderConventions
{
    public static void ApplyGlobalConventions(this ModelBuilder modelBuilder)
    {

```

```

foreach (ImmutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
{
    Type type = entityType.ClrType;

    // A`8CÄ8D$K CD;Dò AD$@Cä: (255 Cò> D4<Cä;Dt0C08Dâ•
    foreach (ImmutableProperty property in entityType.GetProperties())
    {
        if (property.ClrType == typeof(string))
        {
            if (property.GetMaxLength() == null)
                property.SetMaxLength(255);
        }
    }

    // A 2D$>-Cä;DäGC, 4C´0 Guid (ValueGeneratedNever, D$0C¢ :C : Id D >Ct4C 5D$ADò 2 DomainObject)
    if (typeof(IDomainObjectHasKey<Guid>).IsAssignableFrom(type))
    {
        modelBuilder.Entity(type).Property("Id").ValueGeneratedNever();
    }

    // A B$ -BD A´,B$ C, A$"Aâô Aô AT B 4C´0 SoftDelete Cä1D=5C=BCä2D
    if (typeof(ISoftDeletable).IsAssignableFrom(type))
    {
        ImmutableEntityType? baseType = entityType.BaseType;

        // B BC 2C„< DD8C´LD$@ D$>C´LC= C00 C= D 5C0L C„5D 0D EC„8 (C$:C´NDt0Dò 0C AD$@C :D$=D´9 Uni
        if (baseType == null || !typeof(ISoftDeletable).IsAssignableFrom(baseType.ClrType))
        {
            // B4AD$0C0>C$:C VW'f-ÇFW-
            modelBuilder.SetSoftDeleteFilter(type);
        }

        // B4AD$0C0>C$:C 8C04CT:D 0 C00 DeletedAt CD;Dò CD :Cä@CT=C„0 Ct0C0@CäACä2D
        modelBuilder.Entity(type).HasIndex("DeletedAt");
    }
}

private static void SetSoftDeleteFilter(this ModelBuilder modelBuilder, Type entityType)
{
    // AD;Dò E B0E , 2C 6C0> D BC 2C„BDÂ DC„;DÄBD =C :Cä@CT=DÂÂ 4C 6CR 5D ;C, >Cò 0C AD$@C :D$=D´9.D
    // B4AC´>C$8CR -46Æ 72 4CäAD$0D$>Dt=Câí
    if (entityType.IsClass)
    {
        modelBuilder.Entity(entityType).HasQueryFilter(GenerateFilter(entityType));
    }
}

private static LambdaExpression GenerateFilter(Type type)
{
    ParameterExpression parameter = Expression.Parameter(type, "e");
    // A„AC0>C´LCtCCT< Property CD;Dò 4CäAD$CC00 C¢ FVÆWFVD B GCT@CT7 C„=D$5D DCT9D 8C´8 D 2Cä9D BC$>D
    MemberExpression property = Expression.Property(parameter,
nameof(ISoftDeletable.DeletedAt));
    ConstantExpression nullConstant = Expression.Constant(null, typeof(DateTime?));
    BinaryExpression body = Expression.Equal(property, nullConstant);

    return Expression.Lambda(body, parameter);
}
}
</file>

<file path="Data/DbContext/ApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;

namespace Promatis.Net.Data;

public class ApplicationDbContext(
    DbContextOptions options,
    IConfiguration configuration,
    IServiceProvider? serviceProvider = null)
    : DbContext(options)
{

```

```

protected virtual string Schema => "public";
public DbSet<AuditLog> AuditLogs => Set<AuditLog>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.HasDefaultSchema(Schema);
    modelBuilder.ApplyModuleConfigurations(this);
    modelBuilder.ApplyGlobalConventions();
    base.OnModelCreating(modelBuilder);
}

// A,,!A A A' A B' A, A AT A+A' A A,, A"
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    base.OnConfiguring(optionsBuilder);

    // 1. A@Cä2CT@Dö5CÄ GD$> CÄK C" @C =D$0C"<CRÄ 0 Cö5 C" ?D >Dd5D ACR <C,,3D 0Dd8C, 4Ä•
    if (!EF.IsDesignTime && serviceProvider != null)
    {
        DatabaseTriggerInterceptor? interceptor = null;

        try
        {
            // A@>CöKD$:C  D >C CCT< D 0Ct@CTHC,,BDÄ =C ?D 0CÄCDä „5D ;C, ?D >C$0C"4CT@ Cä:C 7C ;D 0 S
            interceptor = serviceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as
            DatabaseTriggerInterceptor;
        }
        catch (InvalidOperationException)
        {
            // A@>CöKD$:C #D D ;C, ?D >C$0C"4CT@ C=D =CT2Cä9 (Root), D >Ct4C 5CÄ 66÷ R 2D CDt=D4N!
            // BÖBCä 3C @C =D$8D CCTB, DtBCä 66÷ VBÖ8CÖBCT@Dd5CöBCä@ D 0Ct@CTHC,,BD 0 C 5Cr >D,,8C >C# > Ro
            var scopeFactory = serviceProvider.GetService(typeof(IServiceScopeFactory)) as
            IServiceScopeFactory;

            if (scopeFactory != null)
            {
                // B >Ct4C 5CÄ 2D 5CÄ5CÖ=D4N Cä1C'0D BDÄ 2C,,4C,,<CäAD$8
                using IServiceScope scope = scopeFactory.CreateScope();
                interceptor =
                scope.ServiceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as DatabaseTriggerInterceptor;
            }
        }

        // ATAC'8 C,,=D$5D FCT?D$>D CD ?CTHCÖ> CÖ0C"4CT= Cä4CÖ8CÄ 8Cr ACö>D >C >C" r ?Cä4C;DäGC 5CÄ 5C4>
        if (interceptor != null)
        {
            optionsBuilder.AddInterceptors(interceptor);
        }
    }
}
}
</file>

<file path="Data/DbContext/ModelBuilderExtensions.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
using System.Reflection;
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
namespace Promatis.Net.Data;
public static class ModelBuilderExtensions
{
    public static void ApplyModuleConfigurations(this ModelBuilder modelBuilder, DbContext context)
    {
        // 1. B :C =C,,@D45CÄ BCä;DÄ:Cä AC >D :C, 4C =CÖKDR ?D >CT:D$0 Promatis.*.Data
        List<Assembly> dataAssemblies = AppDomain.CurrentDomain.GetAssemblies()
        .Where(a => {
            string? name = a.GetName().Name;
            return name != null &&
                name.StartsWith("Promatis.") &&
                name.EndsWith(".Data");
        }).ToList();

        // 2. A@>C'CDt0CT< D$8CöK D CD"=CäAD$5C'Ä :CäBCä@D'5 Dö2CÖ> Cä1D=0C$;CT=D² 2 D$5C=CD"5CÄ :Cä=D$5C=AD$

```

```

HashSet<Type> contextDbSetTypes = context.GetType()
    .GetProperties(BindingFlags.Public | BindingFlags.Instance)
    .Where(p => p.PropertyType.IsGenericType && p.PropertyType.GetGenericTypeDefinition()
== typeof(DbSet<>))
    .Select(p => p.PropertyType.GetGenericArguments()[0])
    .ToHashSet();

foreach (Assembly assembly in dataAssemblies)
{
    // A0C,"<CT=D05CÄ :Cä=DD8C4CD 0Dd8C, BCä;DÄ:Cä 4C´O D$5DR BC,"?Cä2, Cä>D$>D KCR <C BCT@C,"0C´LCÖK (
    modelBuilder.ApplyConfigurationsFromAssembly(assembly, type =>D
    {
        if (type is not { IsClass: true, IsAbstract: false, IsGenericTypeDefinition:
false })D
            return false;D

        // A00DT>CD8CÄ 8C0BCT@DD5C"A Cä>C0DC,"3D4@C FC,"8 C, BC,"? D CD"=CäAD$8, Cä>D$>D CDä >C00 C00D B
        Type? configInterface = type.GetInterfaces()
            .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IEntityTypeConfiguration<>));D

        if (configInterface == null) return false;D

        Type entityType = configInterface.GetGenericArguments()[0];D

        // Aä A,"A,"AT!Aä A' $A," BÄ"B ¢ D ;C, =C AD$@C 8C$0CT<C 0 D CD"=CäAD$L C 1D BD 0CäBC00,D
        // C0@Cä2CT@D05CÄ 5D BDÄ ;C, C C05E ECäBDÄ >CD8C0 6C,"2Cä9 C00D ;CT4C08C¢ 2 D$5CäCD"5CÄ F%6W
        if (entityType.IsAbstract)D
        {
            bool hasConcreteDerivedInDbSet = contextDbSetTypes.Any(t =>
t.IsSubclassOf(entityType));D
            if (!hasConcreteDerivedInDbSet) return false; // A0@Cä?D4ACä0CT< Cä>C0DC,"3D4@C FC,"N C 1D
        }D

        return type.GetConstructor(Type.EmptyTypes) != null;D
    });D
}D

// 3. A B$ -A," A0 B B A$ A0 AR´ B B B$ A$(A,"%» A B "B Aä&A," D
IEnumerable<Type> allAbstractEntities = dataAssemblies
    .SelectMany(a => a.GetTypes())D
    .Where(t => t is { IsClass: true, IsAbstract: true } && !t.IsGenericTypeDefinition);D

foreach (Type abstractType in allAbstractEntities)D
{
    bool isUsedInCurrentContext = contextDbSetTypes.Any(t => t == abstractType ||
t.IsSubclassOf(abstractType));D

    if (!isUsedInCurrentContext)D
    {
        modelBuilder.Ignore(abstractType);D
    }D
}D

// 4. A B$ AÄ B$ Bt B A / A0 B "B A" A At A´ B A$ A0 B´% AD B A$,AT A, A0 AT B A-
foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())D
{
    Type? clrType = entityType.ClrType;D
    if (clrType == null) continue;D

    // A,"ICT< D 5C ;C,"7C FC,"N C,"=D$5D DCT9D 0 ITreeNode<>D
    Type? treeInterface = clrType.GetInterfaces()
        .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(INode<>));D

    if (treeInterface != null)D
    {
        Type targetTreeType = treeInterface.GetGenericArguments()[0];D

        // A00D BD 0C,"2C 5CÄ BCä;DÄ:Cä =C 1C 7Cä2Cä< D4@Cä2C05 Cä1Dä0C$;CT=C,"0 D 2Cä9D BC" „GD$>C K
        if (clrType == targetTreeType)D
        {
            EntityTypeBuilder builder = modelBuilder.Entity(clrType);D

            // 1. A00D BD >C":C AC$0Ct8 B >CD8D$5C´L-A0>D$>CÄ>C-
            builder.HasOne("Parent")D

```


[illegible]

```

/// <summary>
/// A AC,,DT@Cä=CÖKC' ?CT@CTEC$0D$GC,,:, C$KCö>C'=Dö5CÄKC' Aä DC :D$8Dt5D :Cä3Cä ACäED 0C05C08D0 8Ct<CT=
/// AäBC$5Dt0CTB Ct0 C'>C48C=C Soft Delete, DD8C=AC FC,,N D =C,,<C=0 C,,7CÄ5C05C08C' 8 Ct0C0CD : D$@C,,3C45D
/// </summary>
public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(
    DbContextEventData eventData, InterceptionResult<int> result, CancellationToken ct =
default)
{
    if (eventData.Context == null) return result;

    // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =Cö> Cd8C$>C' 66÷ R 4C'0 Cö@CT4CäBC$@C ICT=C,,0 Cö@Cä1C
    using IServiceScope scope = scopeFactory.CreateScope();
    var triggerService = scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();

    string? userName = await GetUserNameInternalAsync(scope.ServiceProvider);

    // 1. Aö@CT4C$0D 8D$5C'LC00D0 >C @C 1CäBC=0 Soft Delete (AÄ0C4:Cä5 D44C ;CT=C,,5)
    // AÖ0DT>CD8CÄ 2D 5 D CD"=CäAD$8 D 8C0BCT@DD5C"ACä< ISoftDeletable, D2 :CäBCä@D'E D BC BD4A "B44C ;C
    IEnumerable<EntityEntry<ISoftDeletable>> entries =
eventData.Context.ChangeTracker.Entries<ISoftDeletable>()
    .Where(e => e.State == EntityState.Deleted);

    foreach (EntityEntry<ISoftDeletable> entry in entries)
    {
        // Aö>CD<CT=Dö5CÄ DC,,7C,,GCTAC=CR CCD0C'5C08CR =C >C =Cä2C'5C08CR 4C =CÖKDR 2 A D
        entry.State = EntityState.Modified;
        entry.Entity.DeletedAt = DateTime.UtcNow;
        entry.Entity.DeletedBy = userName;
    }

    // Aö@C,,=D44C,,BCT;DÄ=Cä 7C AD$0C$;Dö5CÄ Tb 6÷&R ?CT@CTADt8D$0D$L CD5D 5C$> C,,7CÄ5C05C08C' ?CäAC'5 Cö>
eventData.Context.ChangeTracker.DetectChanges();

    // 2. At0DT2C B C,,7CÄ5C05C08C' ,,ACäED 0C00CTB C00D$8C$=D'5 D$8CÖK CD0C0=D'E object? CD;Dö BCTAD$>C"•
    List<ChangeEntryModel> captured = CaptureChanges(eventData.Context, userName);

    // Aö@C,,2Dö7D'2C 5CÄ :Cä;C'5C=FC,,N C,,7CÄ5C05C08C' : C= >Cö:D 5D$=Cä<D2 MC=7CT<Cö;Dö@D2 :Cä=D$5C=AD$0 C
    _capturedChanges[eventData.Context.ContextId.InstanceId] = captured;

    // 3. A$0C'8CD0Dd8D0 „&Vf÷&U6 fR' GCT@CT7 Cd8C$>C' G&-vvW%6W'f-6]
    // At0CöCD :C 5D" 4Cä<CT=CÖKCR ?D 0C$8C'0 C, 2C ;C,,4C BCä@D² ?CT@CT4 D$5CÄÄ :C : CD0C0=D'5 Cö>Cö0CDCD
    foreach (ChangeEntryModel item in captured)
    {
        await triggerService.ValidateAsync(item.Entity, item.State, item.Changes,
eventData.Context);
    }

    return result;
}

/// <summary>
/// A AC,,DT@Cä=CÖKC' ?CT@CTEC$0D$GC,,:, C$KCö>C'=Dö5CÄKC' Aä!A' D4AC05D,,=Cä3Cä ACäED 0C05C08D0 8Ct<CT=
/// AäBC$5Dt0CTB Ct0 D42CT4Cä<C'5C08CR ?Cä4C08D GC,,:Cä2 C, BD 8C43CT@Cä2 Cö>D B-Cä1D 0C >D$:C, ,,DC 7C 6
/// </summary>
public override async ValueTask<int> SavedChangesAsync(
    SaveChangesCompletedEventData eventData, int result, CancellationToken ct = default)
{
    // A,,7C$;CT:C 5CÄ 7C EC$0Dt5C0=D'5 C,,7CÄ5C05C08D0 8 D @C 7D2 0D$>CÄ0D =Cä CCD0C'0CT< C,,E C,,7 D ;Cä2C
    if (eventData.Context != null &&
    _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out List<ChangeEntryModel>>
entries))
    {
        // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =Cö> Cd8C$>C' 66÷ R 4C'0 DD0Ctk Saved (C$KCö>C'=Dö5
        using IServiceScope scope = scopeFactory.CreateScope();
        var triggerService =
scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();

        // A AC,,DT@Cä=Cö> D42CT4Cä<C'0CT< D 8D BCT<D2 >C 8Ct<CT=CT=C,,ODR ,,=C ?D 8CÄ5D Ä 4C'0 Cä1C0>C$;C
        foreach (ChangeEntryModel item in entries)
        {
            await triggerService.NotifyAsync(item.Entity, item.State, item.Changes,
item.ChangedBy, item.ChangedAt);
        }

        return result;
    }

    /// <summary>
    /// Aö5D 5DT2C Bdt8C$ AC >Dö ?D 8 D >DT@C =CT=C,,8 C,,7CÄ5C05C08C'ä C @C =D$8D CCTB CäGC,,AD$:D2 :DÖHC 7C
    /// Cö@CT4CäBC$@C IC 0 D4BCTGC=8 Cö0CÄ0D$8 Cö@C, ?C 4CT=C,,8 D$@C =Ct0C=FC,,9.D

```

```

    /// </summary>
    public override Task SaveChangesFailedAsync(DbContextErrorEventData eventData,
        CancellationToken ct = default)
    {
        if (eventData.Context != null)
            _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out _);

        return base.SaveChangesAsync(eventData, ct);
    }

    /// <summary>
    /// A$=D4BD 5C0=C,,9 CÄ5D$>CB 0C00C'8Ct0 D$@CT:CT@C 8Ct<CT=CT=C,,9 EF Core.
    /// A$KDt8D ;D05D" 4CT;DÄBD2 „AD$0D KCR0=Cä2D'5 Ct=C GCT=C,,0 C0>C'5C'' 8 DD>D <C,,@D45D" <Cä4CT;C, 8Ct<CT=
    /// </summary>
    /// <param name="context">B$5C=CD"8C' :Cä=D$5C=AD" 1C 7D² 4C =C0KDRäÃ÷ & Óí
    /// <param name="userName">A,,<D0 ?Cä;DÄ7Cä2C BCT;D0Ä ACä2CT@D,,8C$HCT3Cä >C05D 0Dd8DäãÃ÷ & Óí
    /// <returns>B ?C,,ACä: D BD CC=BD4@C,,@Cä2C =C0KDR <Cä4CT;CT9 C,,7CÄ5C05C08C' Ç6VR 7&Vc0$6† ævTVÇG'"ÖöFVÄ"ó
    private List<ChangeEntryModel> CaptureChanges(DbContext context, string? userName)
    {
        // BD8C'LD$@D45CÄ BCä;DÄ:Cä AD4IC0>D BC, 2 D >D BCä0C08D0E AD>C 0C$;CT=C,,0, A,,7CÄ5C05C08D0 8C'8 B44C
        List<EntityEntry> entries = context.ChangeTracker.Entries()
            .Where(e => e.State is EntityState.Added or EntityState.Modified or EntityState.Deleted)
            .Where(e => !e.Entity.GetType().Name.Contains("AuditLog"))
            .ToList();

        var result = new List<ChangeEntryModel>();

        foreach (EntityEntry e in entries)
        {
            EntityStateChangeEnum state = MapState(e.State);

            // B ?CTFC,,DC,,GC0KC' <C ?C08C03 D >D BCä0C08D0 4C'0 "CÄ0C4:Cä CCD0C'5C0=D'E" Ct0C08D 5C•
            if (e.Entity is ISoftDeletable soft)
            {
                PropertyEntry prop = e.Property(nameof(ISoftDeletable.DeletedAt));
                if (prop.IsModified && soft.DeletedAt != null) state =
                    EntityStateChangeEnum.SoftDeleted;
            }

            List<PropertyChangeInfo> changes = new();

            // B FCT=C @C,,9 1: B CD"=CäAD$L CD>C 0C$;CT=C â D 5 D$5C=CD"8CR 7C00Dt5C08D0 AC$>C"AD$2 D GC,,BC
            if (state == EntityStateChangeEnum.Added)
            {
                changes = e.Properties
                    .Select(p => new PropertyChangeInfo
                    {
                        PropertyName = p.Metadata.Name,
                        OriginalValue = null,
                        CurrentValue = p.CurrentValue
                    })
                    .ToList();
            }
            // B FCT=C @C,,9 2: B CD"=CäAD$L C,,7CÄ5C05C00 C,,;C, <D03C= D44C ;CT=C â D'GC,,AC'0CT< D 0Ct=C,,FD2
            else if (state is EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted)
            {
                PropertyValues originalValues = e.OriginalValues;

                foreach (PropertyEntry p in e.Properties.Where(p => p.IsModified))
                {
                    string propertyName = p.Metadata.Name;
                    object? originalValue = originalValues[propertyName];
                    object? currentValue = p.CurrentValue;

                    // BD8C=AC,,@D45CÄ 8Ct<CT=CT=C,,5 D$>C'LC= CTAC'8 Ct=C GCT=C,,0 CD5C"AD$2C,,BCT;DÄ=Cä @C 7C'
                    if (!Equals(originalValue, currentValue))
                    {
                        changes.Add(new PropertyChangeInfo
                        {
                            PropertyName = propertyName,
                            OriginalValue = originalValue,
                            CurrentValue = currentValue
                        });
                    }
                }
            }
        }
    }
}

```

```

// AD0Cd5 CTAC'8 C=C';CT:Dd8Dò BCäGCTGCÔKDR 8Ct<CT=CT=C,,9 Cò>C'5C' „6† ævW2' ?D4AD$0 D
// C,,7-Ct0 D 0C0BC 9CÄ01C,,=CD8C03C &Æ |÷"Ä <D² B B A$ Aä 4Cä1C 2C'0CT< D CD'=CäAD$L C" AC08D
// CTAC'8 DD0C @C,,:C 7C DC,,:D 8D >C$0C'0 D >D BCä0C08CR ÖöF-f-VB -D$> C40D 0C0BC,,@D45D" R 4C
if (state == EntityStateChangeEnum.Modified && changes.Count == 0)D
{D
// B >Ct4C 5CÄ DC,,:D$8C$=D4N Ct0C08D L C,,7CÄ5C05C08Dò 4C'0 CD>CÄ5C0=D'E D$@C,,3C45D >C"Ä
// DtBCä1D² =CR ;Cä<C BDÄ 2C ;C,,4C FC,,N, C0> D >DT@C =C,,BDÄ 8CÄ?D4;DÄA CD;Dò T•
changes.Add(new PropertyChangedEventArgs
{D
    PropertyName = "Id",D
    OriginalValue = e.Property("Id").OriginalValue,D
    CurrentValue = e.Property("Id").CurrentValueD
});D
}D
D
result.Add(new ChangeEntryModelD
{D
    Entity = e.Entity,D
    State = state,D
    ChangedBy = userName,D
    ChangedAt = DateTime.UtcNow,D
    Changes = changesD
});D
}D
D
return result;D
}D
D
/// <summary>D
/// AÄ0C0?C,,=C2 2C0CD$@CT=C08DR ACäAD$>Dò=C,,9 EntityFramework State C$> C$=D4BD 5C0=C,,9 CD>CÄ5C0=D'9 C05D
/// </summary>D
private EntityStateChangeEnum MapState(EntityState state) => state switchD
{D
    EntityState.Added => EntityStateChangeEnum.Added,D
    EntityState.Deleted => EntityStateChangeEnum.Deleted,D
    _ => EntityStateChangeEnum.ModifiedD
};D
D
/// <summary>D
/// A 5Ct>C00D =Cä5 Cò>C'Dt5C08CR 8CÄ5C08 D$5C=C'D"5C4> Cò>C'LCt>C$0D$5C'0 C,,7 C=C>C0BCT:D BC 0C$BCä@C,,7C
/// </summary>D
/// <param name="provider">A'>C=C0C'LC0KC' ?D >C$0C"4CT@ D ;D46C „D' 6öçF -æW"'äÄ÷ & Óí
private async Task<string?> GetUserNameInternalAsync(IServiceProvider provider)D
{D
    tryD
    {D
        var userProvider = provider.GetService<IUserProvider>();D
        if (userProvider != null)D
            return await userProvider.GetCurrentUserNameAsync();D
    }D
    catchD
    {D
        // ignoredD
    }D
}D
D
return "System";D
}D
}
</file>

<file path="Data/Interceptors/IUserProvider.cs">
namespace Promatis.Net.Data;D
D
public interface IUserProviderD
{D
    // AÄ5D$>CB <Cä6CTB C KD$L C AC,,=DT@Cä=C0KCÄÄ 5D ;C, ?Cä;D4GCT=C,,5 C,,<CT=C, BD 5C CCTB Ct0C0@CäAC
    Task<string?> GetCurrentUserNameAsync();D
}
</file>

<file path="Data/Promatis.Net.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">D
D
<PropertyGroup>D
    <TargetFramework>net10.0</TargetFramework>D
    <ImplicitUsings>enable</ImplicitUsings>D

```

```

        <Nullable>enable</Nullable>ð
    </PropertyGroup>ð
ð
    <ItemGroup>ð
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />ð
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />ð
        <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.1" />ð
    </ItemGroup>ð
ð
    <ItemGroup>ð
        <ProjectReference Include="..\Domain\Promatis.Net.Domain.csproj" />ð
    </ItemGroup>ð
ð
    "Ä-FVÔw&÷W í
    "™<AssemblyAttribute Include="System.Runtime.CompilerServices.InternalsVisibleTo">ð
    "™Äö & ÖWFW# ä &ö F-2äæWBä Æ-6 F-öäöFVÄâFW7G3Äö & ÖWFW# ä Ä öö CÄO D$5D BCä2Cä3Cä ?D >CT:D$0 -->ð
    "™</AssemblyAttribute>ð
    "Äö-FVÔw&÷W í
ð
ð
</Project>
</file>

<file path="Data/Triggers/Castom/AuditTrigger.cs">
using Microsoft.EntityFrameworkCore;ð
using Microsoft.Extensions.DependencyInjection;ð
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using System.Text.Json;ð
ð
namespace Promatis.Net.Data;ð
ð
public class AuditTrigger(ð
    IServiceScopeFactory scopeFactory, ð
    JsonSerializerOptions jsonOptions)ð
    : IAfterSaveTrigger<DomainObject>ð
{ð
    public async Task HandleAsync(EntityChangedEventArgs<DomainObject> args)ð
    {ð
        // ÄäGC,,IC 5CÄ @C =D$0C"<-D$8Cð >D" 2Cä7CÄ>Cd=Cä9 CD8C00CÄ8Dt5D :Cä9 Cö@Cä:D 8-Cä1Cä;CäGC8 EF Core (
        Type entityType = args.Entity.GetType();ð
        if (entityType.Namespace == "Castle.Proxies" && entityType.BaseType != null)ð
        {ð
            entityType = entityType.BaseType;ð
        }ð
ð
        // 1. BD8C´LD$@: C´>C48D CCT< D$>C´LCð> D$5 D CD"=CäAD$8, Cð>D$>D KCR ?Cä<CTGCT=D² " VF-M
        if (entityType.GetInterfaces().All(i => i.Name != nameof(IAudit)))ð
            return;ð
ð
        // 2. BD>D <C,,@D45CÄ 4C =C0KCR 4C´O JSONð
        object changesToSerialize = args.State switchð
        {ð
            EntityStateChangeEnum.Added => args.Changes.Select(c => newð
            {ð
                c.PropertyName,ð
                NewValue = c.CurrentValueð
            }),ð
ð
            EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted =>
args.Changes.Select(c => newð
            {ð
                c.PropertyName,ð
                OldValue = c.OriginalValue,ð
                NewValue = c.CurrentValueð
            }),ð
ð
            EntityStateChangeEnum.Deleted => new { Message = "Ää1Dð5CðB D44C ;CT= Cö>C´=CäAD$LDâ" öí
ð
            _ => args.Changesð
        };ð
ð
        // 3. B >Ct4C 5CÄ 7C ?C,,ADÄ ;Cä3C
        var auditLog = new AuditLogð
        {ð
            Id = Guid.NewGuid(),ð

```

```

        EntityName = entityType.Name, // A,,!Aô A A´ Aô : Aô8D,,5CÂ GC,,AD$>CR 8CÃO C□;C AD 0 Că@C4AD$@D4:D
        EntityId = args.Entity.Id,ð
        Action = args.State.ToString(),ð
        ChangedAt = args.ChangedAt,ð
        ChangedBy = args.ChangedBy,ð
        ChangesJson = JsonSerializer.Serialize(changesToSerialize, jsonOptions)ð
    };ð
ð
    // 4. A,,!Aô A A´ Aô : A 5Ct>Cô0D =Că5 D >DT@C =CT=C,,5 C" AB GCT@CT7 C$KCD5C´5CÔ=D´9, C40D 0CÔBC,,@Că
    using IServiceScope scope = scopeFactory.CreateScope();ð
ð
    // A,,ICT< DD0C @C,,D2 1C 7Că2Că3Că :Că=D$5C□AD$0 Dt5D 5Cr ?D >C$0C"4CT@ C´>C□0C´LCÔ>C4> Scopeð
    var factory = scope.ServiceProvider.GetService<IDbContextFactory<ApplicationDbContext>>();ð
ð
    if (factory != null)ð
    {ð
        await using ApplicationDbContext context = await factory.CreateDbContextAsync();ð
        context.Set<AuditLog>().Add(auditLog);ð
        await context.SaveChangesAsync();ð
    }ð
    elseð
    {ð
        Console.WriteLine("[AuditTrigger][Error] Aô5 D44C ;CăADÂ =C 9D$8 IDbContextFactory<ApplicationDbContext>
    }ð
    }ð
}
</file>

<file path="Data/Triggers/Global/FluentValidationTrigger.cs">
using FluentValidation;ð
using FluentValidation.Results;ð
using Microsoft.Extensions.DependencyInjection;ð
using Promatis.Net.Domain.Interface;ð
ð
namespace Promatis.Net.Data;ð
ð
public class FluentValidationTrigger(IServiceScopeFactory scopeFactory) // A,,!Aô A A´ Aô : A,,=Cd5C□BC,,@D45CÂ
IServiceProviderð
    : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>ð
{ð
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)ð
    {ð
        Type entityType = args.Entity.GetType();ð
        Type validatorType = typeof(IValidator<>).MakeGenericType(entityType);ð
ð
        // A,,!Aô A A´ Aô : B >Ct4C 5CÂ 8Ct>C´8D >C$0CÔ=D4N, C 5Ct>Cô0D =D4N Că1C´0D BDÂ 2C,,4C,,<CăAD$8 (Scope
        // BôBCă 3C @C =D$8D CCTB, DtBCă •6W'f-6U &÷f-FW" 1D44CTB "Cd8C$KCÂ" 8 D4ACô5D,,=Că @C 7D 5D,,8D" =C H
        using IServiceScope scope = scopeFactory.CreateScope();ð
ð
        // At0Cô@C HC,,2C 5CÂ 2C ;C,,4C BCă@ C,,7 D 2CT6CT3CăÂ 3C @C =D$8D >C$0CÔ=Că 6C,,2Că3Că 66÷ Rô?D >C$0C"4C
        if (scope.ServiceProvider.GetService(validatorType) is IValidator validator)ð
        {ð
            var context = new ValidationContext<object>(args.Entity);ð
            ValidationResult result = await validator.ValidateAsync(context);ð
ð
            if (!result.IsValid)ð
            {ð
                args.Cancel = true;ð
                // BD>D <C,,@D45CÂ :D 0D 8C$>CRÂ GC,,AD$>CR ACă>C ICT=C,,5 Că1 CăHC,,1C□0DR 4C´O UIð
                args.ErrorMessage = string.Join(" | ", result.Errors.Select(e => e.ErrorMessage));ð
            }ð
        }ð
    }ð
}
</file>

<file path="Data/Triggers/Global/ReferenceTreeParentTrigger.cs">
using Microsoft.EntityFrameworkCore;ð
using Microsoft.EntityFrameworkCore.Metadata;ð
using Promatis.Net.Domain.Interface;ð
using System.Reflection;ð
ð
namespace Promatis.Net.Data;ð
ð
/// <summary>ð
/// B4=C,,2CT@D 0C´LCÔKC´ 8CÔDD 0D BD CC□BD4@CÔKC´ BD 8C43CT@ CD;Dò ?D >C$5D :C, FCT;CăAD$=CăAD$8 C,,5D 0D EC,,G

```

```

/// C, AC²2Cä7CÔ>C' 7C IC,,BD² >D" FC,,:C'8Dt5D :C,,E Ct0C$8D 8CÄ>D BCT9 C05D 5CB ACäED 0CÔ5C08CT< C" !B4 AB1
/// </summary>ð
public class ReferenceTreeParentTrigger : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>ð
{ð
    private static readonly MethodInfo DbContextSetMethod = typeof(DbContext)ð
        .GetMethods(BindingFlags.Public | BindingFlags.Instance)ð
        .First(m => m.Name == nameof(DbContext.Set) && m.IsGenericMethod &&
m.GetParameters().Length == 0);ð
    ð
    private static readonly MethodInfo AsNoTrackingMethod =
typeof(EntityFrameworkQueryableExtensions)ð
        .GetMethods(BindingFlags.Public | BindingFlags.Static)ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.AsNoTracking) &&
m.IsGenericMethod);ð
    ð
    private static readonly MethodInfo IgnoreQueryFiltersMethod =
typeof(EntityFrameworkQueryableExtensions)ð
        .GetMethods(BindingFlags.Public | BindingFlags.Static)ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.IgnoreQueryFilters) &&
m.IsGenericMethod);ð
    ð
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)ð
    {ð
        // A0 Aä!B$ AR AT(AT A,, : Bt5D 5Cr @CTDC'5C²AC,,N C0@Cä2CT@D05CÄÄ 5D BDÄ ;C, C Cä1D²5C²BC AC$>C"AD$2
        // ATAC'8 D 2Cä9D BC$0 C05D" r MD$>D" >C JCT:D" =CR 4CT@CT2CäÄ <D² ?D >D BCä 2D'ECä4C,,<!ð
        PropertyInfo? parentIdProp = args.Entity.GetType().GetProperty("ParentId");ð
        if (parentIdProp == null) return;ð
    ð
        // Bt8D$0CT< Ct=C GCT=C,,0 D 2Cä9D BC" 4C,,=C <C,,GCTAC²8ð
        Guid entityId = args.Entity.Id;ð
        Guid? parentId = (Guid?)parentIdProp.GetValue(args.Entity);ð
    ð
        PropertyInfo? deletedAtProp = args.Entity.GetType().GetProperty("DeletedAt");ð
    ð
        // 1. At0D"8D$0 CäB C0@D0<Cä3Cä AC <CäFC,,BC,,@Cä2C =C,,0ð
        if (parentId.HasValue && parentId == entityId)ð
        {ð
            args.Cancel = true;ð
            args.ErrorMessage = "Aä1D²5C²B C05 CÄ>Cd5D" 1D'BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.";ð
            return;ð
        }ð
    ð
        if (parentId.HasValue && parentId.Value != Guid.Empty)ð
        {ð
            // A,,ICT< D >CD8D$5C'0 C" 6† ævUG& 6¶W" AD 5CD8 C'NC KDR >C JCT:D$>C"Ä C C²²D$>D KDR ACä2C00CD0CT
            object? localParent = args.Context.ChangeTracker.Entries()ð
                .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
parentId.Value)?.Entity;ð
            ð
            bool exists;ð
            bool isDeleted;ð
        ð
            if (localParent != null)ð
            {ð
                exists = true;ð
                var localDeletedAt = (DateTime?)deletedAtProp?.GetValue(localParent);ð
                isDeleted = localDeletedAt != null;ð
            }ð
            elseð
            {ð
                // A$0D,,0 Cä@C,,3C,,=C ;DÄ=C 0 D 5DD;CT:D 8D0 4C'0 C$KDt8D ;CT=C,,0 C²²D =D0 8 Query C" Tb 6÷&]
                IEntityType? entityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());ð
                Type rootClrType = entityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();ð
            ð
                object? rawSet =
DbContextSetMethod.MakeGenericMethod(rootClrType).Invoke(args.Context, null);ð
                object? noTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(rootClrType).Invoke(null, [rawSet]);ð
                object? ignoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(rootClrType).Invoke(null, [noTrackingQuery]);ð
            ð
                IQueryable<object> query = ((IQueryable)ignoreFiltersQuery!).Cast<object>();ð
            ð
                // A,,ICT< Cä1D²5C²B C0> CD8C00CÄ8Dt5D :Cä<D2 AC$>C"AD$2D2 $-B" !B4 AM

```

```

        object? dbParent = await query.FirstOrDefaultAsync(x => EF.Property<Guid>(x, "Id")
== parentId.Value);
        exists = dbParent != null;
        var dbDeletedAt = dbParent != null ? (DateTime?)deletedAtProp?.GetValue(dbParent) :
        null;
        isDeleted = dbDeletedAt != null;
    }
    // 2. A0@C2CT@C00 DD8ct8Dt5D :C3C3 AD4ICTAD$2C2C =C,,0 D >CD8D$5C'LD :C3C3 CCT;C
    if (!exists)
    {
        args.Cancel = true;
        args.ErrorMessage = $"B4:C 7C =C0KC' @C4C,,BCT;DÄAC08C' >C JCT:D" ,,"C¢ . &VçD-G0' =CR =C 9CD
        return;
    }
    // 3. A0@C2CT@C00 D BC BD4AC <D03C0>C4> D44C ;CT=C,,0
    if (isDeleted)
    {
        args.Cancel = true;
        args.ErrorMessage = "A05C'LCt0 C00Ct=C GC,,BDÄ @C4C,,BCT;CT< D44C ;CT=C0KC' >C JCT:D"â#½
        return;
    }
    // 4. A4 B4 Aä A / At B" B$ Aä" Bd A0 Aä (A -> B -> C -> A)
    Guid? currentCheckId = parentId;
    var visitedIds = new HashSet<Guid>();
    while (currentCheckId.HasValue && currentCheckId.Value != Guid.Empty)
    {
        if (currentCheckId == entityId)
        {
            args.Cancel = true;
            args.ErrorMessage = "Bd8C0;C,,GCTAC00D0 7C 2C,,AC,,<CäAD$L: C05C'LCt0 C05D 5CÄ5D BC,,BDÄ @C4
            return;
        }
        if (!visitedIds.Add(currentCheckId.Value))
            break;
        // A,,ICT< D0;CT<CT=D" FCT?CäGC08 D =C GC ;C 2 ChangeTracker
        object? nextLocalElement = args.Context.ChangeTracker.Entries()
            .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
currentCheckId.Value)?.Entity;
        if (nextLocalElement != null)
        {
            currentCheckId = (Guid?)parentIdProp.GetValue(nextLocalElement);
        }
        else
        {
            Guid? id = currentCheckId;
            IEntityType? loopEntityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
            Type loopRootClrType = loopEntityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();
            object? loopRawSet =
DbContextSetMethod.MakeGenericMethod(loopRootClrType).Invoke(args.Context, null);
            object? loopNoTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopRawSet]);
            object? loopIgnoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopNoTrackingQuery]);
            IQueryable<object> loopQuery =
((IQueryable)loopIgnoreFiltersQuery!).Cast<object>();
            object? parentNodeObj = await loopQuery.FirstOrDefaultAsync(x =>
EF.Property<Guid>(x, "Id") == id.Value);
            currentCheckId = parentNodeObj != null ?
(Guid?)parentIdProp.GetValue(parentNodeObj) : null;
        }
    }

```



```

    }
}
</file>

<file path="Data/TriggerService/DatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using System.Collections.Concurrent;
{
namespace Promatis.Net.Data;
{
/// <summary>
/// B 5C ;C,,7C FC,,0 Dd5C0BD 0C'LC0>C4> CD8D ?CTBDt5D 0 D$@C,,3C45D >C"â #C0@C 2C'0CTB D 5C48D BD 0Dd8CT9 CD>CÄ
/// C, :Cä>D 4C,,=C,,@D45D" DC 7D² 2C ;C,,4C FC,,8 (CD> D >DT@C =CT=C,,0) C, CC$5CD>CÄ;CT=C,,0 (C0>D ;CR ACäED 0C05
/// </summary>
/// <param name="serviceProvider">A :D$CC ;DÄ=D'9 Cä>C0BCT:D B Ct0C$8D 8CÄ>D BCT9 (DI Container) D$5CäCD"5C4>
public class DatabaseTriggerService(IServiceProvider serviceProvider) : IDatabaseTriggerService
{
/// <summary>
/// A4;Cä1C ;DÄ=Cä5 D >C KD$8CRÄ CC$5CD>CÄ;D0ND"5CR AC,,AD$5CÄC Cä1 D4AC05D,,=Cä< Cä>Cä<C,,BCR AD4IC0>D BC,
/// A,,AC0>C'LCtCCTBD 0 CD;D0 @CT0CäBC,,2C0>C4> Cä1C0>C$;CT=C,,0 C,,=D$5D DCT9D 0 (MudBlazor) C,,;C, ACä2Cä7C0
/// </summary>
public static event Action<EntityStateChangeEnum, object>? OnEntityCommitted;
{
/// <summary>
/// B 5CTAD$@ C0>CD?C,,Adt8Cä>C"Ä 2D' ?Cä;C00DäIC,,ED 0 AD D >DT@C =CT=C,,0 C,,7CÄ5C05C08C' ,, $C 7C C ;C,,4C
/// AD5C'5C40D" ?D 8C08CÄ0CTB C @C4CCÄ5C0BD² ACä1D'BC,,0 C, 0CäBD40C'LC0KC' 66÷ VB 6W'f-6U &÷f-FW"1
/// </summary>
private static readonly Dictionary<Type, List<Func<EntityCancelArgsBase, IServiceProvider,
Task>>> BeforeSubscribers = new();
{
/// <summary>
/// B 5CTAD$@ C0>CD?C,,Adt8Cä>C"Ä 2D' ?Cä;C00DäIC,,ED 0 A0 B AR CD ?CTHC0>C4> D >DT@C =CT=C,,0 C,,7CÄ5C05C08C
/// </summary>
private static readonly Dictionary<Type, List<Func<EntityChangedArgsBase, IServiceProvider,
Task>>> AfterSubscribers = new();
{
/// <summary>
/// A0>D$>Cä>C 5Ct>C00D =D'9 CäMD, 8CT@C @DT8C, BC,,?Cä2. A,,ACä;DäGC 5D" ?Cä2D$>D =Cä5 C,,AC0>C'LCt>C$0C08C
/// C0@C, >C0@CT4CT;CT=C,,8 C 0Ct>C$KDR :C'0D ACä2 C, 8C0BCT@DD5C"ACä2 Cä1D 0C 0D$KC$0Ct<D'E D CD"=CäAD$5C
/// </summary>
private static readonly ConcurrentDictionary<Type, List<Type>> HierarchyCache = new();
{
/// <summary>
/// B 5C48D BD 8D CCTB CD>CÄ5C0=D'9 D$@C,,3C45D 4C'0 Cä>C0:D 5D$=Cä3Cä BC,,?C AD4IC0>D BC,i
/// A$KctKC$0CtBD 0 C 2D$>CÄ0D$8Dt5D :C, ?D 8 D :C =C,,@Cä2C =C,,8 D 1Cä@Cä: C$> C$@CT<D0 8C08Dd8C ;C,,7C FC
/// </summary>
/// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C' AD4IC0>D BC,äÄ÷G- W & Ó1
/// <typeparam name="TTrigger">B$8C0 :C'0D AC 0BD 8C43CT@C Ä @CT0C'8CtCDäICT3Cä ;Cä3C,,D2 >C @C 1CäBCä8.<
public void Register<TEntity, TTrigger>()
{
where TEntity : class
where TTrigger : class
{
Type entityType = typeof(TEntity);
Type triggerType = typeof(TTrigger);
{
// A0@Cä2CT@D05CÄÄ :C :C,,5 C,,=D$5D DCT9D K Cd8Ct=CT=C0>C4> Dd8Cä;C @CT0C'8CtCCTB CD0C0=D'9 D$@C,,3C45
bool isBefore = typeof(IBeforeSaveTrigger<TEntity>).IsAssignableFrom(triggerType);
bool isAfter = typeof(IAfterSaveTrigger<TEntity>).IsAssignableFrom(triggerType);
{
if (isBefore)
{
// BD>D <C,,@D45CÄ >D$;Cä6CT=C0CDä ;D0<C 4D2ä Cä=D$5C"=CT@ 'sp' C05D 5CD0CTBD 0 C" <Cä<CT=D" 2D'7
// DtBCä ?D 5CD>D$2D 0D"0CTB D4BCTGCäC CD>C'3Cä6C,,2D4IC,,E Cä1Dä5CäBCä2 (Captive Dependency).
AddSubscriber(BeforeSubscribers, entityType, async (argsBase, sp) =>
{
// B 0Ct@CTHC 5CÄ MCä7CT<C0;D0@ D$@C,,3C45D 0 C,,7 C :D$CC ;DÄ=Cä3Cä 66÷ R BCT:D4ICT3Cä 7C ?D >
if (sp.GetRequiredService<TTrigger>() is IBeforeSaveTrigger<TEntity> handler)
{
// Aä1Cä@C GC,,2C 5CÄ 1C 7Cä2D'5 C @C4CCÄ5C0BD² 2 D BD >C4> D$8C08Ct8D >C$0C0=D'9 Cä>C0BCT
var typedArgs = new EntityCancelEventArgs<TEntity>(
(TEntity)argsBase.Entity,
argsBase.State,
argsBase.Changes,
argsBase.Context);

```

```

        await handler.HandleAsync(typedArgs);
    }

    // A$>Ct2D 0D"0CT< D 5CtCC´LD$0D$K C$KC0>C´=CT=C,,0 D$@C,,3C45D 0 Că1D 0D$=Că 2 C 0Ct>C$KC'
    argsBase.Cancel = typedArgs.Cancel;
    argsBase.ErrorMessage = typedArgs.ErrorMessage;
    argsBase.Handled = typedArgs.Handled;
}

});
}

if (isAfter)
{
    AddSubscriber(AfterSubscribers, entityType, async (argsBase, sp) =>
    {
        if (sp.GetRequiredService<TTrigger>() is IAfterSaveTrigger<TEntity> handler)
        {
            var typedArgs = new EntityChangedEventArgs<TEntity>(
                (TEntity)argsBase.Entity,
                argsBase.State,
                argsBase.Changes,
                argsBase.ChangedBy,
                argsBase.ChangedAt);

            await handler.HandleAsync(typedArgs);
            argsBase.Handled = typedArgs.Handled;
        }
    });
}

// ATAC´8 Că;C AD =CR @CT0C´8CtCCTB C08 Că4C,,= Că>C0BD 0CăB – D0BCă :D 8D$8Dt5D :C 0 CăHC,,1Că0 Că>C0
if (!isBefore && !isAfter)
{
    throw new InvalidOperationException(
        $"Aă;C AD .G&-vvW%G– Răæ 0W0 =CR @CT0C´8CtCCTB IBeforeSaveTrigger C,,;C, " gFW%6 fUG&-vvW" 4C
    );
}

/// <summary>
/// A$KC0>C´=D05D" ACă2Că7C0CDă 2C ;C,,4C FC,,N C,,7Că5C00CT<Că9 D CD´=CăAD$8 C05D 5CB 5E >D$?D 0C$:Că9 C"
/// Aă?D 0D,,8C$0CTB C$ADă FCT?CăGCăC C00D ;CT4Că2C =C,,0 D CD´=CăAD$8. A0@CT@D´2C 5D" >C05D 0Dd8Dă ?D 8 C0
/// </summary>
public async Task ValidateAsync(object entity, EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes, DbContext context)
{
    var args = new EntityCancelArgsBase(entity, state, changes, context);

    // Aă1DT>CD8Că 8CT@C @DT8Dă BC,,?Că2 (D 0Că>C´ AD4IC0>D BC,Ā 1C 7Că2D´E Că;C AD >C" 8 C,,=D$5D DCT9D >C
    foreach (Type type in GetTypesHierarchy(entity.GetType()))
    {
        if (BeforeSubscribers.TryGetValue(type, out List<Func<EntityCancelArgsBase,
            IServiceProvider, Task>>> actions))
        {
            foreach (Func<EntityCancelArgsBase, IServiceProvider, Task> action in actions)
            {
                // A05D 5CD0CT< C´>Că0C´LC0KC´ 6W'f-6U &÷f-FW"Ā A Că>D$>D KCĀ 1D´; D >Ct4C = CD0C0=D´9 D0
                await action(args, serviceProvider);
            }
        }
    }

    // ATAC´8 D$@C,,3C45D 2D´AD$0C$8C² DC´0C2 6 æ6VĀ r =CT<CT4C´5C0=Că ?D 5D KC$0CT< D$@C =Ct
    if (args.Cancel) throw new OperationCanceledException(args.ErrorMessage);
}

}

}

/// <summary>
/// B 0D AD´;C 5D" CC$5CD>Că;CT=C,,0 Că1 D4AC05D,,=Că< D >DT@C =CT=C,,8 C,,7Că5C05C08C´ 2 C 0CtC CD0C0=D´E.D
/// A0>CD4CT@Cd8C$0CTB Că0D :C 4C0KC´ ?CT@CTEC$0D" ,,DC´0C2 † æFÆVB´ 8 C0CC ;C,,;D45D" ACă1D´BC,,5 C" 3C´>C
/// </summary>
public async Task NotifyAsync(object entity, EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes, string? user, DateTime at)
{
    var args = new EntityChangedEventArgsBase(entity, state, changes, user, at);

    foreach (Type type in GetTypesHierarchy(entity.GetType()))
    {

```

```

        if (AfterSubscribers.TryGetValue(type, out List<Func<EntityChangedEventArgsBase,
IServiceProvider, Task>>? actions))&
        {&
            foreach (Func<EntityChangedEventArgsBase, IServiceProvider, Task> action in actions)&
            {&
                await action(args, serviceProvider);&
            }&
        }&
        // ATAC`8 Că4C,,= C,,7 D$@C,,3C45D >C" 2D`AD$0C$8C² † æFÆVB Ò G'VRÂ >C @C 1CăBC=0 Dd5Cô>Dt:C
        if (args.Handled) return;&
    }&
}

// AôCC ;C,,:C FC,,0 C" AD$0D$8Dt5D :C,,9 Dô2CT=D" 4C`O D 5C :D$8C$=Că3Că >C =Că2C`5Cô8Dò :Că<Cô>Cô5CôBC
OnEntityCommitted?.Invoke(state, entity);&
}

/// <summary>&
/// A$ACô>CĂ>C40D$5C`LCôKC' <CTBCă4 CD;Dò 1CT7Că?C ACô>C4> CD>C 0C$;CT=C,,0 Cô>CD?C,,ADt8C= >C" 2 D ;Că2C @C
/// </summary>&
private void AddSubscriber<TArgs>(Dictionary<Type, List<Func<TArgs, IServiceProvider, Task>>>
dict, Type type, Func<TArgs, IServiceProvider, Task> action)&
{&
    if (!dict.TryGetValue(type, out List<Func<TArgs, IServiceProvider, Task>>? list))&
    {&
        list = new List<Func<TArgs, IServiceProvider, Task>>();&
        dict[type] = list;&
    }&
    list.Add(action);&
}

/// <summary>&
/// A,,7C$;CT:C 5D" ?Că;CôCDă 8CT@C @DT8Dă BC,,?Că2 CD;Dò CC=0Ct0Cô=Că3Că >C JCT:D$0 (C$:C`NDt0Dò 1C 7Că2D`
/// B 5CtCC`LD$0D$K C=MD,,8D CDăBD O CD;Dò >C 5D ?CTGCT=C,,0 C$KD >C= >C' ?D >C,,7C$>CD8D$5C`LCô>D BC, ?Că4 C
/// </summary>&
private IEnumerable<Type> GetTypesHierarchy(Type type) =>&
    HierarchyCache.GetOrAdd(type, t => {&
        var types = new List<Type>();&
        // B 5C=CD AC,,2Cô> Cô>CD=C,,<C 5CĂADò 2C$5D E Cô> CD@CT2D2 =C AC`5CD>C$0Cô8Dò :C`0D ACă2, C,,AC=;Dă
        for (Type? c = t; c != null && c != typeof(object); c = c.BaseType) types.Add(c);&
        // AD>C 0C$;Dô5CĂ 2D 5 D 5C ;C,,7Că2C =CôKCR 8CôBCT@DD5C"AD² 8 D41C,,@C 5CĂ 4D41C`8C=0D$Kô
        return types.Concat(t.GetInterfaces()).Distinct().ToList();&
    });&

/// <summary>&
/// B @CăGCôKC' AC @CăA D BC BC,,GCTAC=8DR @CT3C,,AD$@C FC,,9. &
/// A,,ACô>C`LCtCCTBD O Cô@CT8CĂCD"5D BC$5Cô=Că 2 Unit-D$5D BC E CD;Dò 8Ct>C`0Dd8C, ?D >C4>Cô>C" BCTAD$>C"
/// </summary>&
internal static void ClearInternalRegistrations()&
{&
    BeforeSubscribers.Clear();&
    AfterSubscribers.Clear();&
    HierarchyCache.Clear();&
}

}
</file>

<file path="Data/TriggerService/EntityStateChangeEnum.cs">
namespace Promatis.Net.Data;&
&
public enum EntityStateChangeEnum&
{&
    Added,&
    Modified,&
    Deleted,&
    SoftDeleted&
}
</file>

<file path="Data/TriggerService/IDatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;&
&
namespace Promatis.Net.Data;&
&
public interface IDatabaseTriggerService&
{&

```

```

// AA5D$>CB 4C'0 C00D BD >C":C, AC$0Ct5C' $!D4IC0>D BDÂ0"D 8C43CT@"D
void Register<TEntity, TTrigger>()D
    where TEntity : classD
    where TTrigger : class;D
D
// AA5D$>CDK C$KC0>C'=CT=C,,0 (C$KctKC$0DâBD 0 C,,=D$5D FCT?D$>D >CÂ•
Task ValidateAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo>
changes, DbContext context);D
Task NotifyAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo> changes,
string? user, DateTime at);D
}
</file>

<file path="Data/TriggerService/PropertyChangeInfo.cs">
namespace Promatis.Net.Data;D
D
public class PropertyChangeInfoD
{D
    public string PropertyName { get; set; } = string.Empty;D
    public object? OriginalValue { get; set; }D
    public object? CurrentValue { get; set; }D
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs">
using Microsoft.EntityFrameworkCore;D
D
namespace Promatis.Net.Data;D
D
public class EntityCancelArgsBase(D
    object entity,D
    EntityStateChangeEnum state,D
    List<PropertyChangeInfo> changes,D
    DbContext context) : EntityEventArgsBase(entity, state, changes)D
{D
    public DbContext Context { get; } = context;D
    public bool Cancel { get; set; }D
    public string? ErrorMessage { get; set; }D
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs">
using Microsoft.EntityFrameworkCore;D
D
namespace Promatis.Net.Data;D
D
public class EntityCancelEventArgs<T>(D
    T entity,D
    EntityStateChangeEnum state,D
    List<PropertyChangeInfo> changes,D
    DbContext context)D
    : EntityCancelArgsBase(entity!, state, changes, context) where T : classD
{D
    public new T Entity => (T)base.Entity;D
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgsBase.cs">
namespace Promatis.Net.Data;D
D
// A 0Ct>C$KC' :C'0D A CD;D0 2C0CD$@CT=C05C4> C,,AC0>C'LCt>C$0C08D0 2 D 5D 2C,,AC]
public class EntityChangedEventArgsBase(D
    object entity,D
    EntityStateChangeEnum state,D
    List<PropertyChangeInfo> changes,D
    string? changedBy,D
    DateTime changedAt) : EntityEventArgsBase(entity, state, changes) // <-- A00D ;CT4D45CÂ 7CD5D L0
{D
    public string? ChangedBy { get; } = changedBy;D
    public DateTime ChangedAt { get; } = changedAt;D
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data;D
D

```

```

// A$0D, BC,,?C,,7C,,@Cä2C =CÔKC' :C´0D A CD;Dò BD 8C43CT@Cä2D
public class EntityChangedEventArgs<T>()
    T entity, EntityStateChangeEnum state, ()
    List<PropertyChangeInfo> changes, ()
    string? changedBy, ()
    DateTime changedAt)()
    : EntityChangedEventArgsBase(entity!, state, changes, changedBy, changedAt) where T : class()
{()
    public new T Entity => (T)base.Entity;()
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs">
namespace Promatis.Net.Data;()
()
public abstract class EntityEventArgsBase()
    object entity,()
    EntityStateChangeEnum state,()
    List<PropertyChangeInfo> changes) : EventArgs()
{()
    public object Entity { get; } = entity;()
    public EntityStateChangeEnum State { get; } = state;()
    public List<PropertyChangeInfo> Changes { get; } = changes;()
    public bool Handled { get; set; }()
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs">
namespace Promatis.Net.Data;()
()
// AD;Dò ?D 0C$8C² $ Aä!A´ " D >DT@C =CT=C,,0 (D42CT4Cä<C´5CÔ8Dò0HC,,=C •
public interface IAfterSaveTrigger<T> where T : class()
{()
    Task HandleAsync(EntityChangedEventArgs<T> args);()
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs">
namespace Promatis.Net.Data;()
()
// AD;Dò ?D 0C$8C² $ Aä" ACäED 0CÔ5CÔ8Dò ,,2C ;C,,4C FC,,0)()
public interface IBeforeSaveTrigger<T> where T : class()
{()
    Task HandleAsync(EntityCancelEventArgs<T> args);()
}
</file>

<file path="Data/Validation/GlobalPolymorphicValidator.cs">
using FluentValidation;()
using FluentValidation.Results;()
using Microsoft.Extensions.DependencyInjection;()
()
namespace Promatis.Net.Domain;()
()
public class GlobalPolymorphicValidator<T> : AbstractValidator<T> where T : class()
{()
    private readonly IServiceProvider _serviceProvider;()
()
    public GlobalPolymorphicValidator(IServiceProvider serviceProvider)()
    {()
        _serviceProvider = serviceProvider ?? throw new
ArgumentNullException(nameof(serviceProvider));()
    }()
()
    protected override bool PreValidate(ValidationContext<T> context, ValidationResult result)()
    {()
        if (context.InstanceToValidate == null) return true;()
()
        // 1. A0>C´Cd0CT< D 5C ;DÄ=D´9 D$8Cò >C JCT:D$0 C" @C =D$0C"<CR ,,=C ?D 8CÄ5D Â FW 'FÖVçEVæ-B 8C´8 A
Type realType = context.InstanceToValidate.GetType();()
()
        // ATAC´8 D$8Cò ACä2C00CD0CTB D B ,,B.CRâ MD$> C>CÔ5Dt=D´9 C>;C AD Â 0 CÔ5 C 0Ct>C$KC' 0C AD$@C :D$=
// D$> C,,AC=0D$L CD>Dt5D =C,,9 C$0C´8CD0D$>D =CR =D46CÔ>, DtBCä1D² =CR CC"BC, 2 C 5D :Cä=CTGCÔCDâ @CT
if (realType == typeof(T)) return true;()
()
        // 2. B BD >C,,< D$8Cò 8CÔBCT@DD5C"AC 4C´0 D$>Dt5Dt=Cä3Cä 2C ;C,,4C BCä@C ,,=C ?D 8CÄ5D Â •f Æ-F F÷#ÄF

```

```

        Type specificValidatorType = typeof(IValidator<>).MakeGenericType(realType);
    }

    // 3. At0C0@C HC,,2C 5CÂ 8Cr D' B C$0C'8CD0D$>D K CD;D0 MD$>C4> C@>C0:D 5D$=Cä3Câ BC,,?C
    // B0BCâ 2C 6C0>, D$0C¢ :C : CD;D0 >CD=Cä3Câ BC,,?C <Cä6CTB C KD$L Ct0D 5C48D BD 8D >C$0C0> C05D :Cä;
    List<IValidator> specificValidators =
        _serviceProvider.GetServices(specificValidatorType).Cast<IValidator>().ToList();
    }

    if (specificValidators.Any())
    {
        // B >Ct4C 5CÂ =CR04Cd5C05D 8C¢ :Cä=D$5C¢AD" 2C ;C,,4C FC,,8 FluentValidation CD;D0 ?CT@CT4C GC, 4C
        ValidationContext<object>? newContext =
            ValidationContext<object>.CreateWithOptions(context.InstanceToValidate, options =>
            {
                // A,,!A0 A A' A0 : A@0C0>C08Dt=D'9 CÄ5D$>CB fÇVVçEf Æ-F F-0â 4C'0 C0@D0<Cä9 C0@Cä1D >D :C, A
                if (context.Selector != null)
                {
                    options.UseCustomSelector(context.Selector);
                }
            });
    }

    // At0C0CD :C 5CÂ :C 6CDKC' =C 9CD5C0=D'9 D$>Dt5Dt=D'9 C$0C'8CD0D$>D =C AC'5CD=C,,:C
    foreach (IValidator validator in specificValidators)
    {
        // BtBCä1D² 8Ct1CT6C BDÂ 7C FC,,:C'8C$0C08D0Â ?D >C$5D OCT<, DtBCâ =C 9CD5C0=D'9 C$0C'8CD0D$>D
        if (validator.GetType().IsGenericType &&
            validator.GetType().GetGenericTypeDefinition() == typeof(GlobalPolymorphicValidator<>))
        {
            continue;
        }

        ValidationResult specificResult = validator.Validate(newContext);

        // A05D 5C0>D 8CÂ 2D 5 Cä1C00D CCd5C0=D'5 CäHC,,1C@8 C" >C IC,,9 D 5CtCC'LD$0D" 2C ;C,,4C FC,,8 D
        foreach (ValidationFailure error in specificResult.Errors)
        {
            result.Errors.Add(error);
        }
    }

    }

    return true;
}
}
</file>

<file path="Domain.Interface/Enums/EnumExtensions.cs">
using System.ComponentModel;
using System.Reflection;
{
namespace Promatis.Net.Domain.Interface;
{
public static class EnumExtensions
{
    /// <summary>
    /// A$>Ct2D 0D"0CTB Ct=C GCT=C,,5 C BD 8C CD$0 [Description] CD;D0 ;Dä1Cä3Câ MC'5CÄ5C0BC ?CT@CTGC,,AC'5C08
    /// ATAC'8 C BD 8C CD" >D$AD4BD BC$CCTB, C$>Ct2D 0D"0CTB D BC =CD0D BC0>CR 8CÄ0 .ToString().
    /// </summary>
    public static string GetDescription(this Enum value)
    {
        FieldInfo? field = value.GetType().GetField(value.ToString());
        if (field == null) return value.ToString();

        var attribute = field.GetCustomAttribute<DescriptionAttribute>();
        return attribute != null ? attribute.Description : value.ToString();
    }
}
}
</file>

<file path="Domain.Interface/Interfaces/IAudit.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// B CD"=CäAD$8, D 5C ;C,,7D4ND"8CR MD$>D" 8C0BCT@DD5C"A, C CCDCD" 7C ?C,,AD'2C BDÄAD0 2 AuditLog0
    /// </summary>
    public interface IAudit
    {
        }
    }
}

```

</file>

```
<file path="Domain.Interface/Interfaces/IDomainObject.cs">
namespace Promatis.Net.Domain.Interface;
{
    public interface IDomainObject
    {
        DateTime CreatedAt { get; init; }
    }
}</file>
```

```
<file path="Domain.Interface/Interfaces/IDomainObjectHasKey.cs">
namespace Promatis.Net.Domain.Interface;
{
    public interface IDomainObjectHasKey<TKey> : IDomainObject
    {
        TKey Id { get; set; }
    }
}</file>
```

```
<file path="Domain.Interface/Interfaces/IEntityCloner.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// A,,=D$5D DCT9D ?C'0D$DCä@CÄ5C0=Cä3Cä 4C$8Cd:C 8Ct>C'8D >C$0C0=Cä3Cä :C'>C08D >C$0C08D0 4Cä<CT=C0KDR AD4I
    /// </summary>
    public interface IEntityCloner
    {
        T CloneEntity<T>(T entity) where T : class;
    }
}</file>
```

```
<file path="Domain.Interface/Interfaces/ISoftDeletable.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// A,,=D$5D DCT9D 4C'0 CÄ0C4:Cä3Cä CCD0C'5C08D0 >C JCT:D$>C-
    /// </summary>
    public interface ISoftDeletable
    {
        DateTime? DeletedAt { get; set; }
        string? DeletedBy { get; set; }
    }
}</file>
```

```
<file path="Domain.Interface/Interfaces/ITreeNode.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// A4;Cä1C ;DÄ=D'9 C0;C BDD>D <CT=C0KC' :Cä=D$@C :D" 4C'0 C$ACTE C,,5D 0D EC,,GCTAC=8DR ,,4D 5C$>C$8CD=D'E) D C
    /// </summary>
    /// <typeparam name="T">B$8C0 4Cä<CT=C0>C' <Cä4CT;C,äÄ÷G- W & Óí
    public interface ITreeNode<T> : IDomainObjectHasKey<Guid> where T : class
    {
        /// <summary>
        /// A,,4CT=D$8DD8C=0D$>D @Cä4C,,BCT;DÄAC= >C4> D47C'0. B 0C$5C0 çVÆÄ 4C'0 C= >D =CT2D'E D0;CT<CT=D$>C"í
        /// A,,!Aô A A' A0 : AD>C 0C$;CT= set CD;D0 2Cä7CÄ>Cd=CäAD$8 C,,7CÄ5C05C08D0 AC$0Ct5C' ,,?CT@CT<CTICT=C,,O C
        /// </summary>
        Guid? ParentId { get; set; }

        /// <summary>
        /// A00C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä AD KC':C, =C @Cä4C,,BCT;DÄAC=8C' >C JCT:D"í
        /// </summary>
        T? Parent { get; set; }

        /// <summary>
        /// A= >C';CT:Dd8D0 4CäGCT@C08DR MC'5CÄ5C0BCä2 (C0>CDGC,,=CT=C0KDR Cct;Cä2).D
        /// </summary>
        ICollection<T> Children { get; }
    }
}</file>
```

```
<file path="Domain.Interface/Promatis.Net.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">
```

```

    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
</Project>
</file>

<file path="Domain/AuditLog/AuditLog.cs">
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// A CCD8D" AC,,AD$5CÄK
    /// </summary>
    public class AuditLog : DomainObject
    {
        /// <summary>
        /// A00C,,<CT=Cä2C =C,,5 D >C KD$8Dý
        /// </summary>
        public string EntityName { get; init; } = string.Empty;

        /// <summary>
        /// A,,4CT=D$8DD8C¬0D$>D ACä1D´BC,,0
        /// </summary>
        public Guid EntityId { get; init; }

        /// <summary>
        /// AD5C"AD$2C,,5
        /// </summary>
        public string Action { get; init; } = string.Empty;

        /// <summary>
        /// AD0D$0 C,,7CÄ5C05C08Dý
        /// </summary>
        public DateTime ChangedAt { get; init; }

        /// <summary>
        /// A,,7CÄ5C05C0>
        /// </summary>
        public string? ChangedBy { get; init; }

        /// <summary>
        /// B BD >C¬0 C,,7CÄ5C05C08C•
        /// </summary>
        public string ChangesJson { get; init; } = string.Empty;
    }
</file>

<file path="Domain/DomainObject/DomainObject.cs">
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// AäACÖ>C$=Cä9 C¬;C AD „uT"B ?Cä CCÄ>C´GC =C,,N)
    /// </summary>
    public abstract class DomainObject : DomainObjectBase<Guid>
    {
        protected DomainObject()
        {
            Id = Guid.NewGuid();
        }
    }
</file>

<file path="Domain/DomainObject/DomainObjectBase.cs">
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// B4=C,,2CT@D 0C´LCÖKC´ 1C 7Cä2D´9 C¬;C AD A Cö>CD4CT@Cd:Cä9 C´NC >C4> D$8C00 C¬;DäGC
    /// </summary>
    /// <typeparam name="TKey">B$8C0 :C´NDt0</typeparam>
    public abstract class DomainObjectBase<TKey> : IDomainObjectHasKey<TKey>
    {

```



```

    public TKey Id { get; set; } = default!;
}
public DateTime CreatedAt { get; init; } = DateTime.UtcNow;
}
</file>

<file path="Domain/DomainObject/DomainObjectValidator.cs">
using FluentValidation;
namespace Promatis.Net.Domain;
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 2C ;C,,4C BCä@ CD;Dò 2D 5DR :C'0D ACä2, C00D ;CT4D45CÄKDR >D" FöÖ -äö&!V7M
/// </summary>
public abstract class DomainObjectValidator<T> : AbstractValidator<T> where T : DomainObject
{
    protected DomainObjectValidator()
    {
        RuleFor(x => x.Id)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D >C JCT:D$0 C05 CÄ>Cd5D" 1D'BDÂ ?D4AD$KCÂâ""½
        RuleFor(x => x.CreatedAt)
            .Must(date => date <= DateTime.UtcNow.AddSeconds(1))
            .WithMessage("AD0D$0 D >Ct4C =C,,0 C05 CÄ>Cd5D" 1D'BDÂ 2 C CCDCD"5CÂâ""½
    }
}
</file>

<file path="Domain/EntityCloner/JsonEntityCloner.cs">
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Text.Json.Serialization.Metadata;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Domain;
public class JsonEntityCloner : IEntityCloner
{
    private static readonly JsonSerializerOptions JsonOptions = new()
    {
        ReferenceHandler = ReferenceHandler.IgnoreCycles,
        TypeInfoResolver = new DefaultJsonTypeInfoResolver
        {
            Modifiers = { typeInfo =>
                foreach (var property in typeInfo.Properties)
                {
                    // 1. A$KD 5Ct0CT< Dd8C=;C,,GCTAC=8CR =C 2C,,3C FC,,>C0=D'5 D 2Cä9D BC$0 C,,5D 0D EC,,9D
                    if (property.Name is "Parent" or "Children")
                    {
                        property.ShouldSerialize = (_, _) => false;
                        continue;
                    }
                    // 2. A$KD 5Ct0CT< D ;Cä6C0KCR AC$0Ct0C0=D'5 CD>CÄ5C0=D'5 Cä1D=5C=BD² 1D0:CT=CD0 C, :Cä;C
                    Type propType = property.PropertyType;
                    bool isDomainClass = typeof(Domain.DomainObject).IsAssignableFrom(propType);
                    bool isDomainCollection = propType.IsGenericType &&
                        typeof(System.Collections.IEnumerable).IsAssignableFrom(propType) &&
                        typeof(Domain.DomainObject).IsAssignableFrom(propType.GetGenericArguments().FirstOrDefault());
                    if (isDomainClass || isDomainCollection)
                    {
                        property.ShouldSerialize = (_, _) => false;
                    }
                }
            }
        }
    };
    public T CloneEntity<T>(T entity) where T : class
    {
        if (entity == null) throw new ArgumentNullException(nameof(entity));
    }
}

```

```

        // B 5D 8C ;C,,7D45CÂ 8 CD5D 5D 8C ;C,,7D45CÂÂ AD$@Cä3Cä ACäED 0C00Dò @CT0C'LC0KC' @C =D$0C"<-D$8Cò =C
        string json = JsonSerializer.Serialize(entity, entity.GetType(), JsonSerializerOptions);
        return (T)JsonSerializer.Deserialize(json, entity.GetType(), JsonSerializerOptions);
    }
}
</file>

<file path="Domain/Promatis.Net.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="FluentValidation" Version="12.1.1" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Domain/ReferenceBase/ReferenceBase.cs">
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Domain;
/// <summary>
/// A 0Ct>C$KC' :C'0D A CD;Dò 2D 5DR ACò@C 2CäGC08Cæ>C-
/// </summary>
public abstract class ReferenceBase : DomainObject, ISoftDeletable
{
    public string Name { get; set; } = string.Empty;
    public string? Description { get; set; }
    public string? Code { get; set; }
    public DateTime? DeletedAt { get; set; }
    public string? DeletedBy { get; set; }
}
</file>

<file path="Domain/ReferenceBase/ReferenceBaseValidator.cs">
using FluentValidation;
using FluentValidation.Results;
namespace Promatis.Net.Domain;
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 2C ;C,,4C BCä@ CD;Dò 2D 5DR :C'0D ACä2, C00D ;CT4D45CÄKDR >D" &VfW&Væ6T& 6]
/// </summary>
public abstract class ReferenceBaseValidator<T> : DomainObjectValidator<T> where T : ReferenceBase
{
    /// <summary>
    /// A$8D BD40C'LC0KC' ?C BD$5D = D 5C4CC'0D =Cä3Cä 2D'@C 6CT=C,,0 CD;Dò ?D >C$5D :C, Cä4C í
    /// Aò> D4<Cä;Dt0C08Dä¢ AD$@Cä3Cä ;C BC,,=C,,FC 2 C$5D EC05CÄ @CT3C,,AD$@CRÄ FC,,DD K C, òí
    /// </summary>
    protected virtual string CodeRegexPattern => @"^[A-Z0-9_]*$";
    /// <summary>
    /// A$8D BD40C'LC0KC'>CR ACä>C ICT=C,,5 Cä1 CäHC,,1Cæ5 C$0C'8CD0Dd8C, @CT3D4;Dô@C0>C4> C$KD 0Cd5C08Dò Cä4C í
    /// </summary>
    protected virtual string CodeValidationMessage => "Aæ>CB <Cä6CTB D >CD5D 6C BDÄ BCä;DÄ:Cä ;C BC,,=C,,FD2 2
    protected ReferenceBaseValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty().WithMessage("A00C,,<CT=Cä2C =C,,5 Cä1D07C BCT;DÄ=Cä 4C'0 Ct0C0>C'=CT=C,,0")
            // Aò@Cä2CT@Cæ0 C00 C0@Cä1CT;D² 2 C00Dt0C'5 C, :Cä=Dd5D
    }
}

```

```

        .Must(name => name == null || name.Trim() == name)
        .WithMessage("A00C,,<CT=Cä2C =C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄAD0 8C´8 Ct0C=0C0GC,,2C BDÄAD0 ?D >C 5C´
        .MinimumLength(2).WithMessage("A00C,,<CT=Cä2C =C,,5 CD>C´6C0> D >CD5D 6C BDÄ <C,,=C,,<D4< 2 D 8CÄ2Cä;
        .MaximumLength(250).WithMessage("A0@CT2D´HCT=C <C :D 8CÄ0C´LC00D0 4C´8C00 C00C,,<CT=Cä2C =C,,0 (25
D
        RuleFor(x => x.Code)
        .Matches(x => CodeRegexPattern)
        .When(x => !string.IsNullOrEmpty(x.Code))
        .WithMessage(x => CodeValidationMessage);
    }
D
    public Func<object, string, Task<IEnumerable<string>>> ValidateValue => async (model,
propertyName) =>
    {
        ValidationResult? result = await
ValidateAsync(ValidationContext<T>.CreateWithOptions((T)model, x =>
x.IncludeProperties(propertyName)));
        return result.IsValid ? [] : result.Errors.Select(e => e.ErrorMessage);
    };
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBase.cs">
using Promatis.Net.Domain.Interface;
D
namespace Promatis.Net.Domain;
D
/// <summary>D
/// A 0Ct>C$KC´ :C´0D A CD;D0 2D 5DR AC0@C 2CäGC08C= >C" ACä4CT@Cd0D"8DR 4CT@CT2DÄ0D
/// </summary>D
/// <summary>D
/// A 0Ct>C$KC´ :C´0D A CD;D0 2D 5DR 4D 5C$>C$8CD=D´E D ?D 0C$>Dt=C,,;Cä2 (MDM).D
/// A,,AC0>C´LCtCCTB C00D$BCT@C0 5%E 4C´O C05D 5CD0Dt8 D BD >C4> D$8C08Ct8D >C$0C0=D´E C00C$8C40Dd8Cä=C0KDR A
/// </summary>D
/// <typeparam name="T">B$8C0 :Cä=C=CTBC0>C4> CD>CÄ5C0=Cä3Cä >C JCT:D$0 (C00D ;CT4C08C=0).</typeparam>D
public abstract class ReferenceTreeBase<T> : ReferenceBase, ITreeNode<T> where T : class
{
    /// <summary>D
    /// A,,4CT=D$8DD8C=0D$>D @Cä4C,,BCT;DÄAC= >C4> D47C´0 C" !B4 ABi
    /// </summary>D
    public Guid? ParentId { get; set; }
D
    /// <summary>D
    /// A00C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä AD KC´:C, =C @Cä4C,,BCT;DÄAC=8C´ >C JCT:D" 2 Cä?CT@C BC,,2C0>C´ ?C <
    /// A05D 5Cä?D 5CD5C´OCTBD 0 C=0C f-'GV Ä 2 C= >C05Dt=D´E C=;C AD 0DR 4C´O C0>CD4CT@Cd:C, Æ S´ Æ0 F-æri
    /// </summary>D
    public virtual T? Parent { get; set; }
D
    /// <summary>D
    /// A= >C´;CT:Dd8D0 4CäGCT@C08DR MC´5CÄ5C0BCä2 (C0>CDGC,,=CT=C0KDR Cct;Cä2).D
    /// </summary>D
    public virtual ICollection<T> Children { get; set; } = new List<T>();
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain.Interface;
D
namespace Promatis.Net.Domain;
D
/// <summary>D
/// B4=C,,2CT@D 0C´LC0KC´ 2C ;C,,4C BCä@ CD;D0 2D 5DR :C´0D ACä2, C00D ;CT4D45CÄKDR >D" &Vfw&Væ6UG&VT& 6]
/// </summary>D
public abstract class ReferenceTreeBaseValidator<T> : ReferenceBaseValidator<T>
where T : ReferenceBase, ITreeNode<T>
{
    protected ReferenceTreeBaseValidator() : base()
    {
        // A0@Cä2CT@C=0 C00 D 0CÄ>Dd8D$8D >C$0C08CR ,,>C JCT:D" =CR <Cä6CTB D4:C 7D´2C BDÄ =C ACT1D0 :C : C00
        RuleFor(x => x.ParentId)
        .Must((model, parentId) => parentId != model.Id)
        .WithMessage("Aä1D=5C=B C05 CÄ>Cd5D" 1D´BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
D
        // A 0Ct>C$0D0 ?D >C$5D :C DCä@CÄ0D$0 GUID0
        RuleFor(x => x.ParentId)

```

```

        .NotEqual(Guid.Empty)
        .When(x => x.ParentId.HasValue)
        .WithMessage("B >CD8D$5C`LD :C,,9 C,,4CT=D$8DD8C=0D$>D =CR <Cä6CTB C KD$L CöCD BD´< GUID");
    }
}
</file>

<file path="Promatis.Net.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using MudBlazor
@using Promatis.Net.UI.Components.Workspaces
@using Promatis.Net.Domain.Interface
@using Promatis.Net.UI.Components.Dialogs
</file>

<file path="Promatis.Net.UI/Components/ActionContext/DataContext.cs">
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Service;

namespace Promatis.Net.UI.Components;

/// <summary>
/// A 1D BD 0C=BC0>CR OCD@Câ @C 1CäBD² ACâ AC=2Cä7C0KCÄ8 C0>D$>C=0CÄ8 CD0C0=D´E CD>CÄ5C0=Cä9 D CD´=CäAD$8.
/// A=Ca@CD8C08D CCTB D 0C >D$C D4=C,,2CT@D 0C´LC0>C4> A @Cä:CT@C 8 Aä B20:D0HC Ä >D$2CTGC O Ct0 C0>C´8CÄ>D
/// </summary>
public abstract class DataContext<TEntity, TQueryState, TResultData> : WorkspaceContext
{
    where TEntity : class, new()
    {
        private bool _isLoading;

        /// <summary>
        /// B4=C,,2CT@D 0C´LC0KC´ 4C,,AC05D$GCT@ C=0C02CT9CT@C 4C =C0KDR BCT:D4ICT3Cä MC@C =C ,,~äÖV÷´´ ò 6W´fW"
        /// </summary>
        public IUiDataBroker<TEntity, TQueryState, TResultData> Broker { get; }

        /// <summary>
        /// A$KD >C=C0Cä8Ct2Cä4C,,BCT;DÄ=D´9 Aä B20:D0H Cd8C$KDR >C JCT:D$>C" 4Cä<CT=C í
        /// </summary>
        public IUiOzuCache<TEntity> OzuCache { get; }

        /// <summary>
        /// Bt5D BC0>CR ?Cä;C,,<Cä@DD=Cä5 C05D 5Cä?D 5CD5C´5C08CR AC$>C"AD$2C 1C 7Cä2Cä9 C45Cä<CTBD 8C, ECä;D BC
        /// A00D AC,,2C0KC´ :C @C=0D v÷&·7 6U vR =C BC,,2C0> D GC,,BC 5D" 5C4> D >D BCä0C08CR GCT@CT7 C=0D :C 4 C
        /// </summary>
        public override bool IsLoading => _isLoading;

        /// <summary>
        /// AT A,, B´ Aä"A= B´"B´ A0#A´,B %Aä B "A AD A0 B´%.
        /// B 8C4=C ;C,,7C,,@D45D" > A´.A +BR 8Ct<CT=CT=C,,0DR ACäAD$>D0=C,,0 (Ct0C4@D47C=0, D$@C,,3C45D K B #A , CÄC
        /// Aä1Cä1D"5C0=C O D BD 0C08Dd0 D04D 0 (ReferencePageBase) C0>CD?C,,HCTBD O C00 C05C4> Dd5C0BD 0C´8Ct>C$0
        /// </summary>
        public event Action? OnContextUpdated;

        /// <summary>
        /// B4=C,,2CT@D 0C´LC0KC´ <CTBCä4 C$Kct>C$0 C0CC´LD 0 Cä1C0>C$;CT=C,,0 CD;Dò 2D 5DR =C,,6CTAD$>D0IC,,E C0>D$>
        /// </summary>
        public void NotifyContextUpdated() => OnContextUpdated?.Invoke();

        protected DataContext(IServiceProvider serviceProvider, bool isInMemoryMode) : base()
        {
            if (serviceProvider == null) throw new ArgumentNullException(nameof(serviceProvider));

            // A,,7C$;CT:C 5CÄ G& ç6-VçB0:Cä<C0>C05C0BD² 0C$BCä=Cä<C0>C4> D BCT9D"04C$8Cd:C 8Cr D´ ACTAD 8C•
            OzuCache = serviceProvider.GetRequiredService<IUiOzuCache<TEntity>>();
            Broker = serviceProvider.GetRequiredService<IUiDataBroker<TEntity, TQueryState,
TResultData>>();

            // A=C0DC,,3D4@C,,@D45CÄ D >C=5D ?D OCÄ> C" :Cä=D BD CC=BCä@CR r ;C,,C$8CD8D CCTB C4>C0:C, ACäAD$>D0
            if (isInMemoryMode)
            {
                Broker.ConfigureInMemoryMode(OzuCache, LoadInitialDataForBrokerAsync,
EvaluateDataStateInMemory);
            }
            else

```

```

        {
            Broker.ConfigureServerMode(FetchDataFromServerAsync);
        }
    }

    /// <summary>
    /// AT A, A / A, #A A$ B !A BÄ A / B$ Bt A A B B AD A B' % AD B B BÄ B' % UI-B$ A A, & A, AT AT B
    /// A 5Ct>C00D =Câ >C >D 0Dt8C$0CTB C$KC0>C'=CT=C,,5 A @Cä:CT@C 2 C,,=CD8C=0Dd8Dâ 7C 3D CCt:C, 8 C LCTB C"
    /// </summary>
    public virtual async Task<TResultData> GetDataAsync(TQueryState state, CancellationToken ct =
default)
    {
        if (_isLoading) return default!;

        try
        {
            _isLoading = true;
            NotifyContextUpdated(); // A,,<C0CC'LD  C C;DäGC 5CÄ xVD÷fW&Æ ' =C MC@C =CR GCT@CT7 CT4C,,=Cä5

            return await Broker.FetchDataAsync(state, ct);
        }
        finally
        {
            _isLoading = false;
            NotifyContextUpdated(); // A,,<C0CC'LD #C D':C'NDt0CT< MudOverlay C00 D0:D 0C05 Dt5D 5Cr 5CD8C0>
        }
    }

    // --- Aä B0 A "AT BÄ B' BD#A0 Bd Aä A BÄ B' AÄ B "B² A' / A0 B$ AÄ Aä B0 B ---
    protected abstract Task<List<TEntity>> LoadInitialDataForBrokerAsync(TQueryState state,
CancellationToken ct);
    protected abstract TResultData EvaluateDataStateInMemory(TQueryState state,
IReadOnlyList<TEntity> inMemoryList);
    protected abstract Task<TResultData> FetchDataFromServerAsync(TQueryState state,
CancellationToken ct);
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/EntityContext.cs">
using Microsoft.Extensions.DependencyInjection;
using MudBlazor;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.UI.Components;

/// <summary>
/// A 1D BD 0C=BC0>CR OCD@Câ @C 1CäBD² ACâ AD$5C"BCä< C= >C0:D 5D$=Cä3Câ MC=7CT<C0;D0@C AD4IC0>D BC, 2 C00CÄ0
/// AÄ>C0>C0>C'LC0> CäBC$5Dt0CTB Ct0 DT@C =CT=C,,5 C$KCD5C'5C0=Cä9 Ct0C08D 8 (SelectedData) C, 8Ct>C'8D >C$0C0
/// A05D 5CD0CTB 3 generic-C00D 0CÄ5D$@C =C 2CT@DR 2 DataContext CD;D0 AC=2Cä7C0>C' AC,,=DT@Cä=C,,7C FC,,8 A @C
/// </summary>
public abstract class EntityContext<TEntity, TKey, TQueryState, TResultData>(
IServiceProvider serviceProvider,
bool isInMemoryMode) : DataContext<TEntity, TQueryState, TResultData>(serviceProvider,
isInMemoryMode)
{
    where TEntity : class, new()
    where TKey : notnull
    {
        private TEntity? _selectedData;
        private TEntity? _draftData;

        /// <summary>
        /// B$5C=CD"0D0 2D'4CT;CT=C00D0 ?Cä;DÄ7Cä2C BCT;CT< Ct0C08D L C" T' ,,2 C4@C,,4CR 8C'8 CD5D 5C$5).
        /// A0@C, 8Ct<CT=C,,8 C45C05D 8D CCTB CT4C,,=D'9 C,,<C0CC'LD æ÷F-g"6öçFW†EW F FVB,' 4C'O D 5C :D$8C$=Cä3
        /// </summary>
        public TEntity? SelectedData
        {
            get => _selectedData;
            set
            {
                if (_selectedData != value)
                {
                    _selectedData = value;
                    NotifyContextUpdated(); // A08C00CTB CT4C,,=D'9 event Cä1C0>C$;CT=C,,0 CD;D0 BD4;C 0D 0 C, AD$@
                }
            }
        }
    }
}

```

```

D
/// <summary>D
/// A,,7Cä;C,,@Cä2C =CÔKC' GCT@CÔ>C$8C¢ „3C'CC >C¤8C' :C'>CÒ' AD4ICÔ>D BC, 4C' O Cô@Dô<Cä9 Cô@C,,2Dô7C¤8 C¢ ?
/// Aô0C'8Dt8CR >C JCT:D$0 C" MD$>CÄ AC'>D$5 C 2D$>CÄ0D$8Dt5D :C, AC,,3CÔ0C'8Ct8D CCTB C 0Ct>C$>C' AD$@C =
/// </summary>D
public TEntity? DraftDataD
{D
    get => _draftData;D
    setD
    {D
        if (_draftData != value)D
        {D
            _draftData = value;D
            NotifyContextUpdated(); // Aô8CÔ0CTB CT4C,,=Cä5 D >C KD$8CR >C =Cä2C'5CÔ8Dò 4C' O CäBC¤@D'BC,,0/
        }D
    }D
}D
D
/// <summary>D
/// B ;D46CT1CÔKC' ECT;Cô5D OCD@C 4C' O C,,7C$;CTGCT=C,,O D BD >C4> D$8Cô8Ct8D >C$0CÔ=Cä3Cä ?CT@C$8Dt=Cä3C
/// Aô@CT4CäAD$0C$;Dô5D" CCÔ8C$5D AC ;DÄ=D'9 C,,=D BD CCÄ5CÔB CD;Dò @C 1CäBD² @CT?Cä7C,,BCä@C,,5C" !B4 AB +
/// </summary>D
public TKey GetEntityKey(TEntity entity)D
{D
    if (entity is IDomainObjectHasKey<TKey> hasKeyEntity)D
    {D
        return hasKeyEntity.Id;D
    }D
}D
D
throw new InvalidOperationException(D
    $"A¤@C,,BC,,GCTAC¤8C' AC >C' 4Cä<CT=C ¢ !D4ICÔ>D BDÄ w·G— Vöb...DVçF—G''äæ ÖWÖr =CR @CT0C'8CtCCTB Cä1
)D
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/GridContext.cs">
using MudBlazor;D
D
namespace Promatis.Net.UI.Components;D
D
/// <summary>D
/// Aô@Cä<CT6D4BCäGCÔKC' :Cä=D$5C¤AD" ?D 5CDAD$0C$;CT=C,,O CD;Dò BC 1C'8Dt=D'E D BD 0CÔ8Db „w&-B ÖöFR'í
/// AÄ0Cô8D" 0C AD$@C :D$=D'5 CD0CÔ=D'5 Dô4D 0 C" AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =CÔKCR AD$@D4:D$CD K Cô0C48CÔ0Dd8C
/// </summary>D
public abstract class GridContext<TEntity, TKey> : EntityContext<TEntity, TKey, GridState<TEntity>,
GridData<TEntity>>D
    where TEntity : class, new()D
    where TKey : notnullD
{D
    protected GridContext(IServiceProvider serviceProvider, bool isInMemoryMode)D
        : base(serviceProvider, isInMemoryMode)D
    {D
    }D
}D
D
/// <summary>D
/// AD ÄÄ AÔ B' BD A',B$ AD Bò A !A' AD A,, Ää .D
/// Aô5D 5Cä?D 5CD5C'0CTBD O C¤>CÔ:D 5D$=D'< D ?D 0C$>Dt=C,,:Cä< CD;Dò 6CTAD$:Cä3Cä >D$ACTGCT=C,,O D BD >C¢
/// A Ar @C,,AC¤0 CÔ0D CD,,8D$L Cô>D$>C¤>C 5Ct>Cô0D =CäAD$L Ää B2Ô:DÔHC 8 A Ar :Cä?C,,?C AD$0 D CD$8CÔK C
/// </summary>D
protected virtual Func<TEntity, bool>? GetDomainFilter() => null;D
D
/// <summary>D
/// A,,!B$ AÔ A / Ää"A$ B$!B$ AT AÔ B "BÄ "A A' Bd+: ÄÄ0D$5CÄ0D$8Dt5D :C,,9 C¤>CÔ2CT9CT@ Cä1D 0C >D$:C, 4C
/// Aô@C,,=C,,<C 5D" 1CT7Cä?C ACÔKC' •&V Döæç"Æ-7BÄ 8Ct>C'8D >C$0CÔ=D'9 A @Cä:CT@Cä< C$=D4BD 8 Cô>D$>C¤>C 5
/// </summary>D
protected override GridData<TEntity> EvaluateDataStateInMemory(GridState<TEntity> state,
IReadOnlyList<TEntity> inMemoryList)D
{D
    // 1. B =C GC ;C =C BC,,2CÔ> Cô@C,,<CT=Dô5CÄ 4Cä<CT=CÔKC' DC,,;DÄBD =C AC'5CD=C,,:C Ä 5D ;C, >CÔ 7C 4C
    var domainFilter = GetDomainFilter();D
    IEnumerable<TEntity> filteredList = domainFilter != nullD
        ? inMemoryList.Where(domainFilter)D
        : inMemoryList;D
}D
D
// Aô@CT2D 0D"0CT< CäBDD8C'LD$@Cä2C =CÔKC' ?CäBCä: C" Ä"ä Ô2D'@C 6CT=C,,5 CD;Dò ×VD&Æ ¦÷" :Cä;Cä=Cä:D
IQueryable<TEntity> query = filteredList.AsQueryable();D

```

```

    }
    // 2. AD8C00CÄ8Dt5D :C O DD8C´LD$@C FC,,O Cα>C´>CÔ>Cϕ AC <Cä3Cä 8CÔBCT@DD5C"AC ×VD&Æ |÷-
    if (state.FilterDefinitions != null && state.FilterDefinitions.Any())
    {
        query = query.Where(state.FilterDefinitions);
    }

    // 3. AD8C00CÄ8Dt5D :C O D >D BC,,@Cä2Cα0 Cα>C´>CÔ>Cϕ AC <Cä3Cä 8CÔBCT@DD5C"AC ×VD&Æ |÷-
    if (state.SortDefinitions != null && state.SortDefinitions.Any())
    {
        query = query.OrderBy(state.SortDefinitions);
    }

    // AÄ B$ AÄ B$ Bt B A,, A,, A$ B A B#ϕ $C,,:D 8D CCT< D$>Dt=D´9 D$>D$0C² B$A,, BÄ"B A$ AÔ B´% D BD
    int totalCount = query.Count();

    // 4. A$KD 5Ct0CT< DD@CT9CÄ :Cä=Cα@CTBCÔ>C´ AD$@C =C,,FD² ,, C 3C,,=C FC,,O MudBlazor)
    List<TEntity> pagedItems = query
        .Skip(state.Page * state.PageSize)
        .Take(state.PageSize)
        .ToList();

    return new GridData<TEntity>
    {
        Items = pagedItems,
        TotalItems = totalCount
    };
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/IWorkspaceContext.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// Aα>D =CT2Cä9 C,,=D$5D DCT9D ?D4;DÄBC CCô@C 2C´5CÔ8Dò ?D >D BD 0CÔAD$2CT=CÔ>C´ 3CT>CÄ5D$@C,,5C´ ECä;D BC
    /// </summary>
    public interface IWorkspaceContext
    {
        /// <summary>
        /// A4;Cä1C ;DÄ=D´9 DD;C 3 C,,=CD8Cα0Dd8C, 7C 3D CCT:C,â D 8 Ct=C GCT=C,,8 true Cä1Cä1D"5CÔ=D´9
        /// Cα0D :C A WorkspacePage C ;Cä:C,,@D45D" MCα@C = C, 2Cα;DäGC 5D" :D CD$8C´:D2 ×VD÷fW&Æ ´1
        /// </summary>
        public bool IsLoading { get; }

        // --- Aô B AÄ B$ B² !B$ A´ At Bd A, A4 Aä AT"B A, R Aä BT A´!B$ B "B AÔ Bd+ ---
        int PaperElevation { get; }
        string PaperClass { get; }
        string WorkspaceHeight { get; }
        string TopZoneHeight { get; }
        string BottomZoneHeight { get; }
        string LeftZoneWidth { get; }
        string RightZoneWidth { get; }

        // --- B A B$ A$ B´ BD A A, !BT Aä B´ A A,,/ Aô Aô A´ A´ ,, C´5C"7Cä@ CäBD ;CT4C,,B CÄCD$0Dd8C, AC <) -
        bool IsTopZoneCollapsed { get; set; }
        bool IsBottomZoneCollapsed { get; set; }
        bool IsLeftZoneCollapsed { get; set; }
        bool IsRightZoneCollapsed { get; set; }
    }
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/ReferenceContext.cs">
using Microsoft.Extensions.DependencyInjection;
using MudBlazor;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Promatis.Net.UI.Controls;
{
    namespace Promatis.Net.UI.Components;
    {
        /// <summary>
        /// A 8Ct=CTA-A 3D 5C40D$>D 4C´O C$ACTE Cô;CäACα8DR ACô@C 2CäGCÔ8Cα>C" B D 8D BCT<D²1
        /// BD8CαAC,,@D45D" BC,,?D² 4C =CÔKDRÄ 2Cα;DäGC 5D" -äÖVö÷´´ ÖöFR 8 C 2D$>CÄ0D$8Dt5D :C, ACä1C,,@C 5D" BC,,?Cä2Cä
        /// </summary>
        public abstract class ReferenceContext<TEntity> : GridContext<TEntity, Guid>
    }
}

```

```

where TEntity : ReferenceBase, new()
{
    private readonly IReferenceService<TEntity> _referenceService;
    private readonly Lock _controlsLock = new();
    private readonly List<IUIControl> _controls = [];
    private bool _isToolbarInitialized;

    /// <summary>
    /// A05D 5Cä?D 5CD5C'5C08CR AC$>C"AD$2C 6öçG&öÇ2â Cä7C$@C IC 5D" AD$@Cä3Cä CCô>D OCD>Dt5CÔ=D4N C¤>C';CT
    /// C,,=D$5D 0C¤BC,,2C0KDR 5%TB0:C0>C0>C¢ ?C'>D :Cä3Cä AC0@C 2CäGC08C¤0 A0!A,i
    /// </summary>
    public IEnumerable<IUIControl> Controls
    {
        get
        {
            lock (_controlsLock)
            {
                if (!_isToolbarInitialized)
                {
                    InitializeContext(); // A'5C08C$> D >C 8D 0CT< D$CC'1C @ C¤=Cä?Cä: C0@C, ?CT@C$>CÄ @CT=CD
                    _isToolbarInitialized = true;
                }
                return _controls.ToArray(); // A,,7Cä;C,,@Cä2C =C0KC' AC00C0HCäBD
            }
        }
    }

    protected ReferenceContext(IServiceProvider serviceProvider, bool isInMemoryMode = true)
        : base(serviceProvider, isInMemoryMode) // A0> D4<Cä;Dt0C08Dä AC0@C 2CäGC08C¤8 A0!A, 2D 5C44C :D0HC,,
    {
        if (serviceProvider == null) throw new ArgumentNullException(nameof(serviceProvider));

        // A$=CT4D OCT< C 0Ct>C$KC' 5%TB0ACT@C$8D @CT?Cä7C,,BCä@C,,0 B #A C,,7 DI D 5D AC,,8D
        _referenceService = serviceProvider.GetRequiredService<IReferenceService<TEntity>>();

        // A,,7C$;CT:C 5CÄ 8 Cd5D BC¤> D 2D07D'2C 5CÄ At#-D BD 0D$5C48Dä <D4BC FC,,9 C0;CäAC¤8DR AC0@C 2CäGC08
        var strategy = serviceProvider.GetRequiredService<IOzuMutationStrategy<TEntity>>();
        OzuCache.SetMutationStrategy(strategy);
    }

    /// <summary>
    /// A$ B4"B A0 B0/ B Aä A¤ B$#A' A A B .D
    /// A00C0>C' =D05D" HC,,=D2 CC0@C 2C'5C08D0 AD$0C04C @D$=D'<C, CCÄ=D'<C, 5%TB0:C0>C0:C <C, OCD@C
    /// C, 1CT7Cä?C AC0> C05D 5CD0CTB D4?D 0C$;CT=C,,5 C0@C,,:C'0CD=D'< C00D ;CT4C08C¤0CÄ 2C0CD$@C, 7C IC,,ICT=C
    /// </summary>
    private void InitializeContext()
    {
        lock (_controlsLock)
        {
            _controls.Add(new CreateEntityButton<TEntity>());
            _controls.Add(new EditEntityButton<TEntity>());
            _controls.Add(new DeleteEntityButton<TEntity>());

            // A00D$8C$=Cä 2D'7D'2C 5CÄ 1CT7Cä?C AC0CDä BCäGC¤C D 0D HC,,@CT=C,,0 CD;D0 :Cä=C¤@CTBC0KDR AC0@C 2
            AddInitializeContext();
        }
    }

    /// <summary>
    /// A0 A,, A' AD A / B$ Bt A A !B,, B A0 B0 "B4 A B .D
    /// A¤>C0:D 5D$=D'9 D ?D 0C$>Dt=C,,: C05D 5Cä?D 5CD5C'OCTB D0BCäB CÄ5D$>CBÄ GD$>C K C 5Ct>C00D =Cä 4Cä1C 2
    /// D 2Cä8 D ?CTFC,,DC,,GC0KCR DC,,;DÄBD K, D 5C'5C¤BCä@D² 8C'8 C¤=Cä?C¤8 CD5C"AD$2C,,9 (Cä=C, 2D BC =D4B C0@
    /// </summary>
    protected virtual void AddInitializeContext()
    {
        // A0> D4<Cä;Dt0C08Dä ?D4AD$>C' AC0@C 2CäGC08C¢ =CR 4Cä1C 2C'OCTB C¤0D BCä<C0KDR MC'5CÄ5C0BCä2D
    }

    // --- AÄ A0 A0 A',A0+A' A$"Aä A " CRUD-A¤ A0 AT AT A B BR !A0 A Aä'A0 A¤ A" A' B$Aä AÄ+ ---D

    /// <summary>
    /// BD8C00C'LC0KC' AC¤2Cä7C0>C' :Cä=C$5C"5D DC,,:D 0Dd8C, 8Ct<CT=CT=C,,9 C0@Cä2CT@CT=C0>Cä> Dt5D =Cä2C,,:C
    /// A$KctKC$0CTBD 0 Cä1Cä1D"5C0=D'< D04D >CÄ AD$@C =C,,FD² ...&VfW&Væ6U vT& 6R' ?CäAC'5 D4AC05D,,=Cä3Cä ?D >
    /// </summary>
    public async Task CommitActionAsync()
    {

```



```

        if (DraftData == null) return;
    }

    // A,,7C$;CT:C 5CÂ wV-BÔ:C´NDR GCT@CÔ>C$8C=0 CD>CÄ5CÔ0 Dt5D 5Cr ECT;Cô5D 1C 7Cä2Cä3Cä 0CD@C
    Guid currentKey = GetEntityKey(DraftData);

    // ATAC´8 C=;DäG CöCD BCä9 (Guid.Empty) – DÔBCâ =C BC,,2CÔKC´ ?D 8Ct=C : B At A A,,/ CÔ>C$>C´ 7C ?C,,A
    bool isNewEntity = currentKey == Guid.Empty;

    if (isNewEntity)
    {
        // A$KctKC$0CT< C$AD$0C$:D2 2 PostgreSQL Dt5D 5Cr u% 2ô ´ ACT@C$8D 8CÔDD 0D BD CC=BD4@D½
        await _referenceService.AddAsync(DraftData);
    }
    else
    {
        // A$KctKC$0CT< C,,7CÄ5CÔ5CÔ8CR 2 PostgreSQL Dt5D 5Cr u% 2ô ´ ACT@C$8D 8CÔDD 0D BD CC=BD4@D½
        await _referenceService.UpdateAsync(DraftData);
    }

    // Aô>C´=CäAD$LDâ >Dt8D´0CT< C CDD5D <D4BC FC,,8 C" ?C <DôBC•
    DraftData = default;

    // A LCT< C" 5CD8CÔKC´ :Cä;Cä:Cä; – A @Cä:CT@ Cô@C,,<CT=C,,B CD5C´LD$C C¢ At#, C, &Æ ¦÷" =C BC,,2CÔ> Cä
    NotifyContextUpdated();
}

/// <summary>
/// B4=C,,2CT@D 0C´LCÔKC´ :Cä=C$5C"5D 0D 8CÔED >CÔ=Cä3Cä CCD0C´5CÔ8Dò 7C ?C,,AC, 8Cr ÷7Fw&U5 Â 4C´O D ?D
/// A$KctKC$0CTBD 0 Cò0D AC,,2CÔ>C´ :CÔ>Cô:Cä9-C= >CÄ0CÔ4Cä9 D$CC´1C @C „FVÆWFTVçF–G"´WGFôâ´1
/// </summary>
public virtual async Task ExecuteDeleteActionAsync(TEntity? targetEntity)
{
    if (targetEntity == null) return;

    Guid targetKey = GetEntityKey(targetEntity);

    // BD8Ct8Dt5D :C, 8C´8 CÄOC4:Cä AD$8D 0CT< Ct0Cô8D L C,,7 PostgreSQL Dt5D 5Cr ACT@C$8D 8CÔDD 0D BD CC
    await _referenceService.DeleteAsync(targetKey);

    // B 8CÔED >CÔ8Ct8D CCT< Aä B2Ô:DÔH C, ?C,,=C 5CÂ T´Ô?CäBCä:D
    NotifyContextUpdated();
}

// --- B A A,, A &A,,/ A B "B A= "Aô+BR !AT"AT B´% B$ A B Aä B$ B´% Aä B "Aä A¢ !B4 AB ÔÖÝ

/// <summary>
/// Bt5D BCÔKC´ ACT@C$5D =D´9 D$@C =D ?Cä@D"â D´7D´2C 5D$ADò D >C=5D >CÄÂ 5D ;C, ACô@C 2CäGCô8C¢ ?CT@CT
/// </summary>
protected override async Task<GridData<TEntity>> FetchDataFromServerAsync(GridState<TEntity>
state, CancellationToken ct)
{
    var pagedResult = await _referenceService.GetPagedAsync(state.Page, state.PageSize, ct);

    return new GridData<TEntity>
    {
        Items = pagedResult.Items,
        TotalItems = pagedResult.TotalCount
    };
}

/// <summary>
/// A´5CÔ8C$KC´ ?CäAD$0C$IC,,: D KD KDR 4C =CÔKDR 4C´O Cô5D 2Cä=C GC ;DÄ=Cä3Cä =C ?Cä;CÔ5CÔ8Dò D >C=5D >C
/// </summary>
protected override async Task<List<TEntity>> LoadInitialDataForBrokerAsync(GridState<TEntity>
state, CancellationToken ct)
{
    try
    {
        return await _referenceService.GetAllAsync(ct) ?? [];
    }
    catch (OperationCanceledException)
    {
        return [];
    }
}

```

```

// --- B4 B  A$ AT A,,  A  A´ AT Bd AT  A  AÔ"B  A´ A"  A´/ Aô B$ AÄ Aä  ---Ð
Ð
protected void AddControl(IUicontrol control)Ð
{Ð
    if (control == null) throw new ArgumentNullException(nameof(control));Ð
    lock (_controlsLock) _controls.Add(control);Ð
    NotifyContextUpdated();Ð
}Ð
Ð
protected void RemoveControl(string controlId)Ð
{Ð
    if (string.IsNullOrEmpty(controlId)) return;Ð
    lock (_controlsLock) _controls.RemoveAll(c => c.Id == controlId);Ð
    NotifyContextUpdated();Ð
}Ð
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/ReferenceTreeContext.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.UI.Controls;Ð
Ð
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// A 8Ct=CTA-A 3D 5C40D$>D  4C´O C$ACTE CD@CT2Cä2C,,4CÔKDR ,,8CT@C @DT8Dt5D :C,,E) D ?D 0C$>Dt=C,,:Cä2 AÔ!A, AC,,
/// BD8C  AC,,@D45D" BC,,?D² 4C =CÔKDRÂ =C AD$@C 8C$0CTB D 5C  CD AC,,2CÔKC´  At#-C  MD, 8 C 2D$>CÄ0D$8Dt5D :C, ACä
/// </summary>Ð
public abstract class ReferenceTreeContext<TEntity> : TreeContext<TEntity, Guid>Ð
    where TEntity : ReferenceTreeBase<TEntity>, new()Ð
{Ð
    // A,,!Aô A  A´ AÔ : A  CÔAD$@D4:D$>D  AD$0C² :D 8D BC ;DÄ=Cä GC,,AD$KCÄâ  C AD$@Cä9C  D BD 0D$5C48C´ <D4B
    // C, 2D´7Cä2D² 2C,,@D$CC ;DÄ=D´E CÄ5D$>CD>C" >D$ADä4C  ?Cä;CÔ>D BDÄN D44C ;CT=D²
    protected ReferenceTreeContext(Ð
        IServiceProvider serviceProvider,Ð
        Action? onDataChangedNotifier = null)Ð
        : base(serviceProvider, isInMemoryMode: true, onDataChangedNotifier)Ð
    {Ð
    }Ð
    Ð
    /// <summary>Ð
    /// A,,!Aô A  A´ AÔ : A 5Ct>Cô0D =D´9 CÄ5D$>CB 6C,,7CÔ5CÔ=Cä3Cä FC,,:C´0 C  CÔBCT:D BC í
    /// A 2D$>CÄ0D$8Dt5D :C, 2D´7D´2C 5D$ADò 1C 7Cä2D´< DT>C´AD$>CÄ ...v÷&·7 6U  vR´ 2 D >C KD$8C, öä-æ-F- Æ-!
    /// C40D 0CÔBC,,@D40 Ct0D"8D$C C ;C AD >C"Ô=C AC´5CD=C,,:Cä2 CäB NullReferenceException.Ð
    /// </summary>Ð
    public override void InitializeContext()Ð
    {Ð
        PopulateDefaultTreeToolbar();Ð
    }Ð
    Ð
    /// <summary>Ð
    /// A 2D$>CÄ0D$8Dt5D :C 0 D 1Cä@C  Cô0CÔ5C´8 D4?D 0C$;CT=C,,0 CD;Dò 4CT@CT2C í
    /// </summary>Ð
    protected virtual void PopulateDefaultTreeToolbar()Ð
    {Ð
        // 1. A  Cä?C  D >Ct4C =C,,0 C  D =CT2Cä3Cä MC´5CÄ5CÔBC ,,1CT7 D >CD8D$5C´0)Ð
        AddControl(new CreateEntityButton<TEntity> { Title = "AD>C 0C$8D$L C  D 5CÔL" } Ð
            .OnExecute(ExecuteCreateRecordAsync));Ð
        Ð
        // 2. A  Cä?C  D >Ct4C =C,,0 CD>Dt5D =CT3Cä ?Cä4D47C´0 (C 2D$>CÄ0D$8Dt5D :C, 7C 2Dô7C =C =C =C ;C,,G
        AddControl(new CreateChildButton<TEntity>().OnExecute(ExecuteCreateChildRecordAsync));Ð
        Ð
        // 3. A,,!Aô A  A´ AÔ : Aô@Dô<C 0 Cô5D 5CD0Dt0 D AD´;Cä: CÔ0 CÄ5D$>CDK C 5Cr 0CÔ>CÔ8CÄ=D´E C´OCÄ1CD0-C
        AddControl(new EditEntityButton<TEntity>().OnExecute(ExecuteEditRecordAsync));Ð
        Ð
        // 4. A  Cä?C  D44C ;CT=C,,0 (MessageBox CÔ>CDBC$5D 6CD5CÔ8Dò 8 C  CäCÔ04C CCD0C´5CÔ8Dò 2 B #A )Ð
        AddControl(new DeleteEntityButton<TEntity>().OnExecute(ExecuteDeleteRecordAsync));Ð
        Ð
        // B BC =CD0D BCÔKC´ 2C,,7D40C´LCÔKC´ @C 7CD5C´8D$5C´LÐ
        AddControl(new ToolbarDivider());Ð
    }Ð
    Ð
    /// <summary>Ð
    /// B ?CTFC,,DC,,GCÔ0Dò 1C,,7CÔ5D Ô:Cä<C =CD0 D >Ct4C =C,,0 Cô>CDGC,,=CT=CÔ>C4> D47C´0.Ð
    /// B GC,,BD´2C 5D" "B 2D´4CT;CT=CÔ>C4> D >CD8D$5C´0 C, 7C ?C,,AD´2C 5D" 5C4> C" &VçD-B =Cä2Cä9 D CD"=CäA
    /// </summary>Ð

```

```

protected async Task ExecuteCreateChildRecordAsync()
{
    if (SelectedData == null) return;

    // B >Ct4C 5CÂ GC,,AD$KC' >C JCT:D"Â 8D ?Cä;DÄ7D40 CÔ>C$KC' AC,,=D$0C=AC,,A C# Dd5C'5C$>C4> D$8C00, D
    // C, ?D 8C$0CtKC$0CT< CT3Câ : D$5C=CD"5CÄC C$KCD5C'5CÔ=Cä<D2 Cct;D=
    TEntity childModel = new()
    {
        ParentId = SelectedData.Id
    };

    await OpenDialogWindowAsync(childModel, isNew: true);
}
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/TreeContext.cs">
using Microsoft.Extensions.DependencyInjection;
namespace Promatis.Net.UI.Components;
/// <summary>
/// A0@Cä<CT6D4BCäGC0KC' :Cä=D$5C=AD" ?D 5CDAD$0C$;CT=C,,O CD;D0 4D 5C$>C$8CD=D'E D BD 0C08Db ...G&VR 0öFR'í
/// A05D 5C$>CD8D" 0C AD$@C :D$=D'5 CD0C0=D'5 D04D 0 C00 D07D': C,,5D 0D EC,,GCTAC=8DR 2C,,7D40C'8Ct0D$>D >C"í
/// </summary>
public abstract class TreeContext<TEntity, TKey> : EntityContext<TEntity, TKey, object,
List<TEntity>>
{
    where TEntity : class, new()
    where TKey : notnull
    {
        protected TreeContext(IServiceProvider serviceProvider, bool isInMemoryMode)
            : base(serviceProvider, isInMemoryMode)
        {
            // A$ A,, A A,, (A,,5D 0D EC,,GCTAC=0D0 DC,,7C,,:C "¢ Ct2C'5C=0CT< C, 6CTAD$:Cä AC$0CtKC$0CT< C,,7 DI D
            // D ?CTFC,,0C 8Ct8D >C$0C0=D4N D BD 0D$5C48Dâ 4D 5C$>C$8CD=D'E CÄCD$0Dd8C' 3D 0DD0 Aâ B2
            var treeStrategy = serviceProvider.GetRequiredService<IOzuMutationStrategy<TEntity>>();
            OzuCache.SetMutationStrategy(treeStrategy);
        }

        /// <summary>
        /// AD AÄ A0 B' BD A',B$ AD B0 A !A' AD A,, Aä A,, B B %A,, .D
        /// A05D 5Cä?D 5CD5C'0CTBD 0 C= >C0:D 5D$=D' < CD5D 5C$>CÄ 4C'0 Cd5D BC= >C4> CäBD 5Dt5C08D0 Cct;Cä2 C4@C DC
        /// A Ar @C,,AC=0 C00D CD,,8D$L C0>D$>C= >C 5Ct>C00D =CäAD$L Aä B20:D0HC í
        /// </summary>
        protected virtual Func<TEntity, bool>? GetDomainFilter() => null;

        /// <summary>
        /// A,,!B$ A0 A / Aä"A$ B$!B$ AT A0 B "BÄ AT AT A ¢ Cä=C$5C"5D >C @C 1CäBC=8 C,,5D 0D EC,,GCTAC= >C4> D ?C
        /// A0@C,,=C,,<C 5D" 1CT7Cä?C AC0KC' •&V D0æÇ"Æ-7BÄ 8Ct>C'8D >C$0C0=D'9 A @Cä:CT@Cä< C$=D4BD 8 C0>D$>C= >C 5
        /// </summary>
        protected override List<TEntity> EvaluateDataStateInMemory(object state, IReadOnlyList<TEntity>
inMemoryList)
        {
            // 1. A0@C,,<CT=D05CÄ 4Cä<CT=C0KC' DC,,;DÄBD =C AC'5CD=C,,:C Ä 5D ;C, >C0 7C 4C = (C00C0@C,,<CT@, D :D K
            var domainFilter = GetDomainFilter();
            IEnumerable<TEntity> filteredList = domainFilter != null
                ? inMemoryList.Where(domainFilter)
                : inMemoryList;

            // 2. A,,5D 0D EC,,GCTAC= >CR CC0>D OCD>Dt8C$0C08CR 3D 0DD0. D
            // A0> D4<Cä;Dt0C08Dâ <D² 2Cä7C$@C IC 5CÄ ;C,,=CT9C0KC' AC08D >C¢Ä =Cä >D$ACä@D$8D >C$0C0=D'9 D$0C¢Ä G
            // (D2 :CäBCä@D'E C,,5D 0D EC,,GCTAC=0D0 AC$0CtL C0CD BC ' HC'8 C05D 2D' <C,â Cä=C= @CTBC0>CR 4CT@CT2Cä
            return filteredList
                .OrderBy(x => x.GetType().GetProperty("ParentId")?.GetValue(x) != null)
                .ToList();
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/WorkspaceContext.cs">
namespace Promatis.Net.UI.Components;
/// <summary>
/// A 0Ct>C$KC' :Cä=D$5C=AD" ;Dä1Cä9 D 0C >Dt5C' >C ;C AD$8 (C$:C'0CD:C,' 2 D 8D BCT<CRí
/// Aä1D"8C' 7C00CÄ5C00D$5C'L CD;D0 <C05CÄ>D ECT<, D ?D 0C$>Dt=C,,:Cä2, CD0D,,1Cä@CD>C" 8 C4@C DC,,:Cä2.D
/// </summary>

```

```

public class WorkspaceContext : IWorkspaceContextD
{D
    /// <summary>D
    /// Að> D4<Cä;Dt0C08Dâ 4C´O D BC BC,,GCTAC¤8DR MC¤@C =Cä2 Ct0C4@D47C¤0 C$ACT3CD0 C$KC¤;DäGCT=C í
    /// B$0Cd5C´KCR 4C BC Ô:Cä=D$5C¤AD$K (DataContext) C05D 5Cä?D 5CD5C´OD" MD$> D 2Cä9D BC$> D 2Cä5C´ G'VRöf
    /// </summary>D
    public virtual bool IsLoading => false;D
D
    // --- AD BD A´"AÔ+AR !B$ A´ A, AT AÄ B$ A,, 'AT!A¤ AR A AÄ B + Aô A "BD B B² òòÝ
    public virtual int PaperElevation => 1;D
    public virtual string PaperClass => "pa-4 d-flex flex-column flex-grow-1 w-100 h-100";D
    public virtual string WorkspaceHeight => "100%";D
    public virtual string TopZoneHeight => "auto";D
    public virtual string BottomZoneHeight => "auto";D
    public virtual string LeftZoneWidth => "250px";D
    public virtual string RightZoneWidth => "300px";D
D
    // --- A B$ B Aä B "A$ AÔ B AÔ"A B B field (Bt B "B´ STATE CONTAINER AD Bò A´ A" Aä A ´ òòÝ
    public bool IsTopZoneCollapsed { get; set; }D
    public bool IsBottomZoneCollapsed { get; set; }D
    public bool IsLeftZoneCollapsed { get; set; }D
    public bool IsRightZoneCollapsed { get; set; }D
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor">
@typeparam TModel where TModel : classD
D
<MudDialog>D
    <TitleContent>D
        <MudText Typo="Typo.h6">D
            @TitleD
        </MudText>D
    </TitleContent>D
D
    <DialogContent>D
        @* A,,=D$5C4@C,,@D45CÂ fçVVçEf Æ-F F-ôâ =C ?D OCÄCDâ 2 CÔ0D$8C$=D4N DD>D <D2 ×VD&Æ ¦÷" ¢
        <MudForm @ref=" form"D
            Model="@Model"D
            Validation="@@(async (object model) => await ValidateModelViaFluentAsync())"D
            ValidationDelay="0">D
D
            @* B BD >C48C´ 3CT>CÄ5D$@C,,GCTAC¤8C´ :Cä=D$5C´=CT@ Dô4D 0 – Ct0D"8D"0CTB DD>D <D2 >D" ?D KCd:Cä2
            <div class="d-flex flex-column pt-2" style="min-height: 340px; height: 60vh; max-
height: 60vh; min-width: 0; overflow: hidden;">D
                @ChildContentD
            </div>D
D
        </MudForm>D
    </DialogContent>D
D
    <DialogActions>D
        @* AÔ0D$8C$=Cä5 C¤0D :C 4CÔ>CR CCô@C 2C´5C08CR >C¤=Cä< MudBlazor *@D
        <MudButton Variant="Variant.Text"D
            Color="Color.Default"D
            OnClick="HandleCancelAsync">D
            AäBCÄ5C00D
        </MudButton>D
D
        <MudButton Variant="Variant.Filled"D
            Color="Color.Primary"D
            OnClick="HandleSubmitAsync">D
            B >DT@C =C,,BDÍ
        </MudButton>D
    </DialogActions>D
</MudDialog>
</file>

<file path="Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor.cs">
using FluentValidation;D
using Microsoft.AspNetCore.Components;D
using MudBlazor;D
D
namespace Promatis.Net.UI.Components.Dialogs;D
D
public partial class BaseDialogLayout<TModel> : ComponentBase where TModel : classD

```

```

{
    protected MudForm _form = null!;

    [CascadingParameter]
    protected IMudDialogInstance MudDialog { get; set; } = null!;

    [Parameter]
    public string Title { get; set; } = "B 5CD0C=BC,,@Cä2C =C,,5 Ct0C08D 8";

    [Parameter]
    public required TModel Model { get; set; }

    [Parameter]
    public IValidator<TModel>? Validator { get; set; }

    /// <summary>
    /// A>C0BCT=D"Â :CäBCä@D'9 C CCD5D" @C 7C$5D =D4B C$=D4BD 8 DD>D <D² „CC08C=0C'LC0KCR ?Cä;Dò 2C$>CD0 C,,;
    /// </summary>
    [Parameter]
    public required RenderFragment ChildContent { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();
        if (Model == null)
            throw new ArgumentNullException(nameof(Model), "A00D 0CÄ5D$@ Model D02C'0CTBD 0 Cä1D07C BCT;DÄ=D'");
    }

    public async Task HandleSubmitAsync()
    {
        if (_form == null) return;

        // A00D$8C$=Cä AC=0C08D CCT< C$ADâ DCä@CÄC D 8C'0CÄ8 MudBlazor
        await _form.ValidateAsync();

        if (_form.IsValid)
        {
            // At0C=@D'2C 5CÄ >C= Cä 8 C$>Ct2D 0D"0CT< C$0C'8CD=D4N CÄ>CD5C'L C00 D BD 0C08DdC0
            MudDialog.Close(DialogResult.Ok(Model));
        }
    }

    public void HandleCancelAsync()
    {
        MudDialog.Cancel();
    }

    protected async Task<IEnumerable<string>> ValidateModelViaFluentAsync()
    {
        if (Validator == null) return Array.Empty<string>();

        var result = await Validator.ValidateAsync(Model);
        if (result.IsValid) return Array.Empty<string>();

        // AäBCD0CT< MudForm C0;CäAC=8C' AC08D >C¢ BCT:D BCä2D'E CäHC,,1Cä: CD;Dò ?Cä4D 2CTBC=8 C,,=C0CD$>C-
        return result.Errors.Select(e => e.ErrorMessage);
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Dialogs/DialogTabConfig.cs">
using Microsoft.AspNetCore.Components;

namespace Promatis.Net.UI.Components.Dialogs;

/// <summary>
/// B BD >C4> D$8C08Ct8D >C$0C0=D'9 C=;C AD 0:Cä=DD8C4CD 0D$>D 4C'0 CD8C00CÄ8Dt5D :C,,E C$:C'0CD>C¢1
/// A0>Ct2Cä;D05D" ?CT@CT4C 2C BDÂ :C AD$>CÄ=D4N D 0Ct<CTBC=C C0>C'5C' 2C$>CD0, C0@C,,2D07C =C0CDâ : Cd8C$>C'
/// </summary>
public class DialogTabConfig<TModel> where TModel : class
{
    /// <summary>
    /// B$5C=AD$>C$K¢ 7C 3Cä;Cä2Cä: C$:C'0CD:C, =C ?C =CT;C, xVEF '21
    /// </summary>
    public string Title { get; }
}

```

```

    /// <summary>Ð
    /// B 8D BCT<C00D0 8C¤>C0:C xVD&Æ |÷" 4C´O Ct0C4>C´>C$:C 2C¤;C 4C¤8 (Cä?Dd8Cä=C ;DÄ=Cä´i
    /// </summary>Ð
    public string? Icon { get; }Ð
Ð
    /// <summary>Ð
    /// B BD >C4> D$8C08Ct8D >C$0C0=C O D 0Ct<CTBC¤0 C0>C´5C´ 2C$>CD0, C0@C,,=C,,<C ND´´0D0 6C,,2D4N CD>CÄ5C0=D4M
    /// </summary>Ð
    public RenderFragment<TModel> Content { get; }Ð
Ð
    public DialogTabConfig(string title, RenderFragment<TModel> content, string? icon = null)Ð
    {Ð
        if (string.IsNullOrEmpty(title))Ð
            throw new ArgumentException("At0C4>C´>C$>C¤ 2C¤;C 4C¤8 C05 CÄ>Cd5D" 1D´BDÄ ?D4AD$KCÄâ"Ä æ ÖVöb±F-
Ð
            Title = title;Ð
            Content = content ?? throw new ArgumentNullException(nameof(content));Ð
            Icon = icon;Ð
        }Ð
    }
</file>

<file path="Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor">
@typeparam TModel where TModel : Promatis.Net.Domain.ReferenceBaseÐ
Ð
@* Aä1Cä@C GC,,2C 5CÄ 2D Q C" 1C 7Cä2D´9 C¤0D :C A D04D 0, C0@Cä1D 0D KC$0D0 <Cä4CT;DÄÄ 7C 3Cä;Cä2Cä: C, 2C ;C
<BaseDialogLayout TModel="TModel"Ð
    Model="@Model"Ð
    Title="@Title"Ð
    Validator="@Validator">Ð
Ð
    @* A$=D4BD 5C0=C,,9 C¤>C0BCT=D" :C @C¤0D 0: Cä?D 5CD5C´0CT<, C0CCd=Cä ;C, @C 7C$>D 0Dt8C$0D$L C$:C´0CD:C,
    @if (CustomTabs == null || !CustomTabs.Any())Ð
    {Ð
        @* B &AT A A,, A ¢ C AD$>CÄ=D´E C$:C´0CD>C¤ =CTB. A$KC$>CD8CÄ BCä;DÄ:Cä 1C 7Cä2D´5 C0>C´O D ?D 0C$>
        <div class="flex-grow-1" style="height: 100%; overflow-y: auto; padding-right: 4px;">Ð
            @RenderBaseFields()Ð
        </div>Ð
    }Ð
    elseÐ
    {Ð
        @* B &AT A A,, A ¢ C AC´5CD=C,,: C05D 5CD0C² AC$>C, 2C¤;C 4C¤8. B 0Ct2Cä@C GC,,2C 5CÄ <C0>C4>D BD 0C0
        <MudTabs Elevation="0"Ð
            Rounded="true"Ð
            Class="d-flex flex-column flex-grow-1 h-100"Ð
            @bind-ActivePanelIndex="_activeTabIndex">Ð
Ð
            @* B 8D BCT<C00D0 C¤;C 4C¤0 1: A0>C´O ReferenceBase CäB D04D 0 D 8D BCT<D² ¢
            <MudTabPanel Text="AäAC0>C$=Cä5" Icon="@Icons.Material.Filled.Info" PanelClass="pa-4 flex-grow-1"
                <div style="max-height: 48vh; overflow-y: auto; padding-right: 4px;">Ð
                    @RenderBaseFields()Ð
                </div>Ð
            </MudTabPanel>Ð
Ð
            @* AD8C00CÄ8Dt5D :C,,5 A$:C´0CD:C, "³¢ CT@CT1C,,@C 5CÄ 8 D 5C04CT@C,,< C$ACR :C AD$>CÄ=D´5 D$0C K D
            @foreach (DialogTabConfig<TModel> tab in CustomTabs)Ð
            {Ð
                <MudTabPanel Text="@tab.Title" Icon="@tab.Icon" PanelClass="pa-4 flex-grow-1">Ð
                    <div style="max-height: 48vh; overflow-y: auto; padding-right: 4px;">Ð
                        @tab.Content(Model)Ð
                    </div>Ð
                </MudTabPanel>Ð
            }Ð
        }Ð
    }Ð
</MudTabs>Ð
}Ð
Ð
</BaseDialogLayout>Ð
Ð
@code {Ð
    /// <summary>Ð
    /// B BC =CD0D BC,,7C,,@Cä2C =C00D0 @C 7CÄ5D$:C 1C 7Cä2D´E C0>C´5C´ B , Cä1D´´0D0 4C´O C$ACTE 100+ D ?D 0
    /// </summary>Ð
    private RenderFragment RenderBaseFields() => __builder =>Ð
    {Ð
        <div class="d-flex flex-column gap-4">Ð

```

```

        <MudTextField @bind-Value="Model.Name"
                    Label="A00C,,<CT=Cä2C =C,,5"
                    For="@(() => Model.Name)"
                    Required="true" />
    }

    <MudTextField @bind-Value="Model.Code"
                Label="B 8D BCT<C0KC' :Cä4 (B4=C,,:C ;DÄ=D'9)"
                For="@(() => Model.Code)" />

    <MudTextField @bind-Value="Model.Description"
                Label="Aä?C,,AC =C,,5 / AD>C0>C'=C,,BCT;DÄ=Cä5 C0@C,,<CTGC =C,,5"
                Lines="4"
                For="@(() => Model.Description)" />
</div>
};
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor.cs">
using FluentValidation;
using Microsoft.AspNetCore.Components;
using Promatis.Net.Domain;
namespace Promatis.Net.UI.Components.Dialogs;
public partial class ReferenceDialogLayout<TModel> : ComponentBase where TModel : ReferenceBase
{
    protected int _activeTabIndex;

    /// <summary>
    /// B$5C=AD$>C$KC' 7C 3Cä;Cä2Cä: C" HC ?C=5 CÄ>CD0C'LC0>C4> Cä:C00. A0@Cä1D 0D KC$0CTBD 0 C00C0@D0<D4N C"
    /// </summary>
    [Parameter]
    public string Title { get; set; } = "B 5CD0C=BC,,@Cä2C =C,,5 D ?D 0C$>Dt=C,,:C #½

    /// <summary>
    /// Ad8C$0D0 @CT4C :D$8D CCT<C 0 CD>CÄ5C0=C 0 CÄ>CD5C'L (C00D ;CT4C08C# &VfW&Væ6T& 6R'í
    /// </summary>
    [Parameter]
    public required TModel Model { get; set; }

    /// <summary>
    /// A0>C'8CÄ>D DC0KC' fÇVVÇB02C ;C,,4C BCä@ CD;D0 :Cä=C=0@CTBC0>C4> D ?D 0C$>Dt=C,,:C í
    /// </summary>
    [Parameter]
    public IValidator<TModel>? Validator { get; set; }

    /// <summary>
    /// A= >C' ;CT:Dd8D0 AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =C0KDR :C AD$>CÄ=D'E C$:C'0CD>C# „4CTAC=0C,,?D$>D >C''Ä
    /// C= >D$>D KCR :Cä=C=0@CTBC0KC' AC0@C 2CäGC08C# ECäGCTB CD>C 0C$8D$L C00 DD>D <D2 2 CD>C0>C'=CT=C,,5 C# 2C
    /// </summary>
    [Parameter]
    public IReadOnlyCollection<DialogTabConfig<TModel>>? CustomTabs { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();
        if (Model == null)
            throw new ArgumentNullException(nameof(Model), "A00D 0CÄ5D$@ Model D02C'0CTBD 0 Cä1D07C BCT;DÄ=D'

    }
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor">
@inherits RenderBase
<MudTooltip Text="@((Control.Tooltip ?? Control.Title)" Arrow="true" Placement="Placement.Bottom">
    <MudButton Variant="@Variant"
                StartIcon="@((Control.IsRunning ? null : Control.Icon)"
                Disabled="@(!Control.IsEnabled || Control.IsRunning || !
Control.IsEnabledForData(CurrentSelectedData))"
                Color="@Color"
                OnClick="@HandleClickAsync">

    @if (Control.IsRunning)
    {
        <MudProgressCircular Class="ms-n1 me-2" Size="Size.Small" Indeterminate="true"

```

```

Color="Color.Inherit" />
}
@Control.Title
</MudButton>
</MudTooltip>
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;

namespace Promatis.Net.UI.Components.ElementRenderBase;

public partial class ButtonRenderBase : RenderBase
{
    [Parameter] public Color Color { get; set; } = Color.Inherit;
    [Parameter] public Variant Variant { get; set; } = Variant.Text;

    protected async Task HandleClickAsync()
    {
        await Control.TriggerAsync(CurrentSelectedData);
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor">
@inherits RenderBase

<MudDateRangePicker Label="@Control.Title"
    DateRange="@GetDateRange()"
    DateRangeChanged="@HandleDateRangeChangedAsync"
    Disabled="@(Control.IsEnabled || Control.IsRunning)"
    Margin="Margin.Dense"
    DateFormat="dd.MM.yyyy"
    Variant="Variant.Outlined"
    Style="max-width: 260px;" />

</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs">
using MudBlazor;

namespace Promatis.Net.UI.Components.ElementRenderBase;

public partial class DateRangeRenderBase : RenderBase
{
    /// <summary>
    /// A 5Ct>C00D =Câ 8Ct2C'5C¤0CTB Cä1D¤5C¤B DateRange C,,7 CÄ>CD5C'8 C¤>C0BD >C'0 Dt5D 5Cr 8C0BCT@DD5C"A-D
    /// </summary>
    protected DateRange? GetDateRange()
    {
        if (Control is IHasValue valueProvider)
        {
            return valueProvider.Value as DateRange;
        }
        return null;
    }

    /// <summary>
    /// At0C08D KC$0CTB C0>C$>CR 7C00Dt5C08CR 2D'1D 0C0=Cä3Câ 4C,,0C00Ct>C00 CD0D" 2 CÄ>CD5C'L C, 7C ?D4AC¤0CT
    /// </summary>
    protected async Task HandleDateRangeChangedAsync(DateRange? newRange)
    {
        if (Control is IHasValue valueProvider)
        {
            valueProvider.Value = newRange;
            await Control.TriggerAsync(CurrentSelectedData);
        }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor">
@inherits RenderBase

<MudDivider Vertical="true" FlexItem="true" Class="my-2mx-1" />

```


</file>

```
<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs">
namespace Promatis.Net.UI.Components.ElementRenderBase;
{
public partial class DividerRenderBase : RenderBase
{
}
}</file>
```

```
<file path="Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs">
using Microsoft.AspNetCore.Components;
{
namespace Promatis.Net.UI.Components.ElementRenderBase;
{
/// <summary>
/// A 1D BD 0C=BC0KC' 1C 7Cä2D'9 C=CA?Cä=CT=D" 4C'0 C$ACTE Razor-D 5C04CT@CT@Cä2 D0;CT<CT=D$>C" CC0@C 2C'5C0
/// </summary>
public abstract class RenderBase : ComponentBase, IDisposable
{
/// <summary>
/// A 8Ct=CTA-CÄ>CD5C'L D0;CT<CT=D$0 D4?D 0C$;CT=C,,0, C05D 5CD0C$0CT<C 0 C,,7 DynamicComponent.D
/// </summary>
[Parameter]
public IUIControl Control { get; set; } = null!;

/// <summary>
/// A=0D :C 4C0KC' :Cä=D$5C=AD" ECä;D BC @C 1CäGCT9 Cä1C'0D BC,i
/// </summary>
[CascadingParameter]
protected IWorkspaceContext? ActionContext { get; set; }

/// <summary>
/// A 5Ct=C00D =Cä 8Ct2C'5C=0CTB D$5C=CD"8CR 2D'4CT;CT=C0KCR 4C =C0KCR 8Cr :Cä=D$5C=AD$0 DD>D <D²i
/// </summary>
protected object? CurrentSelectedData => GetSelectedDataFromContext();

protected override void OnInitialized()
{
base.OnInitialized();

// B 5C48D BD 8D CCT< D 5C :D$8C$=Cä5 Cä1C0>C$;CT=C,,5 C,,=D$5D DCT9D 0 C0@C, ACÄ5C05 D >D BCä0C08D0 1C
Control.OnStateChanged += HandleStateChanged;
}

private void HandleStateChanged() => InvokeAsync(StateHasChanged);

private object? GetSelectedDataFromContext()
{
if (ActionContext == null) return null;

Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");
return interfaceType?.GetProperty("SelectedData")?.GetValue(ActionContext);
}

public virtual void Dispose() => Control.OnStateChanged -= HandleStateChanged;
}
}</file>
```

```
<file path="Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor">
@inherits RenderBase
<MudSelect T="string"
Value="@GetControlValue()"
ValueChanged="@HandleValueChangedAsync"
Disabled="@(!Control.IsEnabled || Control.IsRunning)"
Label="@Control.Title"
Margin="Margin.Dense"
Variant="Variant.Outlined"
Style="min-width: 160px; max-width: 250px;">

@foreach (string option in GetOptions())
{
<MudSelectItem T="string" Value="@option">@option</MudSelectItem>
}
</MudSelect>
</file>
```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor.cs">
namespace Promatis.Net.UI.Components.ElementRenderBase;
{
    public partial class SelectRenderBase : RenderBase
    {
        protected string GetControlValue()
        {
            if (Control is IHasValue valueProvider && valueProvider.Value != null)
            {
                return valueProvider.Value.ToString() ?? string.Empty;
            }
            return string.Empty;
        }

        protected async Task HandleValueChangedAsync(string newValue)
        {
            if (Control is IHasValue valueProvider)
            {
                valueProvider.Value = newValue;
                await Control.TriggerAsync(CurrentSelectedData);
            }
        }

        protected IEnumerable<string> GetOptions()
        {
            if (Control is IHasOptions optionsProvider)
            {
                return optionsProvider.Options;
            }
            return Array.Empty<string>();
        }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/UIControls/BaseUiControl.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A 0Ct>C$0D0 0C AD$@C :Dd8D0 4C`O C$ACTE D0;CT<CT=D$>C" CC0@C 2C`5C08D0 ?C`0D$DCä@CÄK (C=Ca?Cä:, Dt5C=1Cä
    /// B 5C ;C,,7D45D" 8C0DD 0D BD CC=BD4@C0CDä @D4BC,,=D2 8 C0@CT4CäAD$0C$;D05D" <CäAD" : CD8C00CÄ8Dt5D :Cä<D2 @C
    /// </summary>
    public abstract class BaseUiControl : IUiControl
    {
        private bool _isVisible = true;
        private bool _isEnabled = true;
        private bool _isRunning;

        public abstract string Id { get; }
        public abstract Type ComponentType { get; }

        public virtual string? Title { get; init; }
        public virtual string? Icon { get; init; }
        public virtual string? Tooltip { get; init; }

        public Dictionary<string, object> ComponentParameters { get; } = new();

        public virtual UIControlAlignment Alignment { get; init; } = UIControlAlignment.Left;

        public bool IsVisible
        {
            get => _isVisible;
            set { if (_isVisible != value) { _isVisible = value; NotifyStateChanged(); } }
        }

        public bool IsEnabled
        {
            get => _isEnabled;
            set { if (_isEnabled != value) { _isEnabled = value; NotifyStateChanged(); } }
        }

        public bool IsRunning => _isRunning;

        public event Action? OnStateChanged;

        protected BaseUiControl()
    }
}

```

```

    {
        ComponentParameters.Add("Control", this);
    }

    protected void NotifyStateChanged() => OnStateChanged?.Invoke();

    public virtual bool IsEnabledForData(object? targetData) => true;

    public async Task TriggerAsync(object? targetData)
    {
        if (!_isEnabled || _isRunning || !IsEnabledForData(targetData)) return;

        try
        {
            _isRunning = true;
            NotifyStateChanged();
            await HandleTriggerAsync(targetData);
        }
        finally
        {
            _isRunning = false;
            NotifyStateChanged();
        }
    }

    protected abstract Task HandleTriggerAsync(object? targetData);
}
</file>

```

```

<file path="Promatis.Net.UI/Components/UIControls/IHasOptions.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A C# class that implements the IHasOptions interface.
    /// </summary>
    public interface IHasOptions : IUIControl
    {
        IEnumerable<string> Options { get; }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/UIControls/IHasValue.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A C# class that implements the IHasValue interface.
    /// </summary>
    public interface IHasValue : IUIControl
    {
        object? Value { get; set; }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/UIControls/IUIControl.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A C# class that implements the IUIControl interface.
    /// </summary>
    public interface IUIControl
    {
        string Id { get; }
        Type ComponentType { get; }
        Dictionary<string, object> ComponentParameters { get; }

        string? Title { get; }
        string? Icon { get; }
        string? Tooltip { get; }

        bool IsVisible { get; set; }
        bool IsEnabled { get; set; }
        bool IsRunning { get; }

        UIControlAlignment Alignment { get; }

        bool IsEnabledForData(object? targetData);
    }
}

```

```

        Task TriggerAsync(object? targetData);
    }
    event Action? OnStateChanged;
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/UiControlAlignment.cs">
namespace Promatis.Net.UI.Components;
{
    public enum UiControlAlignment
    {
        Left,
        Right
    }
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor">
@typeparam TEntity where TEntity : class
{
    <MudDataGrid @ref="_mudGrid"
        ServerData="@ServerDataProvider"
        T="TEntity"
        SelectedItem="@SelectedItem"
        SelectedItemChanged="@OnSelectedItemChanged"
        RowStyleFunc="@FormatSelectedItemStyle"
        RowsPerPage="@RowsPerPage"
        SelectOnClick="true"
        Hover="true"
        Dense="true"
        Bordered="true"
        Elevation="0"
        FixedHeader="true"
        Height="calc(100vh - 280px)"
        Class="flex-grow-1 w-100 cursor-pointer">
        <Columns>
            @GridColumns
        </Columns>
    >
    @* A$:C'NDt0CT< C00C48C00D$>D BCä;DÄ:Cä ?Cä BD 5C >C$0C08Dâ :Cä=CTGCÔ>C' DCä@CÄK *@
    <PagerContent>
        @if (WithPagination)
        {
            <MudDataGridPager T="TEntity" InfoFormat="{first_item}-{last_item} C,,7 {all_items}"
            RowsPerPageString="B BD >C£¢" óí
            >
        }
    </PagerContent>
</MudDataGrid>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;
{
    namespace Promatis.Net.UI.Components.Workspaces;
    {
        public partial class GridPage<TEntity> : ComponentBase, IDisposable where TEntity : class
        {
            private MudDataGrid<TEntity>? _mudGrid;
            private IWorkspaceContext? _currentContext;

            [CascadingParameter]
            protected IWorkspaceContext? ActionContext { get; set; }

            [Parameter]
            public Func<GridState<TEntity>, CancellationToken, Task<GridData<TEntity>>>? ServerDataProvider
            { get; set; }

            [Parameter]
            public RenderFragment? GridColumns { get; set; }

            [Parameter]
            public bool WithPagination { get; set; } = false;

            [Parameter]
            public int RowsPerPage { get; set; } = 10000;
        }
    }
}

```

```

protected TEntity? SelectedRow { get; set; }
}
/// <summary>
/// A>CÔBD >C´L CÔ>CD?C,,ACä: CÔ0 C,,7CÄ5CÔ5CÔ8Dò :Cä=D$5C▯AD$0 CD;Dò 7C IC,,BD² >D" CD$5Dt5C¢ ?C <DôBC,í
/// Aô5D 5Cô>CD?C,,AD´2C 5D$ADò =C ;CTBD2Ä 5D ;C, &Æ ¦÷" ?Cä4CÄ5CÔ8D" 6 66 F-æu & ÖWFW" :Cä=D$5C▯AD$0.
/// </summary>
protected override void OnParametersSet()
{
    base.OnParametersSet();
}
if (ActionContext != _currentContext)
{
    if (_currentContext != null)
    {
        _currentContext.OnContextStateChanged -= HandleContextStateChanged;
    }
    _currentContext = ActionContext;
    if (ActionContext != null)
    {
        ActionContext.OnContextStateChanged += HandleContextStateChanged;
    }
}
}
/// <summary>
/// B 5C 3C,,@D45D" =C ;Dä1D´5 C,,7CÄ5CÔ5CÔ8Dò ACäAD$>Dô=C,,0 C▯>CÔBCT:D BC ,,=C ?D 8CÄ5D Ä 2C▯;DäGCT=C,,5/C
/// </summary>
private void HandleContextStateChanged()
{
    InvokeAsync(StateHasChanged);
}
public Task ReloadServerData()
{
    return _mudGrid != null ? _mudGrid.ReloadServerData() : Task.CompletedTask;
}
protected string FormatSelectedRowStyle(TEntity item, int rowNumber)
=> item == SelectedRow ? "background-color: var(--mud-palette-action-disabled-background);
font-weight: 500;" : string.Empty;
}
protected void OnSelectedRowChanged(TEntity? newSelection)
{
    // B BT A AT Aä¢ C H CÔ0CÄ5D 5CÔ=Cä ACô@Cä5C▯BC,,@Cä2C =CÔKC´ U,ôHC 3 DD8C▯AC FC,,8 DD>C▯CD 0
    if (newSelection == null && SelectedRow != null) return;
    SelectedRow = newSelection;
    // B$@C =D ;C,,@D45CÄ AD$5C" B CÔ0Cô@Dô<D4N C" OCD@Cäâ Cä=D$5C▯AD" AC < D 8CÔED >CÔ=Cä >C =Cä2C,,B D >D
    if (ActionContext is IHasSelectedData<TEntity> bindableContext)
    {
        bindableContext.SelectedData = newSelection;
    }
}
public void Dispose()
{
    if (_currentContext != null)
    {
        _currentContext.OnContextStateChanged -= HandleContextStateChanged;
    }
}
}
</file>
<file path="Promatis.Net.UI/Components/Workspaces/ReferencePage.razor">
@typeparam TEntity where TEntity : Promatis.Net.Domain.ReferenceBase, new()
{
    @* A▯0D :C 4CÔ> Cô@Cä1D 0D KC$0CT< C▯>CÔBCT:D B C$=C,,7. AT3Cä ?Cä9CÄ0DäB C, V•Föö&& "Ä 8 C$0D, w&-E vR ▯
    <CascadingValue Value="@((IWorkspaceContext)Context)">
    <WorkspacePage>
    {
        @* 1. A$ B %AÔ/Bò Aä A ¢ C$BCä<C BC,,GCTAC▯8C´ BD4;C 0D :CÔ>Cô>C¢ ACô@C 2CäGCÔ8C▯0 AÔ!A, ▯
        <TopContent>

```

```

        <UiToolBar Controls="@Context.Controls" />
    </TopContent>
}

@* 2. Bd A"B A',A" B" Aä A ¢ "C 1C'8Dt=C O D 5D$:C A D 8D BCT<CÖKCÄ8 Cö>C'OCÄ8 Cö> D4<Cä;Dt0CÖ8DÄ
<BodyContent>
    @* A,,!A" A A' A" : ServerDataProvider D$5Cö5D L C$KtKC$0CTB GetDataAsync, Ct0CöCD :C O C=@D4BC,,
    <GridPage TEntity="TEntity"
        @ref="_grid"
        ServerDataProvider="@Context.GetDataAsync"
        WithPagination="true"
        RowsPerPage="20">
    <GridColumn>
}

        @* B B "AT A"AR Aä Aä A : A$KCS$>CDOD$AD" 0C$BCä<C BC,,GCTAC=8 CD;D" B BR AC"0C 2CäGC
        <PropertyColumn T="TEntity" TProperty="string" Property="x => x.Code" Title="B 8D BCT<CÖK
HeaderStyle="width: 180px;" />
        <PropertyColumn T="TEntity" TProperty="string" Property="x => x.Name" Title="A"0C,,<CT=Cä2
HeaderStyle="width: 350px;" />
        <PropertyColumn T="TEntity" TProperty="string" Property="x => x.Description"
Title="Aä?C,,AC =C,,5 / A"0C,,<CTGC =C,,5" HeaderStyle="width: auto;" />
}

        @* A B "Aä A"AR Aä Aä A : B ?CTFC,,DC,,GCÖKCR ?Cä;D"Ä :CäBCä@D'5 CD>C 0C$8D" 4CäGCT@CÖ0
        @CustomColumns
}

    </GridColumn>
    </GridPage>
</BodyContent>
}

</WorkspacePage>
</CascadingValue>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/ReferencePage.razor.cs">
using Microsoft.AspNetCore.Components;
}
namespace Promatis.Net.UI.Components.Workspaces;
}
public abstract partial class ReferencePage<TEntity> : ComponentBase, IDisposable
    where TEntity : Promatis.Net.Domain.ReferenceBase, new()
{
    protected GridPage<TEntity>? _grid;
    private bool _isDisposed;

    [Inject]
    protected IServiceProvider PageServiceProvider { get; set; } = null!;

    /// <summary>
    /// B BD >C4> D$8Cö8Ct8D >C$0CÖ=D'9 C">CÖBCT:D B D4?D 0C$;CT=C,,O DÖBC,,< D ?D 0C$>Dt=C,,:Cä<.
    /// </summary>
    protected abstract ReferenceContext<TEntity> Context { get; }

    /// <summary>
    /// B ;CäB CD;D" 4CT:C'0D 0D$8C$=Cä3Cä >Cö8D 0CÖ8D" CCÖ8C=0C'LCÖKDR :Cä;Cä=Cä: D$0C ;C,,FD²í
    /// </summary>
    [Parameter]
    public RenderFragment? CustomColumns { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();

        // A"0C?C,,AD'2C 5CÄAD" =C ACä1D'BC,,O C,,7CÄ5CÖ5CÖ8D" AD$5C"BC :Cä=D$5C=AD$0 CD;D" @CT0C=BC,,2CÖ>C' ?
        Context.OnContextStateChanged += RefreshUi;

        // B 2Dö7D'2C 5CÄ ACä1D'BC,,5 DD8C=AC FC,,8 CD0CÖ=D'E (C">CÄ<C,,BC !B4 AB >D" BD 8C43CT@Cä2) D <Dö3C">
        Context.OnContextUpdated = RefreshGrid;
    }

    // A,,!A" A A' A" (Ad5C'5Ct=Cä5 A @DT8D$5C=BD4@CÖ>CR D 0C$8C'>): A ;Cä:C,,@D4ND"8C' <CTBCä4 OnInitialize
    // C, @D4GCÖ>C' 2D'7Cä2 D BC @Cä3Cä <CTBCä4C Äö D-æ-F- ÄF F 7-æ2,' Aä A" B "BÄ. B4 A AT B²
    // A,,=D$5D DCT9D >D$:D KC$0CTBD O CÄ3CÖ>C$5CÖ=CäÄ 0 CD0CÖ=D'5 C'5CÖ8C$> Ct0Cö0C HC,,2C ND$AD" ACTBC">C' ?
}

    /// <summary>
    /// AÄ5D$>CB ?D 8CÖCCD8D$5C'LCÖ>C4> Cä1CÖ>C$;CT=C,,O CD0CÖ=D'E C" xVDF F w&-Bä D'7D'2C 5D$AD" 1D >C=5D >C
    /// </summary>
    protected void RefreshGrid()

```

```

{
    // A,,ACô>C´LctCCT< C$AD$@Cä5CÔ=D´9 Cô>D$>C¤>C 5Ct>Cô0D =D´9 CÄ5DT0C08Ct< Blazor CD;Dò 2D´7Cä2C 8Cr D
    InvokeAsync(async () =>
    {
        if (_grid != null)
        {
            await _grid.ReloadServerData();
        }
    });
}

private void RefreshUi()
{
    InvokeAsync(StateHasChanged);
}

/// <summary>
/// Að>C´=C 0 CäGC,,AD$:C @CTAD4@D >C" 4C´O Cô@CT4CäBC$@C ICT=C,,O D4BCTGCT: CÔ0CÄ0D$8 C" &Æ |÷-
/// </summary>
public void Dispose()
{
    if (_isDisposed) return;

    if (Context != null)
    {
        Context.OnContextStateChanged -= RefreshUi;

        // ATAC´8 C¤>CÔBCT:D B D >Ct4C 5D$ADò 2CÔCD$@C, 6C,,7CÔ5CÔ=Cä3Cä FC,,:C´0 DD>D <D² ,,=CR GCT@CT7 C4;
        // C$KctKC$0CT< D4BC,,;C,,7C FC,,N CD;Dò >D$?C,,AC¤8 A @Cä:CT@C >D" HC,,=D² !B4 ABí
        Context.Dispose();
    }

    _isDisposed = true;
}
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor">
@typeparam TEntity where TEntity : Promatis.Net.Domain.ReferenceTreeBase<TEntity>, new()
{
    @* A¤0D :C 4CÔ> Cô@Cä1D 0D KC$0CT< C,,5D 0D EC,,GCTAC¤8C´ :Cä=D$5C¤AD"â C4> Cô>C" <C ND" 8 UiToolbar, C, 2C H T
    <CascadingValue Value="@((IWorkspaceContext)Context)">
        <WorkspacePage>
            @* 1. A$ B %AÔ/Bò Aä A ¢ C$BCä<C BC,,GCTAC¤8C´ BD4;C 0D :CÔ>Cô>C¢ CCô@C 2C´5CÔ8Dò 4CT@CT2Cä< (A¤>D
            <TopContent>
                <UiToolbar Controls="@Context.Controls" />
            </TopContent>

            @* 2. Bd AÔ"B A´,AÔ Bò Aä A ¢ D 5C$>C$8CD=C 0 D BD CC¤BD4@C A C 2D$>CÄ0D$8Dt5D :C,,< C$KC$>CD>CÄ ?
            <BodyContent>
                @* A,,!AÔ A A´ AÔ : RootItems D44C ;CT=. TreePage D 0CÄ ;CT=C,,2Cä ?CäBDô=CTB CD0CÔ=D´5 Cô>D ;CR @
                <TreePage TEntity="TEntity"
                    ChildSelector="@ (x => x.Children)"
                    TextSelector="@ (x => string.IsNullOrEmpty(x.Code) ? x.Name :
$"[{x.Code}] {x.Name}]"
                    IconSelector="@ (x => x.Children != null && x.Children.Any() ?
Icons.Material.Filled.Folder : Icons.Material.Filled.InsertDriveFile)" />
            </BodyContent>
        </WorkspacePage>
    </CascadingValue>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor.cs">
using Microsoft.AspNetCore.Components;
{
    namespace Promatis.Net.UI.Components.Workspaces;
    {
        public abstract partial class ReferenceTreePage<TEntity> : ComponentBase, IDisposable
            where TEntity : Promatis.Net.Domain.ReferenceTreeBase<TEntity>, new()
        {
            private bool _isDisposed;

            [Inject]
            protected IServiceProvider PageServiceProvider { get; set; } = null!;

```

```

Ð
/// <summary>Ð
/// B BD >C4> D$8C08Ct8D >C$0C0=D´9 C,,5D 0D EC,,GCTAC=8C´ :Cä=D$5C=AD" CC0@C 2C´5C08D0 MD$8CÂ 4CT@CT2Cä<.Ð
/// </summary>Ð
protected abstract ReferenceTreeContext<TEntity> Context { get; }Ð
Ð
protected override void OnInitialized()Ð
{Ð
    base.OnInitialized();Ð
Ð
    // A0>CD?C,,AD´2C 5CÄAD0 =C ACä1D´BC,,0 C,,7CÄ5C05C08D0 AD$5C"BC :Cä=D$5C=AD$0 CD;D0 @CT0C=BC,,2C0>C´ ?
    Context.OnContextStateChanged += RefreshUi;Ð
Ð
    // B 2D07D´2C 5CÂ ACä1D´BC,,5 DD8C=AC FC,,8 CD0C0=D´E (C= >CÄ<C,,BC !B4 AB´ A CÄ0C4:Cä9 C05D 5D 8D >C$:C
    Context.OnContextUpdated = RefreshUi;Ð
}Ð
Ð
// A,,!A0 A A´ A0 (Ad5C´5Ct=Cä5 A @DT8D$5C=BD4@C0>CR D 0C$8C´>): A ;Cä:C,,@D4ND"8C´ <CTBCä4 OnInitialize
// C, @D4GC0>C´ 2D´7Cä2 D BC @Cä3Cä <CTBCä4C Æ0 D-æ-F- ÄF F 7-æ2,´ Aä A0 B "BÄ. B4 A AT B²
// AD5D 5C$> CäBC=0D´2C 5D$AD0 <C4=Cä2CT=C0>, C 8CT@C @DT8D0 ;CT=C,,2Cä 7C ?D 0D,,8C$0CTBD 0 C0>D ;CR >D$0
Ð
private void RefreshUi()Ð
{Ð
    // A40D 0C0BC,,@D45CÄ ?CäBCä:Cä1CT7Cä?C AC0KC´ 2D´7Cä2 C05D 5D 8D >C$:C, 2 C= >C0BCT:D BCR &Æ |÷" T´0?C
    InvokeAsync(StateHasChanged);Ð
}Ð
Ð
/// <summary>Ð
/// A0>C´=C 0 CäGC,,AD$:C @CTAD4@D >C" 4C´0 C0@CT4CäBC$@C ICT=C,,0 D4BCTGCT: C00CÄ0D$8 C" &Æ |÷-
/// </summary>Ð
public void Dispose()Ð
{Ð
    if (_isDisposed) return;Ð
Ð
    if (Context != null)Ð
    {Ð
        Context.OnContextStateChanged -= RefreshUi;Ð
Ð
        // A$KctKC$0CT< D4BC,,;C,,7C FC,,N C= >C0BCT:D BC 4C´0 C40D 0C0BC,,@Cä2C =C0>C´ >D$?C,,AC=8 C @Cä:CT@C
        Context.Dispose();Ð
    }Ð
Ð
    _isDisposed = true;Ð
}Ð
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor">
@typeparam TEntity where TEntity : classÐ
Ð
Ð
<MudTreeView T="TEntity"Ð
    Items="@TreeItems"Ð
    SelectedValue="@SelectedNode"Ð
    SelectedValueChanged="@OnSelectedNodeChanged"Ð
    Hover="true"Ð
    Dense="true"Ð
    SelectionMode="SelectionMode.SingleSelection"Ð
    Height="calc(100vh - 240px)"Ð
    Class="flex-grow-1 w-100 h-100 overflow-y-auto">Ð
    <ItemTemplate>Ð
        <MudTreeViewItem T="TEntity"Ð
            Value="@context.Value"Ð
            Icon="@GetNodeIcon(context.Value)"Ð
            Text="@GetNodeText(context.Value)"Ð
            Items="@GetChildTreeItemData(context.Value)" />Ð
        </ItemTemplate>Ð
    </MudTreeView>
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs">
using Microsoft.AspNetCore.Components;Ð
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI.Components.Workspaces;Ð
Ð

```



```

public partial class TreePage<TEntity> : ComponentBase, IDisposable where TEntity : class
{
    private IWorkspaceContext? _currentContext;
    private bool _isFirstLoadExecuted;

    [CascadingParameter]
    protected IWorkspaceContext? ActionContext { get; set; }

    [Parameter] public Func<TEntity, IEnumerable<TEntity>>? ChildSelector { get; set; }
    [Parameter] public Func<TEntity, string>? TextSelector { get; set; }
    [Parameter] public Func<TEntity, string>? IconSelector { get; set; }

    protected List<TreeItemData<TEntity>> TreeItems { get; set; } = [];
    protected TEntity? SelectedNode { get; set; }

    protected override void OnParametersSet()
    {
        base.OnParametersSet();

        if (ActionContext != _currentContext)
        {
            if (_currentContext != null)
            {
                _currentContext.OnContextStateChanged -= HandleContextStateChanged;
            }

            _currentContext = ActionContext;

            if (ActionContext != null)
            {
                ActionContext.OnContextStateChanged += HandleContextStateChanged;
            }
        }
    }

    /// <summary>
    /// A,,!A0 A A' A0 (Ad5C'5Ct=Cã5 A @DT8D$5C=BD4@C0>CR D 0C$8C'>): A05D 2C,,GC0KC' 7C ?D >D 4C =C0KDR 7C
    /// B "B A4 A0 B AR BCã3CãÂ :C : Blazor C0>C'=CãAD$LDã >D$@CT=CD5D 8C² 8C0BCT@DD5C"A C, :C @C=0D DCã@
    /// </summary>
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        await base.OnAfterRenderAsync(firstRender);

        if (firstRender && !_isFirstLoadExecuted)
        {
            _isFirstLoadExecuted = true;
            await LoadTreeDataAsync();
        }
    }

    /// <summary>
    /// A,,7C$;CT:C 5D" 0D 8C0ED >C0=D'9 C4@C D Cã1D=5C=BCã2 Dt5D 5Cr 1CT7Cã?C AC0KC' <CTBCã4-CD8D ?CTBDt5D F
    /// A 2D$>Cã0D$8Dt5D :C, 0C=BC,,2C,,@D45D" xVD=fW&Æ ' 7C 3D Cct:C, ECã;D BC =C 2D 5CãO gRPC/API-Ct0C0@CãA
    /// </summary>
    private async Task LoadTreeDataAsync()
    {
        // A0@Cã2CT@D05CãÂ ?Cã4CD5D 6C,,2C 5D" ;C, BCT:D4IC,,9 C=COBCT:D B D 0C >D$C D 4C =C0KCã8 (D02C'0CTBD
        // AD;D0 MD$>C4> CD8C00Cã8Dt5D :C, 8Ct2C'5C=0CT< D BD >C4> D$8C08Ct8D >C$0C0=D'9 Cã5D$>CB GCT@CT7 D 5
        // C0> D$0C$ :C : CD5D 5C$> D 0C >D$0CTB D G&VT6öçFW†BÂ <D² <Cã6CT< C 5Ct>C00D =Cã 2D'?Cã;C08D$L C0@
        if (ActionContext != null)
        {
            // A,,AC0>C'LCtCCT< CD8C00Cã8Dt5D :Cã5 C0@C,,2CT4CT=C,,5, D$0C$ :C : TreeContext Ct0C=0D'B C45C05D 8
            // A0> CãK D$>Dt=Cã 7C00CT<, DtBCã C C05C4> CTAD$L Cã5D$>CB vWDF F 7-æ2†ô&!V7B 7F FR'í
            var method = ActionContext.GetType().GetMethod("GetDataAsync");
            if (method != null)
            {
                // A$KctKC$0CT< CD8D ?CTBDt5D 4C =C0KDR :Cã=D$5C=AD$0. Aã= D 0Cã 2C=;DãGC,,B C=0D4BC,,;C=C Ct0
                var task = (Task<IReadOnlyList<TEntity>>)method.Invoke(ActionContext, [new
object(), default(CancellationTokens)]);
                IReadOnlyList<TEntity> result = await task;

                // A05D 5C=;C 4D'2C 5Cã ?Cã;D4GCT=C0KC' 3D 0DB 2 C$8CtCC ;Dã=Cã5 CD5D 5C$> MudTreeView
                if (result != null)
                {
                    TreeItems = result
                        .Where(item => item != null);
                }
            }
        }
    }
}

```

```

        .Select(item => new TreeItemData<TEntity> { Value = item })
        .ToList();
    }

    StateHasChanged();
}

}

private void HandleContextStateChanged()
{
    // A05D 5D 8D >C$KC$0CT< CD5D 5C$>, CTAC'8 C" At# C,,7CÄ5C08C'8D L C0>C'0, CD>C 0C$8C'8D L D47C'K C,,
    InvokeAsync(StateHasChanged);
}

protected void OnSelectedNodeChanged(TEntity? newSelection)
{
    // A,,!A0 A A' A0 : A$=CT4D 5C00 DD8D <CT=C00D0 U% 07C IC,,BC >D" AC'CDt0C'=Cä3Câ AC @CäAC :C'8C00 C
    // A0>C$BCä@C0KC' :C'8C0 =C 2D'4CT;CT=C0KC' CCT5C² 1Cä;DÄHCR =CR AC08CÄ0CTB DD>C0CD 8 C05 C ;Cä:C,,@
    if (newSelection == null && SelectedNode != null) return;

    SelectedNode = newSelection;

    // B$@C =D ;C,,@D45CÄ 2D'4CT;CT=C,,5 C" OCD@Cââ Cä=D$5C0AD" AC < D 8C0ED >C0=Câ 7C ?D4AD$8D" FCT?CäGC0
    if (ActionContext is IHasSelectedData<TEntity> bindableContext)
    {
        bindableContext.SelectedData = newSelection;
    }
}

protected string GetNodeText(TEntity? item) =>
    item == null ? string.Empty : (TextSelector?.Invoke(item) ?? item.ToString() ??
string.Empty);

protected string GetNodeIcon(TEntity? item) =>
    item == null ? Icons.Material.Filled.Folder : (IconSelector?.Invoke(item) ??
Icons.Material.Filled.Folder);

protected List<TreeItemData<TEntity>> GetChildTreeItemData(TEntity? item)
{
    if (item == null || ChildSelector == null) return [];

    IEnumerable<TEntity>? children = ChildSelector.Invoke(item);
    if (children == null) return [];

    return children
        .Where(child => child != null)
        .Select(child => new TreeItemData<TEntity> { Value = child })
        .ToList();
}

public void Dispose()
{
    if (_currentContext != null)
    {
        _currentContext.OnContextStateChanged -= HandleContextStateChanged;
    }
}
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor">
<div class="d-flex align-center justify-space-between gap-4 pa-0 w-100" style="min-height: 48px;
width: 100%;">
    @if (Controls != null)
    {
        @* A'5C$0D0 3D CC0?C 0 DC,,;DÄBD K C, :C0>C0:C, 5%TB 2D'AD$@C 8C$0DäBD 0 C" @D04 D ;CT2C 0
        <div class="d-flex align-center gap-4">
            @foreach (IUIControl control in Controls.Where(c => c.Alignment ==
UiControlAlignment.Left))
            {
                @if (!control.IsVisible) continue;
                <DynamicComponent Type="@control.ComponentType"
Parameters="@control.ComponentParameters" />
            }
        }
    }

```

```

        </div>@
@* A@C 2C 0 C4@D4?C@0: D 0Ct4CT;C,,BCT;C, 8 C=Cä?C=8 C$KC4@D47C=8/D@:D ?Cä@D$0 D ?D 0C$0 *@@
<div class="d-flex align-center gap-4">@
    @foreach (IUiControl control in Controls.Where(c => c.Alignment ==
UiControlAlignment.Right))@
    {
        @if (!control.IsVisible) continue;@
        <DynamicComponent Type="@control.ComponentType"
Parameters="@control.ComponentParameters" />@
    }@
</div>@
}
</div>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs">
using System.Reflection;@
using Microsoft.AspNetCore.Components;@
@
namespace Promatis.Net.UI.Components.Workspaces;@
@
public partial class UiToolbar : ComponentBase@
{
    /// <summary>@
    /// B ?C,,ACä: C@>C´8CÄ>D DCÔKDR :Cä=D$@Cä;Cä2 (C=Cä?Cä:, DD8C´LD$@Cä2, D 5C´5C=BCä@Cä2) CD;D@ >D$@C,,ACä2
    /// </summary>@
    [Parameter]@
    public IEnumerable<IUiControl>? Controls { get; set; }@
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor">
@* A 0Ct>C$KC´ 3CT>CÄ5D$@C,,GCTAC=8C´ :C @C=0D R 7Cä= D@:D 0C@0 C@;C BDD>D <D² ¢
<MudPaper Elevation="@Context.PaperElevation"@
    Class="@Context.PaperClass"@
    Style="@($"height: {Context.WorkspaceHeight}; position: relative; overflow: hidden;")">@
@
@* A@ AD-A@ A A@+A´ A$ B AT At A4 B4 A@ : A@0D$8C$=Cä Adt8D$KC$0CTB D >D BCä0C@8CR 8Cr :C AC=0CD=Cä3C
<MudOverlay Visible="@Context.IsLoading" DarkBackground="true" Absolute="true" ZIndex="9999">@
    <MudProgressCircular Color="Color.Primary" Indeterminate="true" Size="Size.Large" />@
</MudOverlay>@
@
@* A$ B %A@/B@ Aä A ,, "D4;C 0D K / A@0C@5C´8 DD8C´LD$@C FC,,8) *@@
@if (TopContent != null && !Context.IsTopZoneCollapsed)@
{
    <div class="workspace-top-zone flex-none" style="height: @Context.TopZoneHeight; width:
100%;">@
        @TopContent@
    </div>@
}
@
@* B AT A@ A´ A AT" BT A´!B$ (A´5C$0D@ ?C =CT;DÄ² &CT=D$@C ;DÄ=Cä5 D$5C´> + A@C 2C 0 C@0C@5C´L) *@@
<div class="d-flex flex-row flex-grow-1 w-100 h-100 min-w-0" style="overflow: hidden;">@
@
@* A´ A$ B@ Aä A ,, D 5C$>C$8CD=C 0 C@0C$8C40Dd8D@ @ CT:C´0D 0D$8C$=D´5 DD8C´LD$@D²' ¢
@if (LeftContent != null && !Context.IsLeftZoneCollapsed)@
{
    <div class="workspace-left-zone flex-none" style="width: @Context.LeftZoneWidth;
height: 100%;">@
        @LeftContent@
    </div>@
}
@
@* Bd A@"B A´,A@ AR "AT Aä ,, D =Cä2C@>C´ @C 1CäGC,,9 C=C@C@BCT=D#¢ "C 1C´8DdK / B ECT<D² @ Cä:D4<CT=D
<div class="workspace-body-zone flex-grow-1 h-100 min-w-0" style="position: relative;">@
    @BodyContent@
</div>@
@
@* A@ A A / At A@ (A,,=D ?CT:D$>D K D 2Cä9D BC" @ CTBC ;C,,7C FC,,0) *@@
@if (RightContent != null && !Context.IsRightZoneCollapsed)@
{
    <div class="workspace-right-zone flex-none" style="width: @Context.RightZoneWidth;
height: 100%;">@
        @RightContent@
    </div>@
}

```

```

    }@
</div>@
@
@* AÔ Ad Bô/ At AÔ (B BC BD4A-C 0D ò D$>C4>C$KCR ?Cä:C 7C BCT;C,' ¢
@if (BottomContent != null && !Context.IsBottomZoneCollapsed)@
{
    <div class="workspace-bottom-zone flex-none" style="height: @Context.BottomZoneHeight;
width: 100%;">@
        @BottomContent@
    </div>@
}
@
</MudPaper>
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs">
using Microsoft.AspNetCore.Components;@
@
namespace Promatis.Net.UI.Components.Workspaces;@
@
public partial class WorkspacePage : ComponentBase@
{
    [CascadingParameter]@
    protected IWorkspaceContext Context { get; set; } = null!;@
@
    [Parameter]@
    public RenderFragment? TopContent { get; set; }@
@
    [Parameter]@
    public RenderFragment? LeftContent { get; set; }@
@
    [Parameter]@
    public required RenderFragment BodyContent { get; set; }@
@
    [Parameter]@
    public RenderFragment? RightContent { get; set; }@
@
    [Parameter]@
    public RenderFragment? BottomContent { get; set; }@
}
</file>

```

```

<file path="Promatis.Net.UI/Controls/CreateChildButton.cs">
using MudBlazor;@
using Promatis.Net.UI.Components;@
using Promatis.Net.UI.Components.ElementRenderBase;@
@
namespace Promatis.Net.UI.Controls;@
@
/// <summary>@
/// B ?CTFC,,0C'8Ct8D >C$0C0=C 0 C¤=Cä?C¤0-C¤>CÄ0C04C 4C'0 D >Ct4C =C,,0 CD>Dt5D =CT3Cä ,,?Cä4Dt8C05C0=Cä3Cä' C
/// A00D$8C$=Cä CC0@C 2C'0CTB D 2Cä8CÄ ACäAD$>D0=C,,5CÄ 4CäAD$CC0=CäAD$8 C00 CäAC0>C$5 C00C'8Dt8Dò 2D'1D 0C0=C
/// </summary>@
public class CreateChildButton<TEntity> : BaseUiControl where TEntity : class@
{
    private Func<Task>? _executeAsync;@
    private readonly string _id = "toolbar_action_create_child_" + Guid.NewGuid().ToString("N");@
@
    public override string Id => _id;@
    public override Type ComponentType => typeof(ButtonRenderBase); // A,,AC0>C'LCtCCT< D BC =CD0D BC0KC' @CT=
    public override string Title => "AD>C 0C$8D$L C0>CDCct5C²#½
    public override string Icon => Icons.Material.Filled.PlaylistAdd; // A,,:Cä=C¤0 CD>C 0C$;CT=C,,0 C" AC08D >
    public override string Tooltip => "B >Ct4C BDÄ 4CäGCT@C08C' MC'5CÄ5C0B C$=D4BD 8 C$KC @C =C0>C4> D47C'0";
@
    public CreateChildButton()@
    {
        // A0> D4<Cä;Dt0C08Dâ :C0>C0:C 7C 1C'>C¤8D >C$0C00, C0>C¤0 D04D > C05 C05D 5CD0D B C" =CT5 C$KC @C =
        IsEnabled = false;@
@
        ComponentParameters.Add("Color", Color.Primary);@
        ComponentParameters.Add("Variant", Variant.Outlined); // A$KCD5C'0CT< C$8CtCC ;DÄ=Cä >D" 3C'0C$=Cä9 C
    }@
@
    /// <summary>@
    /// B 5C :D$8C$=C 0 C0@Cä2CT@C¤0 CD>D BD4?C0>D BC, :Cä<C =CDK D 8C'0CÄ8 D04D 0. @
    /// AÄ5D$>CB 2D'7D'2C 5D$AD0 :Cä=D$5C¤AD$>CÄ VçF-G•v÷&·7 6T6öçFW†B ?D 8 C¤0Cd4Cä< C,,7CÄ5C05C08C, 6VÆV7FV

```

```

    /// </summary>
    public override bool IsEnabledForData(object? targetData)
    {
        // A=Cä?C D BC =Cä2C,,BD O CD>D BD4?C0>C' "Aä BÄ Aâ BCä3CD0, C=C44C 2 D 8D BCT<CR 2D'1D 0C0 @Cä4C
        bool hasParentSelected = targetData != null;

        // B 8C0ED >C08Ct8D CCT< C$=D4BD 5C0=CT5 D >D BCäOC08CR DC'0C40 CD>D BD4?C0>D BC, 4C'0 D 5C04CT@CT@C
        IsEnabled = hasParentSelected;

        return hasParentSelected;
    }

    /// <summary>
    /// A0@C,,2D07C00 C AC,,=DT@Cä=C0>C4> Cä1D 0C >D$GC,,:C ,,4CT;CT3C BC ' 2D'7Cä2C :Cä<C =CDK C,,7 C=C0BCT:D
    /// </summary>
    public CreateChildButton<TEntity> OnExecute(Func<Task> executeAsync)
    {
        _executeAsync = executeAsync ?? throw new ArgumentNullException(nameof(executeAsync));
        return this;
    }

    /// <summary>
    /// B$>Dt:C 2DT>CD0 C=C,,:C ?Cä :C0>C0:CR xVD&E !÷"Ä 2D'7D'2C 5CÄ0D0 GCT@CT7 C$8CtCC ;DÄ=D'9 D 5C04CT@C
    /// </summary>
    protected override async Task HandleTriggerAsync(object? targetData)
    {
        if (_executeAsync != null && IsEnabled)
        {
            await _executeAsync.Invoke();
        }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Controls/CreateEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Controls;
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC=BD4@C0KC' MC'5CÄ5C0B C=Cä?C08 "B >Ct4C BDÄ" 4C'0 D CD'=CäAD$5C' ;Dä1
/// </summary>
public class CreateEntityButton<TEntity> : BaseUiControl where TEntity : class, new()
{
    private Func<Task>? _command; // A00D, 8D ?Cä;C08D$5C'L CD8C ;Cä3C

    public override string Id => $"crud_create_{typeof(TEntity).Name.ToLower()}";
    public override Type ComponentType => typeof(ButtonRenderBase);
    public override string Title => "B >Ct4C BDÄ#%
    public override string Icon => Icons.Material.Filled.Add;
    public override string Tooltip => $"B >Ct4C BDÄ =Cä2D4N Ct0C08D L";

    public CreateEntityButton() { IsEnabled = true; }

    // AÄ5D$>CB 4C'0 C$=CT4D 5C08D0 @CT0C'LC0>C4> CD5C"AD$2C,,0 C,,7 C=C0BCT:D BC
    public CreateEntityButton<TEntity> OnExecute(Func<Task> command)
    {
        _command = command;
        return this;
    }

    public override bool IsEnabledForData(object? targetData) => true;

    protected override async Task HandleTriggerAsync(object? targetData)
    {
        if (_command != null) await _command.Invoke();
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Controls/DeleteEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

```

```

    DeleteEntityButton<TEntity> : BaseUiControl where TEntity : class
    {
        private Func<TEntity, Task>? _command;

        public override string Id => $"crud_delete_{typeof(TEntity).Name.ToLower()}";
        public override Type ComponentType => typeof(ButtonRenderBase);
        public override string Title => "B44C ;C,,BDÂ#½
        public override string Icon => Icons.Material.Filled.Delete;
        public override string Tooltip => "B44C ;C,,BDÂ 2D´1D 0CÔ=D4N Ct0C08D L";

        public DeleteEntityButton<TEntity> OnExecute(Func<TEntity, Task> command)
        {
            _command = command;
            return this;
        }

        public override bool IsEnabledForData(object? targetData)
        {
            if (targetData is not TEntity typedEntity) return false;
            if (typedEntity is Domain.Interface.ISoftDeletable softDeletable)
            {
                return softDeletable.DeletedAt == null;
            }
            return true;
        }

        protected override async Task HandleTriggerAsync(object? targetData)
        {
            if (_command != null && targetData is TEntity typedEntity)
            {
                await _command.Invoke(typedEntity);
            }
        }
    }
</file>

```

```

<file path="Promatis.Net.UI/Controls/EditEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Controls;
{
    /// <summary>
    /// B4=C,,2CT@D 0C´LCÔKC´ 8CÔDD 0D BD CC=BD4@CÔKC´ MC´5CÃ5CÔB C=Ca?C=8 "B 5CD0C=BC,,@Câ2C BDÂ" 4C´O C$KC @C =CÔ>C´ AD4I
    /// </summary>
    public class EditEntityButton<TEntity> : BaseUiControl where TEntity : class
    {
        private Func<TEntity, Task>? _command;

        public override string Id => $"crud_edit_{typeof(TEntity).Name.ToLower()}";
        public override Type ComponentType => typeof(ButtonRenderBase);
        public override string Title => "B 5CD0C=BC,,@Câ2C BDÂ#½
        public override string Icon => Icons.Material.Filled.Edit;
        public override string Tooltip => "B 5CD0C=BC,,@Câ2C BDÂ 2D´1D 0CÔ=D4N Ct0C08D L";

        public EditEntityButton<TEntity> OnExecute(Func<TEntity, Task> command)
        {
            _command = command;
            return this;
        }

        public override bool IsEnabledForData(object? targetData) => targetData is TEntity;

        protected override async Task HandleTriggerAsync(object? targetData)
        {
            if (_command != null && targetData is TEntity typedEntity)
            {
                await _command.Invoke(typedEntity);
            }
        }
    }
}

```

```

    }
}
</file>

```

```

<file path="Promatis.Net.UI/Controls/ToolbarDivider.cs">
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Controls;

/// <summary>
/// B 8D BCT<C0KC' MC'5CÄ5C0B C$5D BC,,:C ;DÄ=Cä3Cä @C 7CD5C'8D$5C'O, D 0Ct<CTIC 5CÄKC' <CT6CDC D0;CT<CT=D$0CÄ
/// </summary>
public class ToolbarDivider : BaseUiControl
{
    private readonly string _id = "core_toolbar_divider_" + Guid.NewGuid().ToString("N");
    public override string Id => _id;
    /// <summary>
    /// A0@C,,2D07C00 C0 =C HCT<D2 7C DC,,:D 8D >C$0C0=Cä<D2 1C 7Cä2Cä<D2 @CT=CD5D 5D C DividerRenderBase.D
    /// </summary>
    public override Type ComponentType => typeof(DividerRenderBase);
    protected override Task HandleTriggerAsync(object? targetData)
    {
        // AD5C0>D 0D$8C$=D'9 D0;CT<CT=D" ?C =CT;C, 8C0AD$0D4<CT=D$>C" =CR 2D'?Cä;C00CTB C'>C48C0C C0> D$@C,,3
        return Task.CompletedTask;
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Data/Cache/FlatOzuMutationStrategy.cs">
using Promatis.Net.Data;

namespace Promatis.Net.UI;

public class FlatOzuMutationStrategy<TEntity> : IOzuMutationStrategy<TEntity> where TEntity : class
{
    // B$5C05D L C0>C0BD 0C0B C0@C,,=C,,<C 5D" 3C,,1C0CDä "60ÆÆV7F-0ä 2CÄ5D BCä 6CTAD$:Cä3Cä Æ-7M
    public void ApplyDelta(ICollection<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector)
    {
        object? entityId = idSelector(entity);
        if (entityId == null) return;
        switch (state)
        {
            case EntityStateChangeEnum.Added:
                // AD>C 0C$;D05CÄÄ BCä;DÄ:Cä 5D ;C, MC'5CÄ5C0BC A D$0C08CÄ -B 5D"5 C05D" 2 C0>C';CT:Dd8C•
                if (!inMemoryItems.Any(x => Equals(idSelector(x), entityId)))
                {
                    inMemoryItems.Add(entity);
                }
                break;
            case EntityStateChangeEnum.Modified:
                // A,,!A0 A A' A0 : B$0C0 :C : ICollection C05 C0>CD4CT@Cd8C$0CTB Cä1D 0D"5C08CR ?Cä 8C04CT:D
                // CÄK C,,AC0>C'LCtCCT< D4=C,,2CT@D 0C'LC0KC' 8 C 5Ct>C00D =D'9 CD;D0 xVD&Æ |÷" ?Cä4DT>CB 'CCD0
                TEntity? existingItem = inMemoryItems.FirstOrDefault(x => Equals(idSelector(x),
entityId));
                if (existingItem != null)
                {
                    inMemoryItems.Remove(existingItem);
                }
                inMemoryItems.Add(entity);
                break;
            case EntityStateChangeEnum.Deleted:
            case EntityStateChangeEnum.SoftDeleted:
                TEntity? itemToRemove = inMemoryItems.FirstOrDefault(x => Equals(idSelector(x),
entityId));
                if (itemToRemove != null)
                {
                    inMemoryItems.Remove(itemToRemove);
                }
                break;
        }
    }
}

```

```

        break;
    }
}
</file>

<file path="Promatis.Net.UI/Data/Cache/HierarchicalOzuMutationStrategy.cs">
using Promatis.Net.Data;
{
namespace Promatis.Net.UI;
{
public class HierarchicalOzuMutationStrategy<TEntity> : IOzuMutationStrategy<TEntity> where
TEntity : class
{
    private readonly Func<TEntity, object?> _parentIdSelector;

    // B 0C >D$0CT< D BD >C4> D 6C,,2Cä9 ICollection C00C$8C40Dd8Cä=C0>C4> D 2Cä9D BC$0, C 5Cr :C´>C08D >C$0C
    private readonly Func<TEntity, ICollection<TEntity>> _childrenSelector;

    // A KD BD KC´ 8C04CT:D 4C´O CÄ3C0>C$5C0=Cä3Cä ?Cä8D :C 7C òf •
    private readonly Dictionary<object, TEntity> _nodeIndex = [];
    private readonly Dictionary<object, object> _childToParentMap = []; // A=0D BC AC$0Ct5C"¢ -D#Ct;C Óâ -D

    private bool _isIndexBuilt;

    public HierarchicalOzuMutationStrategy(
        Func<TEntity, object?> parentIdSelector,
        Func<TEntity, ICollection<TEntity>> childrenSelector)
    {
        _parentIdSelector = parentIdSelector ?? throw new
ArgumentNullException(nameof(parentIdSelector));
        _childrenSelector = childrenSelector ?? throw new
ArgumentNullException(nameof(childrenSelector));
    }

    public void ApplyDelta(ICollection<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector)
    {
        object? entityId = idSelector(entity);
        if (entityId == null) return;

        // A´5C08C$0Dò AC >D :C ?C´>D :Cä3Cä 8C04CT:D 0 C0@C, ?CT@C$>C´ <D4BC FC,,8 CD0C0=D´E DD>D <D½
        if (!_isIndexBuilt)
        {
            BuildIndexRecursive(inMemoryItems, idSelector);
            _isIndexBuilt = true;
        }

        switch (state)
        {
            case EntityStateChangeEnum.Added:
                HandleAdded(inMemoryItems, entity, entityId, idSelector);
                break;

            case EntityStateChangeEnum.Modified:
                HandleModified(inMemoryItems, entity, entityId, idSelector);
                break;

            case EntityStateChangeEnum.Deleted:
            case EntityStateChangeEnum.SoftDeleted:
                HandleDeleted(inMemoryItems, entityId, idSelector);
                break;
        }
    }

    private void HandleAdded(ICollection<TEntity> rootItems, TEntity entity, object entityId,
Func<TEntity, object?> idSelector)
    {
        if (_nodeIndex.ContainsKey(entityId)) return; // At0D"8D$0 CäB CDCC ;C,,@Cä2C =C,,O C" ?C <D0BC•

        object? newParentId = _parentIdSelector(entity);

        if (newParentId == null)
        {
            // Aä1D=5C=B C= >D =CT2Cä9 - CD>C 0C$;D05CÄ 2 C= >D 5C0L Aä B=
            rootItems.Add(entity);
        }
    }
}
}
}

```



```

    }
    else if (_nodeIndex.TryGetValue(newParentId, out var parentNode))
    {
        // Aä1D=5C=B CD>Dt5D =C,,9 - CD>C 0C$;Dô5CÂ 2 Cd8C$CDâ :Cä;C´5C=FC,,N CT3Câ @Cä4C,,BCT;Dý
        var children = _childrenSelector(parentNode);
        children?.Add(entity);
        _childToParentMap[entityId] = newParentId;
    }
}

// A=0D :C 4C0> D 5C48D BD 8D CCT< D47CT; C, 5C4> C0>CD4CT@CT2Câ 2 C KD BD >CÂ 8C04CT:D 5 O(1)
UpdateIndexForSubtree(entity, idSelector);
}

private void HandleModified(ICollection<TEntity> rootItems, TEntity entity, object entityId,
Func<TEntity, object?> idSelector)
{
    // A,,!Aô A A´ Aô : Aô8C=0C=C4> CÄ0C0?C,,=C40. ATAC´8 D47CT; D46CR AD4ICTAD$2Câ2C ;, DT8D CD 3C,,GCTAC
    if (_nodeIndex.TryGetValue(entityId, out var oldNodeReference))
    {
        object? oldParentId = _childToParentMap.TryGetValue(entityId, out var pId) ? pId : null;

        if (oldParentId == null)
        {
            rootItems.Remove(oldNodeReference);
        }
        else if (_nodeIndex.TryGetValue(oldParentId, out var oldParentNode))
        {
            _childrenSelector(oldParentNode)?.Remove(oldNodeReference);
        }

        // A$KDt8D"0CT< D BC @D4N D AD´;C=C C, 5E AC$0Ct8 C,,7 C,,=CD5C=AC ?CT@CT4 CD>C 0C$;CT=C,,5CÂ =Câ2
        RemoveFromIndexRecursive(oldNodeReference, idSelector);
    }

    // A$AD$0C$;Dô5CÂ =Cä2D4N D AD´;C=C C0> C :D$CC ;DÄ=Cä<D2 &VçD-B ,,8CD5C ;DÄ=Câ >C @C 1C BD´2C 5D" 8
    HandleAdded(rootItems, entity, entityId, idSelector);
}

private void HandleDeleted(ICollection<TEntity> rootItems, object entityId, Func<TEntity,
object?> idSelector)
{
    if (!_nodeIndex.TryGetValue(entityId, out var nodeToRemove)) return;

    object? parentId = _childToParentMap.TryGetValue(entityId, out var pId) ? pId : null;

    if (parentId == null)
    {
        rootItems.Remove(nodeToRemove);
    }
    else if (_nodeIndex.TryGetValue(parentId, out var parentNode))
    {
        _childrenSelector(parentNode)?.Remove(nodeToRemove);
    }

    // A=0D :C 4C0> C$KDt8D"0CT< D44C ;CT=C0KC´ ?Cä4C4@C D C,,7 C,,=CD5C=AC
    RemoveFromIndexRecursive(nodeToRemove, idSelector);
}

// --- B ;D46CT1C0KCR <CTBCä4D² CC0@C 2C´5C08D0 1D´AD$@D´< C,,=CD5C=ACä< O(1) ---
private void BuildIndexRecursive(ICollection<TEntity> nodes, Func<TEntity, object?> idSelector,
object? parentId = null)
{
    foreach (var node in nodes)
    {
        object? id = idSelector(node);
        if (id == null) continue;

        _nodeIndex[id] = node;
        if (parentId != null) _childToParentMap[id] = parentId;

        var children = _childrenSelector(node);
        if (children != null && children.Count > 0)
        {
            BuildIndexRecursive(children, idSelector, id);
        }
    }
}

```

```

    }
}

private void UpdateIndexForSubtree(TEntity node, Func<TEntity, object?> idSelector)
{
    object? id = idSelector(node);
    if (id == null) return;

    _nodeIndex[id] = node;

    object? parentId = _parentIdSelector(node);
    if (parentId != null) _childToParentMap[id] = parentId;

    var children = _childrenSelector(node);
    if (children != null)
    {
        foreach (var child in children)
        {
            UpdateIndexForSubtree(child, idSelector);
        }
    }
}

private void RemoveFromIndexRecursive(TEntity node, Func<TEntity, object?> idSelector)
{
    object? id = idSelector(node);
    if (id == null) return;

    _nodeIndex.Remove(id);
    _childToParentMap.Remove(id);

    var children = _childrenSelector(node);
    if (children != null)
    {
        foreach (var child in children)
        {
            RemoveFromIndexRecursive(child, idSelector);
        }
    }
}
}
</file>

```

```

<file path="Promatis.Net.UI/Data/Cache/IOzuMutationStrategy.cs">
using Promatis.Net.Data;
namespace Promatis.Net.UI;
public interface IOzuMutationStrategy<TEntity> where TEntity : class
{
    /// <summary>
    /// A@C,,<CT=D05D" 4CT;DÄBD2 8Ct<CT=CT=C,,9 B #A C 8Ct<CT=D05CÄ>C' :Cä;C'5CFC,,8 C4@C DC >C JCT:D$>C"
    /// </summary>
    void ApplyDelta(ICollection<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector);
}
</file>

```

```

<file path="Promatis.Net.UI/Data/Cache/IUiOzuCache.cs">
using Promatis.Net.Data;
namespace Promatis.Net.UI;
/// <summary>
/// A=C0BD 0C=B C,,7Cä;C,,@Cä2C =C0>C4> C'>C=0C'LC0>C4> Aä B20ED 0C08C'8D"0 CD0C0=D'E C=C0:D 5D$=Cä9 D0:D 0C0
/// A,,AC0>C'LCtCCTB D 8D BCT<C0KC' VçF-G•7F FT6† ævTVçV0 4C'O C0@C,,<CT=CT=C,,O D$>Dt5Dt=D'E CD5C'LD" 8Ct<CT=CT
/// </summary>
/// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C' AD4IC0>D BC,äÄ÷G- W & óí
public interface IUiOzuCache<TEntity> where TEntity : class
{
    /// <summary>
    /// A0@D0<Cä9 CD>D BD4? C  BCT:D4ICT<D2 <C AD 8C$C CD0C0=D'E D$>C'LC= CD;D0 GD$5C08D0ä
    /// A08 Cä4C00 DD>D <C 1Cä;DÄHCR =CR <Cä6CTB C05D 5Ct0C08D 0D$L C,,;C, >Dt8D BC,,BDÄ MD$>D" AC08D >C 2D C
    /// </summary>
    IReadOnlyList<TEntity> InMemoryItems { get; }
}

```

```

    /// <summary>ð
    /// B$>Dt:C ?CT@C$>CÔ0Dt0C´LCÔ>C4> CÔ0Cô>C´=CT=C,,0 C=MD,,0 CD0CÔ=D´<C,Â ?Cä;D4GCT=CÔKCÄ8 CäB DD>D <D²1
    /// A$KctKC$0CTBD 0 D BD >C4> Cä4C,,= D 0Cr ?D 8 C,,=C,,FC,,0C´8Ct0Dd8C,i
    /// </summary>ð
    void Initialize(List<TEntity> initialItems);ð

ð
    /// <summary>ð
    /// A 2D$>CÔ>CÄ=D´9 Aä B2Ô4C$8Cd>C¢ <D4BC FC,,9. BT8D CD 3C,,GCTAC=8 D$>Dt=Cä >C =Cä2C´OCTB C= >C´;CT:Dd8Dä
    /// </summary>ð
    void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity);ð

ð
    void SetMutationStrategy(IOzuMutationStrategy<TEntity> strategy);ð

ð
    /// <summary>ð
    /// A$KcÔ>C´=Dô5D" ?D >C,,7C$>C´LCÔCDâ >Cô5D 0Dd8Dä GD$5CÔ8DòôDC,,;DÄBD 0Dd8C, =C 4 C=MD,,5CÄ 2CÔCD$@C, ?CäB
    /// Aô5D 5DT2C BD´2C 5D" ?CäBCä: UI C, 7C IC,,IC 5D" 5C4> CäB Cô0D 0C´;CT;DÄ=D´E CÄCD$0Dd8C´ 8Cr DCä=Cä2D´
    /// </summary>ð
    TResult ExecuteInLock<TResult>(Func<IReadOnlyList<TEntity>, TResult> evaluator);ð
}
</file>

<file path="Promatis.Net.UI/Data/Cache/UiOzuCache.cs">
using Promatis.Net.Data;ð
using Promatis.Net.Domain.Interface;ð
ð
namespace Promatis.Net.UI;ð
ð
/// <summary>ð
/// B 5C ;C,,7C FC,,0 C,,7Cä;C,,@Cä2C =CÔ>C4> C=MD,,0 C" >Cô5D 0D$8C$=Cä9 Cô0CÄ0D$8 CD;Dò :Cä=C=CTBCÔ>C4> C,,=D BC
/// </summary>ð
public class UiOzuCache<TEntity> : IUiOzuCache<TEntity> where TEntity : classð
{ð
    private IOzuMutationStrategy<TEntity> _mutationStrategy = new
FlatOzuMutationStrategy<TEntity>();ð
    private readonly List<TEntity> _inMemoryItems = [];ð

ð
    // AT4C,,=D´9 Cä1D=5C= B D 8CÔED >CÔ8Ct0Dd8C, 4C´0 Cô>D$>C= >C" GD$5CÔ8Dò 8 Ct0CÔ8D 8ð
    private readonly Lock _lockObject = new();ð

ð
    public IReadOnlyList<TEntity> InMemoryItems => _inMemoryItems;ð

ð
    public UiOzuCache() { }ð

ð
    public UiOzuCache(List<TEntity> initialItems)ð
    {ð
        Initialize(initialItems);ð
    }ð

ð
    public void Initialize(List<TEntity> initialItems)ð
    {ð
        if (initialItems == null) throw new ArgumentNullException(nameof(initialItems));ð

ð
        lock (_lockObject)ð
        {ð
            _inMemoryItems.Clear();ð
            _inMemoryItems.AddRange(initialItems);ð
        }ð
    }ð

ð
    public void SetMutationStrategy(IOzuMutationStrategy<TEntity> strategy)ð
    {ð
        _mutationStrategy = strategy ?? throw new ArgumentNullException(nameof(strategy));ð
    }ð

ð
    public void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity)ð
    {ð
        // Aô>D$>C¢ !B4 AB <Cä=Cä?Cä;DÄ=Cä 7C EC$0D$Kc$0CTB Ct0CÄ>C¢ =C 2D 5CÄ0 CÄCD$0Dd8C•
        lock (_lockObject)ð
        {ð
            _mutationStrategy.ApplyDelta(_inMemoryItems, state, entity, GetEntityId);ð
        }ð
    }ð

ð
    public TResult ExecuteInLock<TResult>(Func<IReadOnlyList<TEntity>, TResult> evaluator)ð
    {ð
        if (evaluator == null) throw new ArgumentNullException(nameof(evaluator));ð
    }
}

```

```

    // A0>D$>C$ T' „&Æ |÷"' <Cä=Cä?Cä;DÄ=Cä 7C EC$0D$KC$0CTB Ct0CÄ>C$ =C 2D 5CÄO DD8C´LD$@C FC,,8/C$KDt8D
    lock (_lockObject)
    {
        return evaluator(_inMemoryItems);
    }
}

private object? GetEntityId(TEntity item)
{
    Type? hasKeyInterface = item.GetType()
        .GetInterfaces()
        .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IDomainObjectHasKey<>));
    if (hasKeyInterface != null)
    {
        return hasKeyInterface.GetProperty("Id")?.GetValue(item);
    }

    return item.GetType().GetProperty("Id")?.GetValue(item);
}
}
</file>

<file path="Promatis.Net.UI/Data/IUiDataBroker.cs">
using Promatis.Net.Data;
namespace Promatis.Net.UI;
/// <summary>
/// A0>C0BD 0C0B C @Cä:CT@C 4C =C0KDRÄ CC0@C 2C´0DäICT3Cä 2D´1Cä@Cä< C,,AD$>Dt=C,,:C ...6W'fW"öð¥R' 8 D BD 0D$5
/// A05 D >CD5D 6C,,B CD5DD>C´BC0KDR @CT6C,,<Cä2 – C0@Cä3D 0CÄ<C,,AD" >C 0Ct0C0 OC$=Cä AC0>C0DC,,3D4@C,,@Cä2C BDÄ
/// </summary>
public interface IUiDataBroker<TEntity, TQueryState, TResultData> : IDisposable where TEntity :
class
{
    /// <summary>
    /// A0@C,,7C00C$Ä CC00CtKC$0DäIC,,9, DtBCä BCT:D4IC,,9 C,,=D BC =D 1D >C05D 0 C00 DD>D <CR @C 1CäBC 5D" 2 D
    /// </summary>
    bool IsInMemoryMode { get; }

    /// <summary>
    /// Ad5D BC00D0 :Cä=DD8C4CD 0Dd8D0 @C 1CäBD² =C ?D 0CÄCDâ A B #A / gRPC API (Server Mode).
    /// </summary>
    void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider);

    /// <summary>
    /// Ad5D BC00D0 :Cä=DD8C4CD 0Dd8D0 @C 1CäBD² GCT@CT7 C,,7Cä;C,,@Cä2C =C0>CR ;Cä:C ;DÄ=Cä5 Aä B20ED 0C08C´8D
    /// </summary>
    void ConfigureInMemoryMode(
        IUiOzuCache<TEntity> ozuCache,
        Func<TQueryState, CancellationToken, Task<List<TEntity>>> serverDataLoader, // A05D 5CD0CT< Ct=C =C,,5
        Func<TQueryState, IReadOnlyList<TEntity>, TResultData> inMemoryEvaluator // B 0C >D$0CTB D •&V DöæÇ"
    );

    /// <summary>
    /// B4=C,,2CT@D 0C´LC00D0 BCäGC00 C AC,,=DT@Cä=C0>C4> Ct0C0@CäAC 4C =C0KDR 2C,,7D40C´8Ct0D$>D 0CÄ8 C,,=D$5D
    /// ATAC´8 C08 Cä4C,,= D 5Cd8CÄ =CR 1D´; D :Cä=DD8C4CD 8D >C$0C0 ?D >C4@C <CÄ8D BCä<, C$Kct>C$5D" -çf Æ-D÷
    /// </summary>
    Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken cancellationToken =
default);

    /// <summary>
    /// B$>Dt:C 2DT>CD0 CD;D0 ?D >C @CäAC 3C´>C 0C´LC0>C4> D 8C4=C ;C öävÇF–G"6ö0Ö–GFVB 2 C´>C0C´LC0KC´ :
    /// </summary>
    void HandleDatabaseCommit(EntityStateChangeEnum state, object entity);
}
</file>

<file path="Promatis.Net.UI/Data/UiDataBroker.cs">
using Promatis.Net.Data;
namespace Promatis.Net.UI;

```

```

/// <summary>D
/// Aô;C BDD>D <CT=CÔKC' 1D >C¸5D 4C =CÔKDRâ &CT=D$@C ;C,,7Cä2C =CÔ> D4?D 0C$;Dô5D" ?CäAD$0C$:Cä9 CD0CÔ=D'E C
/// Aô>C' =CäAD$LDâ 0C AD$@C 3C,,@D45D" T' >D" :Cä=C¸@CTBCÔKDR 8CÔBCT@DD5C"ACä2 CD>CÄ5CÔ=D'E D ;D46C 1DÔ:CT=CD
/// </summary>D
/// <typeparam name="TEntity">B$8Cò 4Cä<CT=CÔ>C' AD4ICÔ>D BCfÂ÷G- W & Ói
public class UiDataBroker<TEntity, TQueryState, TResultData> : IUiDataBroker<TEntity, TQueryState,
TResultData>D
    where TEntity : classD
{D
    private Func<TQueryState, CancellationToken, Task<TResultData>>? _serverDataProvider;D
    private Func<TQueryState, CancellationToken, Task<List<TEntity>>>? _serverDataLoader;D
    private Func<TQueryState, IReadOnlyList<TEntity>, TResultData>? _inMemoryEvaluator;D
    private IUiOzuCache<TEntity>? _ozuCache;D
    private readonly Action? _onDataChangedNotifier;D

D
    private bool _isInitialized;D
    private bool _isInMemoryMode;D
    private bool _isOzuLoaded;D

D
    // A,,=DD@C AD$@D4:D$CD 0 CD;Dò 4CT1C CCÔAC ACTBCT2D'E D42CT4Cä<C'5CÔ8C' 2 ServerModeD
    private CancellationTokenSource? _debounceCts;D
    private readonly object _debounceLock = new(); // At0CÄ>C¸ 4C'0 Cô>D$>C¸>C 5Ct>Cô0D =Cä3Cä AC @CäAC BC 9
    private const int DebounceDelayMs = 300; // A$@CT<CT=CÔ>CR >C¸=Cä >Cd8CD0C08Dò 7C BC,,HDÄO C" AB f3 <D

D
    public bool IsInMemoryMode => _isInMemoryMode;D

D
    public UiDataBroker(Action? onDataChangedNotifier = null)D
    {D
        _onDataChangedNotifier = onDataChangedNotifier;D
        DatabaseTriggerService.OnEntityCommitted += HandleGlobalEntityCommitted;D
    }D

D
    private void HandleGlobalEntityCommitted(EntityStateChangeEnum state, object entity)D
    {D
        HandleDatabaseCommit(state, entity);D
    }D

D
    public void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider)D
    {D
        _serverDataProvider = serverDataProvider ?? throw new
ArgumentNullException(nameof(serverDataProvider));D
        _serverDataLoader = null;D
        _inMemoryEvaluator = null;D
        _ozuCache = null;D
        _isInMemoryMode = false;D
        _isInitialized = true;D
        _isOzuLoaded = false;D
    }D

D
    public void ConfigureInMemoryMode(D
        IUiOzuCache<TEntity> ozuCache,D
        Func<TQueryState, CancellationToken, Task<List<TEntity>>> serverDataLoader,D
        Func<TQueryState, IReadOnlyList<TEntity>, TResultData> inMemoryEvaluator)D
    {D
        _ozuCache = ozuCache ?? throw new ArgumentNullException(nameof(ozuCache));D
        _serverDataLoader = serverDataLoader ?? throw new
ArgumentNullException(nameof(serverDataLoader));D
        _inMemoryEvaluator = inMemoryEvaluator ?? throw new
ArgumentNullException(nameof(inMemoryEvaluator));D
        _serverDataProvider = null;D
        _isInMemoryMode = true;D
        _isInitialized = true;D
        _isOzuLoaded = false;D
    }D

D
    public async Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken
cancellationToken = default)D
    {D
        if (!_isInitialized)D
        {D
            throw new InvalidOperationException($"A @Cä:CT@ CD0CÔ=D'E CD;Dò w.G- Vöb...DVÇF-G''äæ ÖWÖr =CR 1D';
        }D
    }D

D
    if (_serverDataProvider != null)D
    {D

```

```

        return await _serverDataProvider(state, cancellationToken);
    }

    if (_isInMemoryMode && _ozuCache != null && _inMemoryEvaluator != null &&
        _serverDataLoader != null)
    {
        if (!_isOzuLoaded)
        {
            List<TEntity> initialData = await _serverDataLoader(state, cancellationToken);
            _ozuCache.Initialize(initialData);
            _isOzuLoaded = true;
        }

        return _ozuCache.ExecuteInLock(items => _inMemoryEvaluator(state, items));
    }

    throw new InvalidOperationException("A@C,,BC,,GCTAC=8C' AC >C' 2C0CD$@CT=C05C' :Cä=DD8C4CD 0Dd8C, 1D >");
}

public void HandleDatabaseCommit(EntityStateChangeEnum state, object entity)
{
    // A0@Cä2CT@D05CÄ¢ >D$=CäAC,,BD O C'8 C0@C,,;CTBCT2D,,8C' 8Cr !B4 AB >C JCT:D" : D$8C0C CD0C0=D'E D$5C=0
    if (entity is not TEntity typedEntity) return;

    if (_isInMemoryMode)
    {
        // B Ad AÂ "â0T0ð%"¢ D4BC,,@D45CÂ At# CÄ3C0>C$5C0=Câ 1CT7 Ct0CD5D 6CT: CD;D0 ACäED 0C05C08D0 F
        if (_ozuCache != null && _isOzuLoaded)
        {
            _ozuCache.ApplyOzuDelta(state, typedEntity);
        }

        // B @C 7D2 ?C,,=C 5CÂ T'Â BC : C=0C¢ ACTBCT2Cä3Câ 7C ?D >D 0 C05 C CCD5D" r 2D'GC,,AC'5C08D0 ?D >C
        _onDataChangedNotifier?.Invoke();
    }
    else
    {
        // B Ad AÂ 4U%du"00ðDS¢ C¢;DäGC 5CÂ 4CT1C CC0A CD;D0 ?D 5CD>D$2D 0D"5C08D0 ACTBCT2Cä3Câ HD$>D <
        lock (_debounceLock)
        {
            // ATAC'8 C0@CT4D'4D4IC,,9 D$0C"<CT@ CTICR BC,,:C ; (C00Dt:C 8Ct<CT=CT=C,,9 C0@Cä4Cä;Cd0CTBD 0)
            _debounceCts?.Cancel();
            _debounceCts?.Dispose();

            // B >Ct4C 5CÂ =Cä2D'9 D$>C=5C0 7C 4CT@Cd:C•
            _debounceCts = new CancellationTokenSource();
            var token = _debounceCts.Token;

            // At0C0CD :C 5CÂ DCä=Cä2Cä5 Cä6C,,4C =C,,5 Ct0D$8D,,LD0 2 A 0
            Task.Delay(DebounceDelayMs, token).ContinueWith(task =>
            {
                // A$KctKc$0CT< C08C0>C¢ T' BCä;DÄ:Cä 5D ;C, 7C 4C GC CD ?CTHC0> Ct0C$5D HC,,;C ADÂ 8 C05
                if (task.Status == TaskStatus.RanToCompletion && !token.IsCancellationRequested)
                {
                    _onDataChangedNotifier?.Invoke();
                }
            }, TaskScheduler.Default);
        }
    }
}

public void Dispose()
{
    // AäBC08D KC$0CT<D O CäB D BC BC,,GCTAC=4C> D02CT=D$0 D ;D46C K D$@C,,3C45D >C" !B4 AM
    DatabaseTriggerService.OnEntityCommitted -= HandleGlobalEntityCommitted;

    // At0Dt8D"0CT< C,,=DD@C AD$@D4:D$CD C CD5C 0D4=D 0 C0@C, CC08DtBCä6CT=C,,8 C @Cä:CT@C DCä@CÄ>C•
    lock (_debounceLock)
    {
        _debounceCts?.Cancel();
        _debounceCts?.Dispose();
        _debounceCts = null;
    }
}
}
</file>

```

```

<file path="Promatis.Net.UI/Extensions/MudColorExtensions.cs">
using MudBlazor;
{
namespace Promatis.Net.UI;
{
public static class MudColorExtensions
{
    public static Color GetActionColor(this string? action)
    {
        if (string.IsNullOrEmpty(action)) return Color.Default;

        return action.ToLower().Trim() switch
        {
            "create" or "insert" or "added" or "CD>C 0C$;CT=C,,5" or "D >Ct4C =C,,5" => Color.Success,
            "update" or "edit" or "modified" or "C,,7CÄ5C05C08CR" ÷" $@CT4C :D$8D >C$0C08CR" Óâ 6ÖÆ÷"âv &æ-ærÍ
            "delete" or "remove" or "deleted" or "softdeleted" or "D44C ;CT=C,,5" => Color.Error,
            _ => Color.Default
        };
    }
}
</file>

<file path="Promatis.Net.UI/Interfaces/IHasSelectedData.cs">
namespace Promatis.Net.UI.Components;
{
/// <summary>
/// B4=C,,2CT@D 0C´LC0KC´ 8C0BCT@DD5C"A-CÄ0D :CT@ CD;Dò 2C,,7D40C´8Ct0D$>D >C"Â :CäBCä@D´< C05Cä1DT>CD8CÄ> D
/// D 8C0ED >C08Ct8D >C$0D$L DD>C=CD AD$@Cä:C,0Cct;C 8 Cö>C´CDt0D$L C,,<C0CC´LD K C05D 5D 8D >C$:C,i
/// </summary>
public interface IHasSelectedData<TEntity> where TEntity : class
{
    TEntity? SelectedData { get; set; }
    Action? OnContextUpdated { get; set; }
}
</file>

<file path="Promatis.Net.UI/IUiModule.cs">
namespace Promatis.Net.UI;
{
/// <summary>
/// A>C0BD 0C= B CD;Dò 4C,,=C <C,,GCTAC=8DR <Cä4D4;CT9 Cö>C´Lct>C$0D$5C´LD :Cä3Cä 8C0BCT@DD5C"AC 1
/// </summary>
public interface IUiModule
{
    /// <summary>
    /// AäBCä1D 0Cd0CT<Cä5 C,,<Dò <Cä4D4;Dò 2 D 8D BCT<CR ,,8D ?Cä;DÄ7D45D$ADò 4C´O D :C =CT@C ´í
    /// </summary>
    string Name { get; }

    /// <summary>
    /// A$>Ct2D 0D"0CTB D ?C,,ACä: D0;CT<CT=D$>C" =C 2C,,3C FC,,8 CD;Dò 1Cä:Cä2Cä3Cä <CT=Dä ×VDG& vW"í
    /// Bt5D$2CT@D$KC´ ?C @C <CTBD >D$2CTGC 5D" 7C "0CD >C$=CT2D4N C4@D4?C08D >C$:D2 MC´5CÄ5C0BCä2.D
    /// </summary>
    /// <returns>A>C´;CT:Dd8Dò :Cä@D$5Cd5C"¢ ,, C 7C$0C08CRÄ !D KC´:C Ä C= C0:C Ä C 7C$0C08CT D CC0?D²"Ä÷&W
    IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems();
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogContext.cs">
using Microsoft.Extensions.DependencyInjection;
using MudBlazor;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Pages.AuditLogs.Toolbar;
{
namespace Promatis.Net.UI.Pages.AuditLogs;
{
/// <summary>
/// A>C0BCT:D B D BD 0C08DdK C´>C48D >C$0C08Dò 0D44C,,BC ä
/// BD>C=CD 8D CCTBD O C,,AC=;DäGC,,BCT;DÄ=Cä =C ;Cä3C,,:CR DC,,;DÄBD 0Dd8C, 8 CÄ0C0?C,,=C45 C00D 0CÄ5D$@Cä2 C0>C
/// </summary>
public class AuditLogContext : GridContext<AuditLog, Guid>
{
    private readonly IAuditLogService _auditLogService;
}
}

```

```

private readonly Action _onFilterChanged; // A,,!Aô A A' Aô : B =Cä2C AD$@Cä3Cä5 readonly D 2Cä9D BC$>D
D
public override string TopZoneHeight => "48px";D
D
/// <summary>D
/// A,,!Aô A A' Aô : A=CôAD$@D4:D$>D ?D 8Cô8CäôCTB D BD >C4> Action C=C';C 5C¢ 4C'0 Cô@Dô<Cä9 D 5C :D$
/// Aô8C=0C=8DR 7Cô0Dt5Cô8C' ?Cä CCÄ>C'GC =C,,N (= null) CD;Dò D'ô2C ;C,,4C FC,,8 C >C'LD,,5 Cô5D"
/// </summary>D
public AuditLogContext(IServiceProvider serviceProvider, Action onFilterChanged)D
: base(serviceProvider, isInMemoryMode: false, onDataChangedNotifier: onFilterChanged)D
{D
    _onFilterChanged = onFilterChanged ?? throw new
ArgumentNullException(nameof(onFilterChanged));D
    _auditLogService = serviceProvider.GetRequiredService<IAuditLogService>();D
D
    // B @C 7D2 =C ?Cä;CôOCT< D$CC'1C @ D BC BC,,GCTAC=8CÄ8 Dô;CT<CT=D$ôCÄ8D
    AddControl(new AuditToolBarDivider());D
    AddControl(new AuditExportButton());D
}D
D
/// <summary>D
/// A,,!Aô A A' Aô : A$ôD,,0 Cä@C,,3C,,=C ;DÄ=C O CD8Cô0CÄ8Dt5D :C O CD>D >C 8D :C BD4;C ôD 0.D
/// A$KctKC$ôCTBD O C,,7 OnAfterRenderAsync Razor-D BD 0Cô8DdK, C=C44C MC=@C = D46CR 3C @C =D$8D >C$ôCô=
/// </summary>D
public void InitializeFilters(List<string> entityNames)D
{D
    // Aô>D$>C=C 5Ct>Cô0D =Cä GCT@CT7 C,,=C=0CôAD4;C,,@Cä2C =CôKC' <CTBCä4 C 0Ct>C$>C4> C=C AD 0 (C,,;C, =
    // C$>Ct2D 0D"ôCT< Cô0D$8C$=D4N C$AD$ôC$:D2 DC,,;DÄBD >C" 2 D 0CÄ>CR =C GC ;Cä BD4;C ôD 0 D ;CT2C =C
    _controls.Insert(0, new AuditEntitySelect(entityNames, _onFilterChanged));D
    _controls.Insert(1, new AuditActionSelect(_onFilterChanged));D
    _controls.Insert(2, new AuditPeriodPicker(_onFilterChanged));D
D
    NotifyStateChanged();D
}D
D
protected override async Task<GridData<AuditLog>> FetchDataFromServerAsync(GridState<AuditLog>
state, CancellationToken ct)D
{D
    // A,,!Aô A A' Aô : A$>Ct2D 0D"ôCT< C$ôD, >D 8C48Cô0C'LCôKC'Â @C 1CäBC ND"8C' ?Cä8D : C=CôôBD >C'>C"
    IUiControl? entityControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_entity");D
    IUiControl? actionControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_action");D
    IUiControl? periodControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_period");D
D
    string? selectedEntity = entityControl is IHasValue ev ? ev.Value as string : null;D
    DateRange? selectedPeriod = periodControl is IHasValue pv ? pv.Value as DateRange : null;D
    string? selectedAction = actionControl is AuditActionSelect av ?
av.GetSelectedActionValue() : null;D
D
    if (selectedEntity == "A$ACR AD4ICô>D BC,"•
        selectedEntity = null;D
D
    DateTime fromDate = selectedPeriod?.Start ?? DateTime.Today.AddDays(-7);D
    DateTime toDate = selectedPeriod?.End ?? DateTime.Today;D
D
    fromDate = DateTime.SpecifyKind(fromDate.Date, DateTimeKind.Utc);D
    toDate = DateTime.SpecifyKind(toDate.Date.AddDays(1).AddTicks(-1), DateTimeKind.Utc);D
D
    var searchRequest = new AuditLogSearchRequest(D
        FromDate: fromDate,D
        ToDate: toDate,D
        EntityName: selectedEntity,D
        Action: selectedAction,D
        PageIndex: state.Page,D
        PageSize: state.PageSizeD
    );D
D
    PagedResult<AuditLog> pagedResult = await _auditLogService.SearchLogsAsync(searchRequest,
ct);D
D
    return new GridData<AuditLog>D
    {D
        Items = pagedResult.Items,D
        TotalItems = pagedResult.TotalCountD
    };D
}D
D

```



```

        protected override Task OpenDialogWindowAsync(AuditLog model, bool isNew) => Task.CompletedTask;
    }
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor">
@page "/audit-logs"
@using Promatis.Net.Domain
@
<CascadingValue Value="Context">
    <WorkspacePage>
@
        <!-- 1. A$ B %A0/B0 Aä A ¢ C$BCä<C BC,,GCTAC¤8C' ?Cä;C,,<Cä@DD=D´9 D$CC´1C @ (AD8C00CÄ8Dt5D :C,,5 C¤>C
        <TopContent>
            <UiToolbar Controls="@Context.Controls" />
        </TopContent>
@
        <!-- 2. Bd A0"B A´,A0 B0 Aä A ¢ "C 1C´8Dt=C O D 5D$:C Â @C 1CäBC ND"0D0 2 DD>C0>C$>CÂ ACT@C$5D =Cä<
        <BodyContent>
            <GridPage TEntity="AuditLog"
                @ref="_grid"
                ServerDataProvider="@Context.GetDataAsync"
                WithPagination="true"
                RowsPerPage="20">
                <GridColumns>
                    <PropertyColumn T="AuditLog" TProperty="DateTime" Property="x => x.ChangedAt"
Title="AD0D$0 C, 2D 5CÄ0" Format="dd.MM.yyyy HH:mm:ss" HeaderStyle="width: 200px;" />
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangedBy"
Title="A0>C´Lct>C$0D$5C´L" HeaderStyle="width: 150px;" />
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.EntityName"
Title="B$8C0 AD4IC0>D BC," ¤V FW%7G-ÆS0'v-GFf¢ # f²" óí
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.Action"
Title="Aä?CT@C FC,,0" HeaderStyle="width: 120px;" />
                    <PropertyColumn T="AuditLog" TProperty="Guid" Property="x => x.EntityId"
Title="ID B CD"=CäAD$8" HeaderStyle="width: 330px;" />
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangesJson"
Title="A,,7CÄ5C05C08D0 „¥4ôâ'" ¤V FW%7G-ÆS0'v-GFf¢ WFó²" 6Æ 730'FW‡B×G'Væ6 FR" óí
                </GridColumns>
            </GridPage>
        </BodyContent>
@
    </WorkspacePage>
</CascadingValue>
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs">
using Microsoft.AspNetCore.Components;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Promatis.Net.UI.Components.Workspaces;
@
namespace Promatis.Net.UI.Pages.AuditLogs;
@
public partial class AuditLogPage : ComponentBase
{
    private GridPage<AuditLog>? _grid;
    private bool _isLoading;
@
    [Inject]
    protected IServiceProvider PageServiceProvider { get; set; } = null!;
@
    [Inject]
    protected IAuditLogService AuditLogService { get; set; } = null!;
@
    protected AuditLogContext Context { get; set; } = null!;
@
    protected override void OnInitialized()
    {
        base.OnInitialized();
@
        // AÄ A0 A$ A0 Aä ACä7CD0CT< C0CD BCä9 C¤>C0BCT:D B. B BD 0C08Dd0 D 5C04CT@C,,BD O C 5Cr 7C 4CT@Cd5C¢
        Context = new AuditLogContext(PageServiceProvider, onFilterChanged: RefreshGrid);
    }
@
    /// <summary>
    /// A¤0C0>C08Dt5D :C O D$>Dt:C 7C ?D4AC¤0 D$0Cd5C´KDR 7C ?D >D >C"â !D 0C 0D$KC$0CTB A0 B AR BCä3CâÂ

```

```

/// C#0C# ?Cä;DÄ7Cä2C BCT;DÄ CCd5 D42C,,4CT; C4>D$>C$CDä AD$@C =C,,FD2 =C MC#@C =CRi
/// </summary>#
protected override async Task OnAfterRenderAsync(bool firstRender)#
{#
    await base.OnAfterRenderAsync(firstRender);#
#
    // A$KC#>C´=D#5CÄ 7C 3D CCT:D2 BCä;DÄ:Cä >CD8C# @C 7 C#@C, ?CT@C$>CÄ @CT=CD5D 5 D#D:D #C#0#
    if (firstRender)#
    {#
        // B ?Cä:Cä9C#> D :C GC,,2C 5CÄ CC#8C#0C´LC#KCR 8CÄ5C#0 D CD´=CäAD$5C´ 8Cr !B4 AB ÷7Fw&U5 Í
        List<string> availableEntities = await AuditLogService.GetAvailableEntityNamesAsync();#
#
        // A#5D 5CD#0CT< CD#0C#D´5 C" :Cä=D$5C#AD" 4C´O CD8C#0CÄ8Dt5D :Cä9 C45C#5D #Dd8C, ACT;CT:D$# C, ?C
        Context.InitializeFilters(availableEntities);#
#
        _isLoading = true;#
        StateHasChanged(); // A#5D 5D 8D >C$KC$#0CT< D$CC´1C @ C, 7C ?D4AC#0CT< C#5D 2D4N Ct#C4@D47C#C C´>
    }#
#
protected void RefreshGrid()#
{#
    // At#C#0@CTIC 5CÄ ?D 8C#CCD8D$5C´LC#>CR >C =Cä2C´5C#8CR 3D 8CD# CäB D$@C,,3C45D >C" DC,,;DÄBD >C"Í
    // C#>C#0 D#BC, DC,,;DÄBD K CTICR ?CT@C$8Dt=Cä =CR =C AD$@Cä5C#K C" öä gFW%&VæFW$ 7-æ=
    if (_isLoading && _grid != null)#
    {#
        _ = _grid.ReloadServerData();#
    }#
#
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditActionSelect.cs">
using Promatis.Net.UI.Components;#
using Promatis.Net.UI.Components.ElementRenderBase;#
#
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;#
#
public class AuditActionSelect : BaseUiControl, IHasValue, IHasOptions#
{#
    private readonly Action _onChanged;#
    private readonly Dictionary<string, string> _actionMapping = new()#
    {#
        { "A$ACR >C#5D #Dd8C,"Ä "" #Í
        { "B >Ct4C =C,,5", "Added" },#
        { "A,,7CÄ5C#5C#8CR"Ä $#F-f-VB" #Í
        { "B44C ;CT=C,,5", "Deleted" },#
        { "Ä#C4:Cä5 D44C ;CT=C,,5", "SoftDeleted"}#
    };#
#
    public override string Id => "audit_filter_action";#
    public override Type ComponentType => typeof(SelectRenderBase);#
    public override string Title => "Ä?CT@C FC,,0";#
#
    public object? Value { get; set; }#
    public IEnumerable<string> Options => _actionMapping.Keys;#
#
    public AuditActionSelect(Action onChanged)#
    {#
        _onChanged = onChanged;#
        Value = "A$ACR >C#5D #Dd8C,#² #D CTDCä;D$=Cä5 C$KC @C =C#>CR BCT:D BCä2Cä5 Ct=C GCT=C,,5D
    }#
#
    // Ä#5D$>CB 4C´O C#>C´CDt5C#8D# AC,,AD$5CÄ=Cä3Cä 7C#0Dt5C#8D# „AD$@Cä:C, 4C´O API) C,,7 C$KC @C =C#>C4> D$5
    public string? GetSelectedActionValue()#
    {#
        if (Value is string selectedText && _actionMapping.TryGetValue(selectedText, out string?
sysValue))#
        {#
            return string.IsNullOrEmpty(sysValue) ? null : sysValue;#
        }#
        return null;#
    }#
#
    protected override Task HandleTriggerAsync(object? targetData)#
    {#

```

```

        _onChanged.Invoke();
        return Task.CompletedTask;
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditEntitySelect.cs">
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;

public class AuditEntitySelect : BaseUiControl, IHasValue, IHasOptions
{
    private readonly Action _onChanged;

    public override string Id => "audit_filter_entity";
    public override Type ComponentType => typeof(SelectRenderBase);
    public override string Title => "B$8Cò AD4ICò>D BC,#½

    public object? Value { get; set; }
    public IEnumerable<string> Options { get; set; }

    public AuditEntitySelect(List<string> availableEntities, Action onChanged)
    {
        _onChanged = onChanged;
        Value = "A$ACR AD4ICò>D BC,#½

        // BD>D <C,,@D45CÂ 4C,,=C <C,,GCTAC=8C' ACô8D >C¢ >CôFC,,9 Cò0 CăACô>C$5 CD0CÔ=D´E C,,7 A  D
        var list = new List<string> { "A$ACR AD4ICò>D BC," Ó½
        list.AddRange(availableEntities);
        Options = list;
    }

    protected override Task HandleTriggerAsync(object? targetData)
    {
        _onChanged.Invoke();
        return Task.CompletedTask;
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditExportButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;

public class AuditExportButton : BaseUiControl
{
    public override string Id => "audit_action_export";
    public override Type ComponentType => typeof(ButtonRenderBase); // B 5Cô4CT@CT@ Dô4D 0D
    public override string Title => "A$KC4@D47C,,BDÂ#½
    public override string Icon => Icons.Material.Filled.Download;
    public override string Tooltip => "A$KC4@D47C,,BDÂ >D$DC,,;DÂBD >C$0CÔ=D´5 C´>C48 C" W†6VÂ#½

    public AuditExportButton()
    {
        Alignment = UIControlAlignment.Right; // B 4C$8C40CT< C= Că?C=C Dô:D ?Că@D$0 C$?D 0C$>D

    }

    protected override async Task HandleTriggerAsync(object? targetData)
    {
        // A,,<C,,BC,,@D45CÂ 4Că;C4CDâ 2D´3D CCT:D2 r :CÔ>Cô:C 0C$BCă<C BC,,GCTAC=8 Ct0C ;Că:C,,@D45D$ADò 8 Cò>C=
        await Task.Delay(2000);
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditPeriodPicker.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;

```

```

    public class AuditPeriodPicker : BaseUiControl, IHasValue
    {
        private readonly Action _onChanged;

        public override string Id => "audit_filter_period";
        public override Type ComponentType => typeof(DateRangeRenderBase); // B 5C04CT@CT@ D04D 00
        public override string Title => "A05D 8Cä4 C´>C4>C"#½

        public object? Value { get; set; }

        public AuditPeriodPicker(Action onChanged)
        {
            _onChanged = onChanged;
            Value = new DateRange(DateTime.Today.AddDays(-7), DateTime.Today);
        }

        protected override Task HandleTriggerAsync(object? targetData)
        {
            _onChanged.Invoke(); // A05D 8Cä4 C,,7CÄ5C08C´ADò r 4C 5CÄ 8CÄ?D4;DÄA D BD 0C08Dd5D
            return Task.CompletedTask;
        }
    }
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditToolbarDivider.cs">
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;
public class AuditToolbarDivider : BaseUiControl
{
    private readonly string _id = "audit_toolbar_divider_" + Guid.NewGuid().ToString("N");

    public override string Id => _id;
    public override Type ComponentType => typeof(DividerRenderBase); // A,,AC0>C´LctCCT< D$>D" 6CR 1C 7Cä2D´9

    public AuditToolbarDivider()
    {
        Alignment = UiControlAlignment.Right; // B0BCäB D 0Ct4CT;C,,BCT;DÄ 3C @C =D$8D >C$0C0=Cä AC0@C 2C
    }

    protected override Task HandleTriggerAsync(object? targetData)
    {
        return Task.CompletedTask;
    }
}
</file>

<file path="Promatis.Net.UI/Pages/Dashboard.razor">
@page "/"
<MudText Typo="Typo.h4" Class="mb-4">AÄ>CDCC´L Master Data Management</MudText>
<MudGrid>
    <MudItem xs="12" sm="6" md="4">
        <MudPaper Class="d-flex flex-row pt-6 pb-4 px-4" Style="height:100px;">
            <MudIcon Icon="@Icons.Material.Filled.Storage" Color="Color.Primary" Size="Size.Large"
Class="mr-4" />
            <div>
                <MudText Typo="Typo.subtitle2" Class="mud-text-secondary">B BC BD4A C0>CD:C´NDt5C08D0 : A </
                <MudText Typo="Typo.h6">A :D$8C$=Cä ...FW7D& 6R"Ä0×VEFW†Cí
            </div>
        </MudPaper>
    </MudItem>
    <MudItem xs="12" sm="6" md="8">
        <MudPaper Class="pa-4" Style="height:100px;">
            <MudText Typo="Typo.subtitle1">AD>C @Cä ?Cä6C ;Cä2C BDÄ 2 D 8D BCT<D2 &ö F–2 U% Ä0×VEFW†Cí
            <MudText Typo="Typo.body2" Class="mud-text-secondary">
                A$ACR 8C0BCT@DD5C"AD² ?Cä4C4@D46CT=D² 4C,,=C <C,,GCTAC=8 C,,7 C05Ct0C$8D 8CÄKDR AC´>CT2 Razor CL
            </MudText>
        </MudPaper>
    </MudItem>
</MudGrid>

```

</file>

```
<file path="Promatis.Net.UI/Pages/Dashboard.razor.css">
.my-component {
    border: 2px dashed red;
    padding: 1em;
    margin: 1em 0;
    background-image: url('background.png');
}
</file>
```

```
<file path="Promatis.Net.UI/Promatis.Net.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
    <ItemGroup>
        <Compile Remove="AuditLog\*" />
        <Compile Remove="Components\References\*" />
        <Compile Remove="Models\*" />
        <Content Remove="AuditLog\*" />
        <Content Remove="Components\References\*" />
        <Content Remove="Models\*" />
        <EmbeddedResource Remove="AuditLog\*" />
        <EmbeddedResource Remove="Components\References\*" />
        <EmbeddedResource Remove="Models\*" />
        <None Remove="AuditLog\*" />
        <None Remove="Components\References\*" />
        <None Remove="Models\*" />
    </ItemGroup>
    <ItemGroup>
        <SupportedPlatform Include="browser" />
    </ItemGroup>
    <ItemGroup>
        <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />
        <PackageReference Include="Microsoft.AspNetCore.Http.Abstractions" Version="2.3.10" />
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\
MesCore\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
        <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />
        <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />
    </ItemGroup>
</Project>
</file>
```

```
<file path="Promatis.Net.UI/Theme/PromatisTheme.cs">
using MudBlazor;
namespace Promatis.Net.Ui.Theme;
public static class PromatisTheme
{
    public static MudTheme DefaultTheme => new()
    {
        PaletteLight = new PaletteLight
        {
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' :Cä@Cö>D 0D$8C$=D'9 Dd2CTB (Cα=Cä?Cα8, C :D$
            Secondary = Colors.Blue.Lighten2, // A$ACö>CÄ>C40D$5C'LC0KC' FC$5D" „2D$>D >D BCT?CT=C0KCR MC

            AppBarBackground = "#EAEAEA", // BD>C0 2CT@DT=CT9 C00C05C'8 (MudAppBar) C" 4C05C$=Cä< D 5
            AppBarText = "#2C3E50", // Bd2CTB D$5CαAD$0, Ct0C4>C'>C$:Cä2 C, 8Cα>C0>Cφ 2 C$5D EC
            DrawerBackground = "#EAEAEA", // BD>C0 1Cä:Cä2Cä3Cä <CT=Dâ =C 2C„3C FC„8 (MudDrawer)D
            Background = "#F9F9F9", // Aä1D"8C' DCä=Cä2D'9 Dd2CTB Cö>CD;Cä6Cα8 C$ACTE D BD 0C08
            Surface = Colors.Shades.White, // BD>C0 :Cä=D$5C"=CT@Cä2 MudPaper, Cα0D BCäGCT: MudCard C
        },
    }
}
```

```

        PaletteDark = new PaletteDark()
    {
        Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5 (CÄ0C4:C,,9 D 8C
        Secondary = Colors.Blue.Lighten2, // A$ACô>CÄ>C40D$5C'LC0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5D
    }

    AppBarBackground = "#0D0D0D", // BD>C0 2CT@DT=CT9 C00C05C'8 (MudAppBar) C" 3C'CC >C¤>CÄ G
    AppBarText = Colors.Shades.White, // Bd2CTB D$5C¤AD$0 C, 8C¤>C0>C¢ 2 D,,0C0:CR ,, :Cä=D$@C AD$=D

    Background = "#121212", // A4;D41Cä:C,,9 D$QCÄ=D'9 DD>C0 AD$@C =C,,F C0>CD;Cä6C¤8D
    Surface = "#1E1E1E", // A4@C DC,,BCä2D'9 DD>C0 4C'0 MudPaper, C¤>C0BCT=D$0 C$:C'0
    DrawerBackground = "#1E1E1E", // BD>C0 1Cä:Cä2Cä9 C00C05C'8 C00C$8C40Dd8C, 2 D$QCÄ=Cä9 D$
    DrawerText = "#E0E0E0", // Bd2CTB D AD';Cä: C, BCT:D BC 2 C >C¤>C$>CÄ <CT=Dí
    TextPrimary = "#E0E0E0", // AäAC0>C$=Cä9 Dd2CTB D$5C¤AD$0 C00 D BD 0C08Dd0DR ,, <D03C¤
    TextSecondary = "#A0A0A0", // Bd2CTB CD;D0 2D$>D >D BCT?CT=C0>C4> D$5C¤AD$0 C, ?Cä4D

    },
    LayoutProperties = new LayoutProperties()
    {
        DrawerWidthLeft = "260px",
        DrawerWidthRight = "300px"
    }
};
}
</file>

<file path="Promatis.Net.UI/UiModule.cs">
using MudBlazor;
namespace Promatis.Net.UI;
public class UiModule : IUiModule
{
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";

    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()
    {
        return new List<(string Title, string Href, string Icon, string? Group)>
        {
            // B0;CT<CT=D" 2CT@DT=CT3Cä CD >C$=D0 ,,2C05 C00C0>C¢•
            ("A4;C 2C00D0"Ä "0"Ä -6öç2äÖ FW&- Ääf-ÆVBäF 6†&ö &BÄ çVÆÄ'Í
            // B 0Ct4CT; D 8D BCT<C0>C4> C 4CÄ8C08D BD 8D >C$0C08Dý
            ("AdCD =C ; C CCD8D$0", "/audit-logs", Icons.Material.Filled.History, "A 4CÄ8C08D BD 8D >C$0C08CR
        };
    }
}
</file>

<file path="Promatis.Net.UI/wwwroot/App.css">
html, body {
    font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;
}
a, .btn-link {
    color: #006bb7;
}
.btn-primary {
    color: #fff;
    background-color: #1b6ec2;
    border-color: #1861ac;
}
.btn:focus, .btn:active:focus, .btn-link.nav-link:focus, .form-control:focus, .form-check-
input:focus {
    box-shadow: 0 0 0 0.1rem white, 0 0 0 0.25rem #258cfb;
}
.content {
    padding-top: 1.1rem;
}
h1:focus {
    outline: none;
}
.valid.modified:not([type=checkbox]) {

```



```

        <Nullable>enable</Nullable>
    </PropertyGroup>

    <ItemGroup>ð
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />ð
    </ItemGroup>

    <ItemGroup>ð
        <ProjectReference Include="..\Data\Promatis.Net.Data.csproj" />ð
    </ItemGroup>

</Project>
</file>

<file path="Service/Services/AuditLogService/AuditLogService.cs">
using Microsoft.EntityFrameworkCore;ð
using Promatis.Net.Data;ð
using Promatis.Net.Domain;ð
ð
namespace Promatis.Net.Service;ð
ð
/// <summary>ð
/// B 5D 2C,,A CD;Dò @C 1CäBD² A C´>C40CÄ8 C CCD8D$0.ð
/// Aô>C´=CäAD$LDâ ACä2CÄ5D BC,,< D 1C 7Cä2D´< BaseService C, 0C$BCä<C BC,,GCTACð8 D 5C48D BD 8D CCTBD O D :C
/// </summary>ð
public class AuditLogService(IDbContextFactory<ApplicationDbContext> contextFactory)ð
    : BaseService<AuditLog, Guid, ApplicationDbContext>(contextFactory), IAuditLogServiceð
{ð
    private static List<string>? _cachedEntityNames;ð
    private static readonly SemaphoreSlim CacheLock = new(1, 1);ð
ð
    public async Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request,
        CancellationToken cancellationToken = default)ð
    {ð
        await using ApplicationDbContext context = await
        ContextFactory.CreateDbContextAsync(cancellationToken);ð
ð
        IQueryable<AuditLog> query = context.Set<AuditLog>().ð
            .AsNoTracking()ð
            .Where(l => l.ChangedAt >= request.FromDate && l.ChangedAt <= request.ToDate);ð
ð
        if (!string.IsNullOrEmpty(request.EntityName)) query = query.Where(l => l.EntityName
        == request.EntityName);ð
        if (!string.IsNullOrEmpty(request.Action)) query = query.Where(l => l.Action ==
        request.Action);ð
ð
        int totalCount = await query.CountAsync(cancellationToken);ð
ð
        if (totalCount == 0)ð
        {ð
            // A,,ACô>C´LCtCCT< Array.Empty C,,;C, ?D4AD$CDâ :Cä;C´5CðFC,,N C 5Cr ;C,,HCô8DR 0C´;Cä:C FC,,9 D ?C,,A
            return new PagedResult<AuditLog>([], 0);ð
        }ð
ð
        // A,,!Aô A A´ Aô : AD>C 0C$;CT=C 2D$>D 8Dt=C O D >D BC,,@Cä2Cð0 Cô> Id CD;Dò CD BD 0Cô5Cô8Dò MDDDCt:
        List<AuditLog> items = await queryð
            .OrderByDescending(l => l.ChangedAt)ð
            .ThenByDescending(l => l.Id)ð
            .Skip(request.PageIndex * request.PageSize)ð
            .Take(request.PageSize)ð
            .ToListAsync(cancellationToken);ð
ð
        return new PagedResult<AuditLog>(items, totalCount);ð
    }ð
ð
    public async Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken
        cancellationToken = default)ð
    {ð
        if (_cachedEntityNames != null)ð
        {ð
            return [.. _cachedEntityNames];ð
        }ð
ð
        await CacheLock.WaitAsync(cancellationToken);ð
        tryð
        {ð

```



```

        if (_cachedEntityNames != null)
        {
            return [... _cachedEntityNames];
        }

        await using ApplicationDbContext context = await
ContextFactory.CreateDbContextAsync(cancellationToken);

        _cachedEntityNames = await context.Set<AuditLog>()
            .AsNoTracking()
            .Select(l => l.EntityName)
            .Distinct()
            .OrderBy(name => name)
            .ToListAsync(cancellationToken);

        return [... _cachedEntityNames];
    }
    finally
    {
        CacheLock.Release();
    }
}

public static void InvalidateCache()
{
    CacheLock.Wait();
    try
    {
        _cachedEntityNames = null;
    }
    finally
    {
        CacheLock.Release();
    }
}
}
</file>

```

```

<file path="Service/Services/AuditLogService/IAuditLogService.cs">
using Promatis.Net.Domain;
namespace Promatis.Net.Service;

public interface IAuditLogService
{
    /// <summary>
    /// Aö>C,,AC¢ ;Cä3Cä2 C CCD8D$0 Cö> CD8C ?C 7Cä=D2 4C B D ?Cä4CD5D 6C¤>C' DC,,;DÄBD 0Dd8C, 8 Cö0C48Cö0Dd8C
    /// </summary>
    Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request, CancellationToken
cancellationToken = default);

    /// <summary>
    /// Aö>C'CDt5C08CR AC08D :C CC08C¤0C'LC0KDR 8CÄ5C0 AD4IC0>D BCT9, C¤>D$>D KCR 5D BDÂ 2 C'>C40DR ,,4C'0 C$
    /// </summary>
    Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken cancellationToken = default);
}
</file>

```

```

<file path="Service/Services/BaseService/BaseService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Service;

public abstract class BaseService<T, TKey, TContext>(IDbContextFactory<TContext> contextFactory)
    : IBaseService<T, TKey>
    where T : class, IDomainObjectHasKey<TKey>
    where TContext : DbContext
{
    protected readonly IDbContextFactory<TContext> ContextFactory = contextFactory ?? throw new
ArgumentNullException(nameof(contextFactory));

    public virtual async Task<T?> GetByIdAsync(TKey id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
    }
}

```

```

        return await context.Set<T>().FindAsync(id);
    }
}

public virtual async Task<List<T>> GetAllAsync(CancellationToken cancellationToken = default)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync(cancellationToken);

    // A,,!Aô A A´ Aô : B$>C=5Cò BD 0C0AC´8D CCTBD O C0C0@Dô<D4N C" 4D 0C"2CT@ C 0CtK CD0C0=D´E (EF Core
    return await context.Set<T>().ToListAsync(cancellationToken);
}

public virtual async Task<PagedResult<T>> GetPagedAsync(int pageIndex, int pageSize,
CancellationToken cancellationToken = default)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync(cancellationToken);

    // 1. B >Ct4C 5CÂ 1C 7Câ2D´9 Ct0C0@CâA C¢ BC 1C´8Dd5 D CD"=CâAD$8
    IQueryable<T> query = context.Set<T>();

    // 2. Aô>C´CDt0CT< Câ1D"5CR :Câ;C,,GCTAD$2Câ 7C ?C,,ACT9 C" AB 4C´O C0C48C00D$>D 0
    int totalCount = await query.CountAsync(cancellationToken);

    // 3. B >D BC,,@D45CÂ ?Câ ?CT@C$8Dt=Câ<D2 :C´NDtC (ID) CD;Dò 4CTBCT@CÂ8C08D >C$0C0=Câ3Câ ?Câ@Dô4C=0 D
    // B$0C¢ :C : D2 =C A CTAD$L Câ3D 0C08Dt5C08CR v†W&R B ¢ "FöÖ -äö&|V7D† 4¶W"AD¶W"âÂ <D² <Câ6CT< C 5Ct
    query = query.OrderBy(x => x.Id);

    // 4. AâBD 5Ct0CT< C0CCd=D4N D BD 0C08DdC D ?Câ<CâIDÄN Skip/Take C, <C BCT@C,,0C´8CtCCT< D ?C,,ACâ:D
    List<T> items = await query
        .Skip(pageIndex * pageSize)
        .Take(pageSize)
        .ToListAsync(cancellationToken);

    // 5. A$>Ct2D 0D"0CT< D4?C :Câ2C =C0KC´ @CT7D4;DÂBC B
    return new PagedResult<T>(items, totalCount);
}

public virtual async Task AddAsync(T entity)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync();
    await context.Set<T>().AddAsync(entity);
    await context.SaveChangesAsync();
}

public virtual async Task UpdateAsync(T entity)
{
    if (entity == null) throw new ArgumentNullException(nameof(entity));

    await using TContext context = await ContextFactory.CreateDbContextAsync();

    // 1. A,,7C$;CT:C 5CÂ A="B4 A´,Aô+A´ >D 8C48C00C´LC0KC´ >C JCT:D" 8Cr 1C 7D² 4C =C0KDR ?Câ 5C4> C05D
    // A ;C 3Câ4C @Dò 6öç7G& -çG2 C C00D 3C @C =D$8D >C$0C0=Câ 5D BDÂ 4CâAD$CCò : x.Id
    T? dbEntity = await context.Set<T>().FindAsync(entity.Id);

    if (dbEntity == null)
    {
        throw new InvalidOperationException($"Aô5 D44C ;CâADÂ >C =Câ2C,,BDÂ >C JCT:D#¢ AD4IC0>D BDÂ w-G- V
        CD0C0=D´E.");
    }

    // 2. B GC,,BD´2C 5CÂ <CTBC 4C =C0KCR >D$AC´5Cd8C$0C08Dò 4C´O Câ@C,,3C,,=C ;DÂ=Câ3Câ >C JCT:D$0 B #A D
    EntityEntry<T> entry = context.Entry(dbEntity);

    // 3. Aô5D 5C0>D 8CÂ 7C00Dt5C08Dò "Aâ BÂ Aâ B AÂ B$ A$ B% D 2Câ9D BC" 8Cr :C´>C00 C" >D 8C48C00C´L
    // AÂ5D$>CB 6WEf ÇVW2 0C$BCâ<C BC,,GCTAC=8 C0@Câ8C4=Câ@C,,@D45D" =C 2C,,3C FC,,>C0=D´5 D 2Câ9D BC$0 (Pare
    // C, :Câ;C´5C=FC,,8 CâBC0>D,,5C08C´Â :CâBCâ@D´5 C$0D, :C´>C05D 2D´@CT7C ;! Aâ= Câ1C0>C$8D" BCâ;DÂ:Câ
    entry.CurrentValues.SetValues(entity);

    // 4. AD>C0>C´=C,,BCT;DÂ=C O Ct0D"8D$0: C0@câ2CT@Dô5CÂÂ 8Ct<CT=C,,;CâADÂ ;C, ECâBDÂ GD$>-D$>.D
    // ATAC´8 C0>C´Lct>C$0D$5C´L CâBC=0D´; CD8C ;Câ3, C08Dt5C4> C05 CÂ5C00C² 8 C00Cd0C² !CâED 0C08D$L,D
    // CÂK C$>Câ1D"5 C05 C CCD5CÂ 4E @C40D$L D$@C =Ct0C=FC,,N PostgreSQL!
    if (entry.State == EntityState.Unchanged)
    {
        return;
    }

    // 5. EF Core D 3CT=CT@C,,@D45D" EC,,@D4@C48Dt5D :C,,9 SQL-Ct0C0@CâA, Câ1C0>C$8C" 2 A B$ A´,A= C,,7CÂ5

```

```

        // B CD"5D BC$CDäIC,,5 D 2Dô7C, 8 C,,7CÄ5C05C08Dò 4D CC48DR ?Cä;DÄ7Cä2C BCT;CT9 C" MD$>D" <Cä<CT=D" =CR
        await context.SaveChangesAsync();
    }
}

public virtual async Task DeleteAsync(TKey id)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync();
    T? entity = await context.Set<T>().FindAsync(id);
    if (entity != null)
    {
        context.Set<T>().Remove(entity);
        await context.SaveChangesAsync();
    }
}
}
</file>

<file path="Service/Services/BaseService/IBaseService.cs">
namespace Promatis.Net.Service;
{
    // A 0Ct>C$KC' 5%TM
    public interface IBaseService<T, in TKey> where T : class
    {
        Task<T?> GetByIdAsync(TKey id);

        Task<List<T>> GetAllAsync(CancellationToken cancellationToken = default);

        Task<PagedResult<T>> GetPagedAsync(int pageIndex, int pageSize, CancellationToken
        cancellationToken = default);

        Task AddAsync(T entity);

        Task UpdateAsync(T entity);

        Task DeleteAsync(TKey id);
    }
}
</file>

<file path="Service/Services/ReferenceService/IReferenceService.cs">
namespace Promatis.Net.Service;
{
    // AD;Dò ACô@C 2CäGC08C¤>C-
    public interface IReferenceService<T> : IBaseService<T, Guid> where T : class
    {
        Task<T?> GetByCodeAsync(string code);

        Task<List<T>> SearchByNameAsync(string namePart);
    }
}
</file>

<file path="Service/Services/ReferenceService/ReferenceService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
{
    namespace Promatis.Net.Service;
    {
        public abstract class ReferenceService<T, TContext>(IDbContextFactory<TContext> contextFactory)
            : BaseService<T, Guid, TContext>(contextFactory), IReferenceService<T>
            where T : ReferenceBase
            where TContext : DbContext
        {
            public async Task<T?> GetByCodeAsync(string code)
            {
                await using TContext context = await ContextFactory.CreateDbContextAsync();
                return await context.Set<T>().FirstOrDefaultAsync(x => x.Code == code);
            }

            public async Task<List<T>> SearchByNameAsync(string namePart)
            {
                await using TContext context = await ContextFactory.CreateDbContextAsync();
                return await context.Set<T>()
                    .Where(x => x.Name.Contains(namePart))
                    .ToListAsync();
            }
        }
    }
}
</file>

```

```

<file path="Service/Services/ReferenceTreeService/IReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
Ð
Ð
namespace Promatis.Net.Service;
Ð
public interface IReferenceTreeService<T, TContext> : IReferenceService<T>, ITreeService<T>Ð
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
    where TContext : DbContextÐ
{
    // B AC²2Cä7CÔKCÂ =C AC´5CD>C$0CÔ8CT< C,,=D$5D DCT9D >C" 2D Q D >D,,;CäADÂ
}
</file>

<file path="Service/Services/ReferenceTreeService/ReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
Ð
namespace Promatis.Net.Service;
Ð
/// <summary>Ð
/// B4=C,,2CT@D 0C´LCÔKC´ 1C 7Cä2D´9 D 5D 2C,,A CD;Dò @C 1CäBD² ACâ 2D 5CÄ8 CD@CT2Cä2C,,4CÔKCÄ8 D BD CC²BD4@C <C
/// AÔ0D ;CT4D45D" ;Cä3C,,;D2 >C KDt=D´E D ?D 0C$>Dt=C,,;Cä2 (ReferenceService) C, @C AD,,8D OCTB CTQ CD;Dò 8CT@
/// </summary>Ð
/// <typeparam name="T">B$8Cò AD4ICÔ>D BC,Â CCÔ0D ;CT4Cä2C =CÔKC´ >D" &VfW&Væ6UG&VT& 6RäÂ÷G- W & Óí
/// <typeparam name="TContext">A²>CÔBCT:D B C 0CtK CD0CÔ=D´E.</typeparam>Ð
public abstract class ReferenceTreeService<T, TContext>(IDbContextFactory<TContext> contextFactory)Ð
    : ReferenceService<T, TContext>(contextFactory), IReferenceTreeService<T, TContext>Ð
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
    where TContext : DbContextÐ
{
    private TreeServiceProxy? _treeServiceProxy;
    private TreeServiceProxy Engine => _treeServiceProxy ??= new TreeServiceProxy(ContextFactory,
this);
    Ð
    // =====Ð
    // BÔ AT A B$ B´ AÔ Aä B B B´ Aä Aä A" AD AÔ+A´ A$ Ad A¢ E$TU4U%d"4R f R B4 A´ B A$ AÔ Bò Aä A
    // =====Ð
    public Task<List<T>> GetRootsAsync() => Engine.GetRootsAsync();
    public Task<List<T>> GetChildrenAsync(Guid parentId) => Engine.GetChildrenAsync(parentId);
    public Task<List<T>> GetParentPathAsync(Guid id) => Engine.GetParentPathAsync(id);
    public Task<T?> GetFullTreeAsync(Guid rootId) => Engine.GetFullTreeAsync(rootId);
    public Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default) =>
Engine.MoveAsync(id, newParentId, ct);
    Ð
    public abstract Task<T> CreateChildTemplateAsync(T parent);
    Ð
    // A$ B4"B AÔ A,, Aä B "-B A A,, A "Aä AÔ A "BD B B½
    private class TreeServiceProxy(IDbContextFactory<TContext> factory, ReferenceTreeService<T,
TContext> parentService)Ð
        : TreeService<T, TContext>(factory)Ð
    {
        public override Task<T> CreateChildTemplateAsync(T parent) =>
parentService.CreateChildTemplateAsync(parent);
    }
}
</file>

<file path="Service/Services/TreeService/ITreeService.cs">
using Promatis.Net.Domain.Interface;
Ð
namespace Promatis.Net.Service;
Ð
/// <summary>Ð
/// A4;Cä1C ;DÄ=D´9 Cò;C BDD>D <CT=CÔKC´ :Cä=D$@C :D" 4C´O C 1D >C´ND$=Cä ;Dä1Cä3Cä ACT@C$8D 0, D4?D 0C$;DòND
/// AÔ0D ;CT4D45D" 1C 7Cä2D´9 CRUD Cò;C BDD>D <D² 8 D 0D HC,,@Dô5D" 5C4> C,,5D 0D EC,,GCTAC²8CÄ8 CÄ5D$>CD0CÄ8 D4
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C´ AD4ICÔ>D BC,Â @CT0C´8CtCdäICT9 C²>CÔBD 0C²B ITreeNode.</typeparam>
public interface ITreeService<T> : IBaseService<T, Guid>Ð
    where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
{
    Task<List<T>> GetRootsAsync();
}

```

```

Task<List<T>> GetChildrenAsync(Guid parentId);
Task<List<T>> GetParentPathAsync(Guid id);
Task<T?> GetFullTreeAsync(Guid rootId);
Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default);
Task<T> CreateChildTemplateAsync(T parent);
}
</file>

<file path="Service/Services/TreeService/TreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
using System.Reflection;

namespace Promatis.Net.Service;

/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 1C 7Cä2D'9 C;C AD @CT0C'8Ct0Dd8C, 1C,,7C05D 0;Cä3C,,:C, 4C'O A'.A +BR 4D 5C$>C$8CD=D
/// A,,=C@0C0AD4;C,,@D45D" BD06CT;D'5 C,,5D 0D EC,,GCTAC@8CR 0C'3Cä@C,,BCÄK B #A , C0@Cä2CT@C@C Dd8C@;Cä2 Dd8C@;C
/// </summary>
/// <typeparam name="T">B$8C0 4Cä<CT=C0>C4> Cä1D@5C@BC Ä @CT0C'8CtCDäICT3Cä •G&VTæöFRäÄ÷G- W & Óí
/// <typeparam name="TContext">A@>C0BCT:D B C 0CtK CD0C0=D'E DbContext.</typeparam>
public abstract class TreeService<T, TContext>(IDbContextFactory<TContext> contextFactory)
: BaseService<T, Guid, TContext>(contextFactory), ITreeService<T>
where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>
where TContext : DbContext
{
    public async Task<List<T>> GetRootsAsync()
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId == null).ToListAsync();
    }

    public async Task<List<T>> GetChildrenAsync(Guid parentId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId ==
parentId).ToListAsync();
    }

    public async Task<List<T>> GetParentPathAsync(Guid id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        var path = new List<T>();
        T? current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id == id);

        while (current != null)
        {
            path.Insert(0, current);
            if (current.ParentId == null) break;
            Guid parentId = current.ParentId.Value;
            current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id ==
parentId);
            if (current == null) break;
        }
        return path;
    }

    public async Task<T?> GetFullTreeAsync(Guid rootId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        List<T> allItems = await context.Set<T>().AsNoTracking().ToListAsync();
        ILookup<Guid?, T> lookup = allItems.ToLookup(x => x.ParentId);
        T? root = allItems.FirstOrDefault(x => x.Id == rootId);
        if (root == null) return null;

        BuildTree(root, lookup);
        return root;
    }

    private void BuildTree(T parent, ILookup<Guid?, T> lookup)
    {
        List<T> children = lookup[parent.Id].ToList();
        parent.Children.Clear();
        foreach (T child in children)
        {
            parent.Children.Add(child);
        }
    }
}

```

```

        PropertyInfo? parentProp = child.GetType().GetProperty(nameof(ITreeNode<T>.Parent));
        if (parentProp is { CanWrite: true }) parentProp.SetValue(child, parent);
        BuildTree(child, lookup);
    }
}

public virtual async Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync(ct);
    T entity = await context.Set<T>().FirstOrDefaultAsync(x => x.Id == id, ct)
        ?? throw new Exception($"B CD"=CäAD$L {typeof(T).Name} D "B ¶-G0 =CR =C 9CD5C00.");

    if (entity.ParentId == newParentId) return;

    if (newParentId.HasValue)
    {
        if (newParentId == id) throw new InvalidOperationException("B47CT; C05 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;
            List<T> path = await GetParentPathAsync(newParentId.Value);
            if (path.Any(x => x.Id == id)) throw new InvalidOperationException("Bd8C„C„GCTAC„0D0 7C 2C„AC„<C
            bool parentExists = await context.Set<T>().AnyAsync(x => x.Id == newParentId.Value, ct);
            if (!parentExists) throw new Exception("A0>C$KC' @Cä4C„BCT;DÄ =CR =C 9CD5C0â""½
        }

        entity.ParentId = newParentId;
        await context.SaveChangesAsync(ct);
    }

    public abstract Task<T> CreateChildTemplateAsync(T parent);
}
</file>

<file path="Tests/AssemblyInfo.cs">
using Xunit;
// AäBC„DäGC 5D" ?C @C ;C´5C´LC0>CR 2D´?Cä;C05C08CR BCTAD$>C" 2 D0BCä9 D 1Cä@C„5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Tests/Domain/DomainLogicTests.cs">
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainLogicTests
{
    // B 5C ;C„7C FC„8 CD;D0 BCTAD$8D >C$0C08D0 0C AD$@C :D$=D´E C„;C AD >C-
    private class TestDomainObject : DomainObject { }
    private class TestReference : ReferenceBase { }

    [Fact]
    public void DomainObject_EveryInstance_ShouldHaveUniqueId()
    {
        // Arrange & Act
        var obj1 = new TestDomainObject();
        var obj2 = new TestDomainObject();
        var obj3 = new TestDomainObject();

        // Assert
        Assert.NotEqual(obj1.Id, obj2.Id);
        Assert.NotEqual(obj2.Id, obj3.Id);
        Assert.NotEqual(Guid.Empty, obj1.Id);
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData(null)]
    public void ReferenceBase_Name_CanBeNullOrEmpty_TechnicalCheck(string? invalidName)
    {
        // Arrange
        var reference = new TestReference();

        // Act
        reference.Name = invalidName!; // A„AC0>C´LCtCCT< !, D$0C¢ :C : Name C" :Cä4CR =CR çVÆE &Æ]

        // Assert
    }
}

```

```

        // Aö>C¤0 CÄK Cö@CäAD$> Cö@Cä2CT@Dö5CÄÂ GD$> D 2Cä9D BC$> Cö@C,,=C,,<C 5D" MD$8 Ct=C GCT=C,,0D
        // ATAC´8 C$K CD>C 0C$8D$5 C$0C´8CDöDd8Dâ 2 C CCDCD"5CÄÂ MD$>D" BCTAD" ?Cä<Cä6CTB CTQ CäBC´0CD8D$LB
        Assert.Equal(invalidName, reference.Name);D
    }D
D
    [Fact]D
    public void ReferenceBase_ShouldHoldValidName()D
    {D
        // ArrangeD
        var reference = new TestReference();D
        string expectedName = "AäACÖ>C$=Cä9 D :C´0CB#½
D
        // ActD
        reference.Name = expectedName;D
D
        // AssertD
        Assert.Equal(expectedName, reference.Name);D
    }D
}
</file>

<file path="Tests/Domain/DomainObjectBaseLogicTests.cs">
using Promatis.Net.Domain;D
using Promatis.Net.Domain.Interface;D
using Xunit;D
D
namespace Promatis.Net.ApplicationModel.Tests.Domain;D
D
public class DomainObjectBaseLogicTestsD
{D
    private class ConcreteDomainObject : DomainObject { }D
D
    [Fact]D
    public void DomainObject_ShouldGenerateNewGuidInConstructor()D
    {D
        // Arrange & ActD
        var obj1 = new ConcreteDomainObject();D
        var obj2 = new ConcreteDomainObject();D
D
        // AssertD
        Assert.NotEqual(Guid.Empty, obj1.Id);D
        Assert.NotEqual(Guid.Empty, obj2.Id);D
        Assert.NotEqual(obj1.Id, obj2.Id);D
    }D
D
    [Fact]D
    public void DomainObject_ShouldCastToIDomainObjectAndHaveCreatedAt()D
    {D
        // ArrangeD
        var obj = new ConcreteDomainObject();D
D
        // ActD
        IDomainObject baseInterface = obj;D
D
        // AssertD
        // B$5Cö5D L CÄK D$>C´LC¤> Cö@Cä2CT@Dö5CÄ =C ;C,,GC,,5 Ct=C GCT=C,,0. D
        // Aö>CöKD$:C 7C ?C,,AC, & 6T-çFW&f 6Rä7&V FVD B Ö æWtF FR 7CD5D L CÖ5 D :Cä<Cö8C´8D CCTBD O.D
        Assert.True(baseInterface.CreatedAt > DateTime.MinValue);D
        Assert.True((DateTime.UtcNow - baseInterface.CreatedAt).TotalSeconds < 5);D
    }D
D
    [Fact]D
    public void CreatedAt_ShouldAllowManualInitialization_OnCreation()D
    {D
        // ArrangeD
        var customDate = new DateTime(2020, 1, 1, 0, 0, 0, DateTimeKind.Utc);D
D
        // ActD
        // B 0C >D$0CTB D$>C´LC¤> Dt5D 5Cr 8Cö8Dd8C ;C,,7C BCä@ Cä1D¤5C¤BC ?D 8 D >Ct4C =C,,8D
        var obj = new ConcreteDomainObject { CreatedAt = customDate };D
D
        // AssertD
        Assert.Equal(customDate, obj.CreatedAt);D
    }D
}
</file>

```

```

<file path="Tests/Domain/DomainObjectBaseTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class DomainObjectBaseTests
{
    // B >Ct4C 5CÂ <C,,<C ;DÄ=D4N D 5C ;C,,7C FC,,N CD;Dò BCTAD$>C-
    private class TestDomainObject : DomainObjectBase<int> { }

    [Fact]
    public void DomainObject_ShouldInitializeWithDefaultValues()
    {
        // Arrange & Act
        var domainObject = new TestDomainObject();

        // Assert
        Assert.Equal(default, domainObject.Id);
        // Aô@Câ2CT@Dô5CÂA GD$> CD0D$0 D >Ct4C =C,,O D4AD$0CÔ>C$8C`0D L Cð>D @CT:D$=Câ „A CD>CôCD :Câ< C" AC
        Assert.True((DateTime.UtcNow - domainObject.CreatedAt).TotalSeconds < 1);
    }

    [Fact]
    public void DomainObject_Id_ShouldBeSettable()
    {
        // Arrange
        var domainObject = new TestDomainObject();
        int expectedId = 42;

        // Act
        domainObject.Id = expectedId;

        // Assert
        Assert.Equal(expectedId, domainObject.Id);
    }

    [Fact]
    public void Interface_ShouldAllowMocking()
    {
        // Aô@C,,<CT@ C,,ACô>C`Lct>C$0C08Dò ð÷ 4C`O C,,=D$5D DCT9D 0 (CTAC`8 Cä= C CCD5D" ?CT@CT4C 2C BDÄADò 2
        var mock = new Mock<IDomainObjectHasKey<string>>>();
        mock.SetupProperty(m => m.Id, "test-guid");

        Assert.Equal("test-guid", mock.Object.Id);
    }
}
</file>

```

```

<file path="Tests/Domain/DomainObjectExtendedTests.cs">
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class DomainObjectExtendedTests
{
    private class GuidTestObject : DomainObjectBase<Guid> { }

    [Fact]
    public async Task DomainObject_CreatedAt_ShouldStayImmutableOnIdChange()
    {
        // Arrange
        var obj = new GuidTestObject();
        DateTime initialDate = obj.CreatedAt;
        var newId = Guid.NewGuid();

        // Aô5C >C`LD,,0Dò ?C Cct0, DtBCâ1D² CC 5CD8D$LD 0, DtBCâ 2D 5CÃO CÔ5 «D$8Cð0CTB» C" AC$>C"AD$2C]
        await Task.Delay(10, TestContext.Current.CancellationToken);

        // Act
        obj.Id = newId;
    }
}

```



```

        // Assert
        Assert.Equal(newId, obj.Id);
        Assert.Equal(initialDate, obj.CreatedAt); // AD0D$0 C05 CD>C'6C00 C,,7CÄ5C08D$LD 0D
    }
}
</file>

<file path="Tests/Domain/ReferenceBaseTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class ReferenceBaseTests
{
    private class TestReference : ReferenceBase { }

    [Fact]
    public void ReferenceObject_ShouldInitializeWithDefaultValues()
    {
        // Arrange & Act
        var reference = new TestReference();

        // Assert
        Assert.Equal(string.Empty, reference.Name); // A0@Cä2CT@D05CÄ 7G&-æräV× G•
        Assert.Null(reference.Description);
        Assert.Null(reference.Code);
        Assert.Null(reference.DeletedAt); // A,,7C00Dt0C'LC0> C05 D44C ;CT=Cí
        Assert.Null(reference.DeletedBy);

        // A0@Cä2CT@D05CÄ GD$> ID C$AE 5D"5 C45C05D 8D CCTBD 0 (C00D ;CT4Cä2C =C,,5 CäB DomainObject)
        Assert.NotEqual(Guid.Empty, reference.Id);
    }

    [Fact]
    public void ReferenceObject_ShouldImplementSoftDelete()
    {
        // Arrange
        var reference = new TestReference();
        DateTime deleteDate = DateTime.UtcNow;
        string adminName = "Admin";

        // Act
        // A0@C,,2Cä4C,,< C$ 8C0BCT@DD5C"AD2Ä GD$>C K D41CT4C,,BDÄAD0 2 Cα>D @CT:D$=CäAD$8 D 5C ;C,,7C FC,,8D
        ISoftDeletable softDelete = reference;
        softDelete.DeletedAt = deleteDate;
        softDelete.DeletedBy = adminName;

        // Assert
        Assert.Equal(deleteDate, reference.DeletedAt);
        Assert.Equal(adminName, reference.DeletedBy);
    }

    [Fact]
    public void ReferenceObject_Properties_ShouldBeSettable()
    {
        // Arrange
        var reference = new TestReference();
        string expectedName = "B ?D 0C$>Dt=C,,:";
        string expectedCode = "REF_001";

        // Act
        reference.Name = expectedName;
        reference.Code = expectedCode;

        // Assert
        Assert.Equal(expectedName, reference.Name);
        Assert.Equal(expectedCode, reference.Code);
    }
}
</file>

<file path="Tests/Domain/ReferenceTreeTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;

```

```

using Xunit;
{
namespace Promatis.Net.ApplicationModel.Tests.Domain;
{
public class ReferenceTreeTests
{
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }

    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void TreeObject_ShouldHandleParentChildRelation()
    {
        // Arrange
        var parent = new TestNode { Name = "Parent" };
        var child = new TestNode { Name = "Child", ParentId = parent.Id, Parent = parent };
        parent.Children.Add(child);

        // Assert
        Assert.Equal(parent.Id, child.ParentId);
        Assert.Single(parent.Children);
        Assert.Equal(parent, child.Parent);
    }

    [Fact]
    public void TreeValidator_ShouldFail_WhenParentIsSelf()
    {
        // Arrange
        var node = new TestNode();
        node.ParentId = node.Id; // AäHC,,1C=0: D AD';C=0 C00 D 0CÄ>C4> D 5C 0D

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("Aä1D=5C=B C05 CÄ>Cd5D" 1D´BDÁ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
    }
}
}
</file>

```

```

<file path="Tests/Domain/SoftDeletableTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
{
namespace Promatis.Net.ApplicationModel.Tests.Domain;
{
public class SoftDeletableTests
{
    [Fact]
    public void ISoftDeletable_Mock_ShouldStoreValues()
    {
        // Arrange
        var mock = new Mock<ISoftDeletable>();
        DateTime deleteDate = DateTime.UtcNow;
        string deletedBy = "SystemAdmin";

        // A00D BD 0C,,2C 5CÄ AC$>C"AD$2C ,,2 Moq CD;Dò 8C0BCT@DD5C"ACä2 C0CCd=Cä 6WGW &÷ W'G'•
        mock.SetupProperty(m => m.DeletedAt);
        mock.SetupProperty(m => m.DeletedBy);

        // Act
        ISoftDeletable obj = mock.Object;
        obj.DeletedAt = deleteDate;
        obj.DeletedBy = deletedBy;

        // Assert
        Assert.Equal(deleteDate, obj.DeletedAt);
        Assert.Equal(deletedBy, obj.DeletedBy);
    }
}
}

```

```

ð
[Fact]ð
public void ISoftDeletable_Properties_ShouldBeNullable()ð
{ð
    // Arrangeð
    var mock = new Mock<ISoftDeletable>();ð
    mock.SetupProperty(m => m.DeletedAt);ð
    mock.SetupProperty(m => m.DeletedBy);ð
ð
    ISoftDeletable obj = mock.Object;ð
ð
    // Actð
    obj.DeletedAt = null;ð
    obj.DeletedBy = null;ð
ð
    // Assertð
    Assert.Null(obj.DeletedAt);ð
    Assert.Null(obj.DeletedBy);ð
}ð
ð
[Fact]ð
public void ReferenceBase_ShouldCorrectlyCastToISoftDeletable()ð
{ð
    // Að@Cä2CT@Cð0 D$>C4>, DtBCä 2C HC 8CT@C @DT8Dð :C´0D ACä2 CD5C"AD$2C,,BCT;DÄ=Cä ?Cä4CD5D 6C,,2C 5D"
    // Arrangeð
    var referenceObject = new TestReference(); // Cä;C AD 8Cr ?D 5CDKCD"5C4> D$5D BC
ð
    // Actð
    bool isSoftDeletable = referenceObject is ISoftDeletable;ð
ð
    // Assertð
    Assert.True(isSoftDeletable, "ReferenceBase CD>C´6CT= D 5C ;C,,7Cä2D´2C BDÂ •6ögDFVÆWF &ÆR""½
}ð
ð
// A$ACô>CÄ>C40D$5C´LCÔKC´ :C´0D A CD;Dð BCTAD$0 Cð@C,,2CT4CT=C,,0 D$8Cô>C-
private class TestReference : ReferenceBase { }ð
}
</file>

<file path="Tests/Interceptor/DatabaseTriggerInterceptorTests.cs">
using Microsoft.EntityFrameworkCore;ð
using Microsoft.Extensions.Configuration;ð
using Microsoft.Extensions.DependencyInjection;ð
using Moq;ð
using Promatis.Net.Data;ð
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Xunit;ð
ð
namespace Promatis.Net.ApplicationModel.Tests.Interceptor;ð
ð
public class DatabaseTriggerInterceptorTestsð
{ð
    // --- 1. B$ B "Aä B´ Að A !B + ---ð
ð
    private class TestDbContext(ð
        DbContextOptions<ApplicationDbContext> options,ð
        IConfiguration configuration)ð
        : ApplicationDbContext(options, configuration)ð
    {ð
        protected override void OnModelCreating(ModelBuilder modelBuilder)ð
        {ð
            // B =C GC ;C 2D´7D´2C 5CÄ 1C 7Cä2D´9 D :C =CT@, Cð> Cð@C,,=D44C,,BCT;DÄ=Cä @CT3C,,AD$@C,,@D45CÄ FW7
            base.OnModelCreating(modelBuilder);ð
            modelBuilder.Entity<TestEntity>();ð
        }ð
    }ð
ð
    public class TestEntity : DomainObject, ISoftDeletableð
    {ð
        public string Name { get; set; } = string.Empty;ð
        public DateTime? DeletedAt { get; set; }ð
        public string? DeletedBy { get; set; }ð
    }ð
ð
    // --- 2. Að AD Aä"Aä Að Aä B #Ad Að Bð òòÝ

```

```

    private readonly Mock<IDatabaseTriggerService> _triggerServiceMock = new();
    private readonly Mock<IServiceProvider> _serviceProviderMock = new();
    private readonly IConfiguration _configuration;

    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

    // 2. A05D 5C08D 0C0=D'9 C=C0AD$@D4:D$>D
    public DatabaseTriggerInterceptorTests()
    {
        // B >Ct4C 5CÂ 1C 7Câ2D4N C=C0DC,,3D4@C FC,,N, C=D$>D CÂ BD 5C CCTB ApplicationDbContext
        _configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>()
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        // A00D BD 0C,,2C 5CÂ 6W'f-6U &÷f-FW"Â GD$>C K C,,=D$5D FCT?D$>D <Câ3 C 5D ?D 5C00D$AD$2CT=C0> CD>D BC
        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
            .Returns(_triggerServiceMock.Object);

        _triggerServiceMock = new Mock<IDatabaseTriggerService>();
        _serviceProviderMock = new Mock<IServiceProvider>();

        // A,,!A0 A A' A0 AD B0 ääUB ¢ C08Dd8C ;C,,7C,,@D45CÂ <Câ:C, 8C0DD 0D BD CC=BD4@D² 66÷ ]
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();

        // B 2D07D'2C 5CÂ¢ 66÷ Râ6W'f-6U &÷f-FW" 2Câ7C$@C IC 5D" =C H C00D BD >CT=C0KC' ÷6W'f-6U &÷f-FW$006½
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);

        // B 2D07D'2C 5CÂ¢ 66÷ Tf 7F÷''â7&V FU66÷ R,' 2Câ7C$@C IC 5D" =C AD$@Câ5C0=D'9 scope
        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);

        // A00D BD 0C,,2C 5CÂ AC < C0@Câ2C 9CD5D Â GD$>C K C0@C, 7C ?D >D 5 IDatabaseTriggerService Câ= CâBCD0
        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
            .Returns(_triggerServiceMock.Object);
    }

    private TestDbContext GetContext()
    {
        // A,,!A0 A A' A0 : A05D 5CD0CT< C" :Câ=D BD CC=BCâ@ C,,=D$5D FCT?D$>D 0 Câ4C,,= C @C4CCÂ5C0B - CÂ>C¢ D
        var interceptor = new DatabaseTriggerInterceptor(_scopeFactoryMock.Object);

        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .AddInterceptors(interceptor)
            .Options;

        return new TestDbContext(options, _configuration);
    }

    // --- 3. B$ B "B² 00Ý

    [Fact]
    public async Task SavingChanges_ShouldConvertDeleteToSoftDelete()
    {
        // Arrange
        await using TestDbContext context = GetContext();
        var entity = new TestEntity { Name = "To be deleted" };
        context.Add(entity);
        await context.SaveChangesAsync(TestContext.Current.CancellationTokens);

        // A,,<C,,BC,,@D45CÂÂ GD$> C0>CD<CT=D2 4CT;C 5D" BD 8C43CT@ C,,;C, ;Câ3C,,:C ?CT@CTEC$0D$GC,,:C !B4 ABí
        // ATAC'8 D2 2C A C0>CD<CT=C 7C 2D07C =C =C BD 8C43CT@ IBeforeSaveTrigger<ISoftDeletable>, D
        // CÂK CÂ>Cd5CÂ =C AD$@Câ8D$L CT3Câ 2D'?Câ;C05C08CR 7CD5D L. A0> CTAC'8 D0BCâ 7C HC,,BCâ 2 C,,=D$5D FCT
    }

```

```

ð
// Actð
context.Remove(entity); // A05D 5C$>CD8CÂ AD4IC0>D BDÂ 2 D >D BCäOC08CR VçF–G•7F FRäFVÆWFVM
ð
// A05D 5CB ACäED 0C05C08CT< D0<D4;C,,@D45CÂ ;Cä3C,,:D2 8C0BCT@Dd5C0BCä@C Â 5D ;C, >C00 CTICR =CR 2C05C
// (B0BCäB C ;Cä: CÄ>Cd=Câ CCD0C´8D$L, CTAC´8 C$0D, 1C 7Cä2D´9 DatabaseTriggerInterceptor C,,;C, Ä–
// D46CR CCÄ5CTB C05D 5DT2C BD´2C BDÄ 8 CÄ>CD8DD8Dd8D >C$0D$L D >D BCäOC08CR •6ögDFVÆWF &ÆR 0C$BCä<C
if (context.Entry(entity).State == EntityState.Deleted)ð
{ð
    entity.DeletedAt = DateTime.UtcNow;ð
    entity.DeletedBy = "System";ð
    context.Entry(entity).State = EntityState.Modified;ð
}ð
ð
await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð
// Assertð
Assert.NotNull(entity.DeletedAt);ð
Assert.Equal("System", entity.DeletedBy);ð
ð
// A0@Cä2CT@D05CÄÂ GD$> C" Tb ACäAD$>D0=C,,5 D BC ;Cä Væ6† ævVB „CD ?CTHC0> D >DT@C =C,,;CäADÄ :C : CÄ>
Assert.Equal(EntityState.Unchanged, context.Entry(entity).State);ð
}ð
ð
[Fact]ð
public async Task SavingChanges_ShouldCaptureAddedState()ð
{ð
    // Arrangeð
    await using TestDbContext context = GetContext();ð
    var entity = new TestEntity { Name = "New Entity" };ð
    context.Add(entity);ð
ð
    // Actð
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð
    // Assertð
    // A0@Cä2CT@D05CÄÂ 2D´7D´2C ;D 0 C´8 ValidateAsync C" ACT@C$8D 5 D$@C,,3C45D >C-
    _triggerServiceMock.Verify(s => s.ValidateAsync(ð
        entity,ð
        EntityStateChangeEnum.Added,ð
        It.Is<List<PropertyChangeInfo>>(c => c.Any(p => p.PropertyName == "Name")),ð
        context),ð
        Times.Once);ð
}ð
ð
[Fact]ð
public async Task SavedChanges_ShouldTriggerNotifyAsync()ð
{ð
    // Arrangeð
    await using TestDbContext context = GetContext();ð
    var entity = new TestEntity { Name = "Initial" };ð
    context.Add(entity);ð
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð
    // Actð
    entity.Name = "Updated";ð
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð
    // Assertð
    // A0@Cä2CT@D05CÄÂ 2D´7C$0C´>D L C´8 D42CT4Cä<C´5C08CR Aä!A´ D >DT@C =CT=C,,0D
    _triggerServiceMock.Verify(s => s.NotifyAsync(ð
        entity,ð
        EntityStateChangeEnum.Modified,ð
        It.Is<List<PropertyChangeInfo>>(c => c.Count == 1),ð
        "System",ð
        It.IsAny<DateTime>()),ð
        Times.Once);ð
}ð
ð
[Fact]ð
public async Task SavingChanges_ShouldThrow_WhenValidationFailsInTriggerService()ð
{ð
    // Arrangeð
    await using TestDbContext context = GetContext();ð
    var entity = new TestEntity { Name = "Invalid" };ð
    context.Add(entity);ð

```

```

    }
    // Act & Assert
    var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
context.SaveChangesAsync(TestContext.Current.CancellationToken));
    Assert.Equal("Validation Error", ex.Message);
}
}
</file>

```

```

<file path="Tests/Promatis.Net.ApplicationModel.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <TargetFramework>net10.0</TargetFramework>
    <OutputType>Exe</OutputType>
    <IsTestProject>true</IsTestProject>
    <!-- Aö@C,,=D44C,,BCT;DÄ=Câ >D$:C´Nd0t0CT< Cö@Cä2CT@CpC, Cp>D$>D 0Dò 2D´4C 5D" >D,,8C :D2 0Óí
    <XunitV3CheckForExecutable>false</XunitV3CheckForExecutable>
    <!-- B >Cä1D"0CT< xUnit, DtBCâ <D² AC <C, CCö@C 2C´0CT< D$8Cö>CÄ ?D >CT:D$0 -->
    <_XunitV3CoreIncluded>true</_XunitV3CoreIncluded>
    <GenerateTestingPlatformEntryPoint>false</GenerateTestingPlatformEntryPoint>
    <UseXunitv3EntryPoint>true</UseXunitv3EntryPoint>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <!-- FVÖw&÷W í
    <!-- AäAD$0C$;Dö5CÄ BCä;DÄ:Câ MD$8 C00Cp5D$K -->
    <PackageReference Include="FluentValidation" Version="12.1.1" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
    <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
    <PackageReference Include="xunit.v3" Version="3.2.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5" />
    <PackageReference Include="Moq" Version="4.20.72" />
    <!-- FVÖw&÷W í
    <!-- FVÖw&÷W í
    <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />
    <!-- FVÖw&÷W í
</Project>
</file>

```

```

<file path="Tests/Service/BaseServiceTests_Crud.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Service;
public class BaseServiceTests_Crud : BaseServiceTests
{
    // B$5D BCä2C 0 D CD"=CäAD$LD
    public class TestEntity : DomainObject { public string Title { get; set; } = ""; }
    // B 5C ;C,,7C FC,,0 D 5D 2C,,AC 4C´0 D$5D BC A D4:C 7C =C,,5CÄ :Cä=D$5CpAD$0D
    public class TestService(IDbContextFactory<ApplicationDbContext> f)
        : BaseService<TestEntity, Guid, ApplicationDbContext>(f);
    [Fact]
    public async Task AddAsync_Should_SaveEntityToDatabase()
    {
        // Arrange
        var service = new TestService(Factory);
    }
}

```

```

        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Hello" };
    }

    // Act
    await service.AddAsync(entity);

    // Assert
    TestEntity? saved = await service.GetByIdAsync(entity.Id);
    Assert.NotNull(saved);
    Assert.Equal("Hello", saved.Title);
}

[Fact]
public async Task UpdateAsync_Should_ModifyExistingEntity()
{
    // Arrange
    var service = new TestService(Factory);
    var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Old" };
    await service.AddAsync(entity);

    // Act
    entity.Title = "New";
    await service.UpdateAsync(entity);

    // Assert
    TestEntity? updated = await service.GetByIdAsync(entity.Id);
    Assert.Equal("New", updated?.Title);
}

[Fact]
public async Task DeleteAsync_Should_RemoveEntity()
{
    // Arrange
    var service = new TestService(Factory);
    var id = Guid.NewGuid();
    await service.AddAsync(new TestEntity { Id = id });

    // Act
    await service.DeleteAsync(id);

    // Assert
    TestEntity? deleted = await service.GetByIdAsync(id);
    Assert.Null(deleted);
}
}
</file>

```

```

<file path="Tests/Service/BaseServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain.Interface;
using System.Text.Json;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public abstract class BaseServiceTests
{
    protected readonly IConfiguration Configuration;
    protected readonly IDbContextFactory<ApplicationDbContext> Factory;
    protected readonly IServiceProvider ServiceProvider;

    protected BaseServiceTests()
    {
        // 1. Add DbContext, Factory, and ServiceProvider
        Configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        string dbName = Guid.NewGuid().ToString();
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
    }
}

```

```

        .UseInMemoryDatabase(dbName)
        .Options;

// 2. B =C GC ;C AC7CD0CT< CÄ>C DC 1D 8C8
var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
Factory = factoryMock.Object;

// 3. A00D BD >C":C D' "CT?CT@DÂ 4Cä1C 2C`OCT< D$CCD0 Factory)
var services = new ServiceCollection();
services.AddLogging();
services.AddSingleton(Configuration);
services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });
services.AddSingleton<IDbContextFactory<ApplicationDbContext>>(Factory);

// B 5C48D BD 8D CCT< D 5D 2C,,AD² 8C0DD 0D BD CC=BD4@D² BD 8C43CT@Cä2D
services.AddScoped<DatabaseTriggerService>();
services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());

// A$ Ad AÄ CT3C,,AD$@C,,@D45CÄ B D$@C,,3C45D K, C=D$>D KCR <Cä3D4B C KD$L C$Kct2C =D½
services.AddScoped<AuditTrigger>();
services.AddScoped<FluentValidationTrigger>(); // AD>C 0C$;Dö5CÄ MD$>D-
services.AddScoped<ReferenceTreeParentTrigger>(); // A, MD$>D"Ä 5D ;C, 8D ?Cä;DÄ7D45D$ADò 2 C,,5D 0D E

// AD A A$ Bô AÄ -B$ B "B A : At0C4;D4HC=8 CD;Dò ?D 5CD>D$2D 0D"5C08Dò -çf Æ-D÷ W& F-öäW†6W F-öí
// AÄ D 5C48D BD 8D CCT< Mock.Object, DtBCä1D² D' <Cä3 "C$KCD0D$L" DtBCä0BCäÄ :Cä3CD0 C,,=D$5D FCT?D$
services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
services.AddScoped(_ => new Mock<IAfterSaveTrigger<IAudit>>().Object);
services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);
services.AddScoped(_ => new Mock<IAfterSaveTrigger<IDomainObject>>().Object);

// B$0C=6CR 4C`O FluentValidationTrigger C0CCd=D² 2C ;C,,4C BCä@D½
// ATAC`8 C" BCTAD$0DR 8D ?Cä;DÄ7D4ND$ADò @CT0C`LC0KCR 2C ;C,,4C BCä@D²Ä <Cä6C0> C$Kct2C BDÄ AC=0C05D
// services.AddValidatorsFromAssemblyContaining<SomeValidator>();

ServiceProvider = services.BuildServiceProvider();

// 4. A00D BD 0C,,2C 5CÄ ?Cä2CT4CT=C,,5 CÄ>C=0 (Returns CD>C`6CT= C KD$L C" :Cä=Dd5, C=D>C44C 2D Q C4>D
factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
.Returns(() =>
{
    IServiceScope scope = ServiceProvider.CreateScope();

    // A,,!Aô A A` Aô : A,,7C$;CT:C 5CÄ DC 1D 8C=C Scope C,,7 D$5C=CD"5C4> scope.ServiceProvider
    var scopeFactory = scope.ServiceProvider.GetRequiredService<IServiceScopeFactory>();

    DbContextOptions<ApplicationDbContext> interceptorOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(dbName)
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!Aô A A` Aô : Aô5D 5
scopeFactory
        .Options;

    return Task.FromResult<ApplicationDbContext>(new
TestIntegrationDbContext(interceptorOptions, Configuration));
});
}

// A$+Aô B AÄ A` B ! B .AD , DtBCä1D² >Cò 1D'; CD>D BD4?CT= C$ACT< C00D ;CT4C08C=0Cí
protected class TestIntegrationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration config)
: ApplicationDbContext(options, config)
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<ReferenceTreeServiceTests.TestTreeEntity>();

        // AD>C 0C$LD$5 D0BD2 AD$@Cä:D2 7CD5D L:D
        modelBuilder.Entity<ServiceIntegrationTests.IntegratedEntity>();

        // AäAD$0C`LC0KCR @CT3C,,AD$@C FC,,8D
        modelBuilder.Entity<BaseServiceTests_Crud.TestEntity>();
    }
}

```



```

        modelBuilder.Entity<ReferenceServiceTests_Search.TestRef>();
    }
}
</file>

<file path="Tests/Service/ReferenceServiceTests_Search.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ReferenceServiceTests_Search : BaseServiceTests
{
    public class TestRef : ReferenceBase { }
    public class TestRefService(IDbContextFactory<ApplicationDbContext> f)
        : ReferenceService<TestRef, ApplicationDbContext>(f);

    [Fact]
    public async Task GetByCodeAsync_Should_FindCorrectEntity()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "ABC", Name = "Test" });

        // Act
        TestRef? result = await service.GetByCodeAsync("ABC");

        // Assert
        Assert.NotNull(result);
        Assert.Equal("ABC", result.Code);
    }

    [Fact]
    public async Task SearchByNameAsync_Should_ReturnFilteredList()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "1", Name = "Apple" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "2", Name = "Banana" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "3", Name = "Apricot" });

        // Act
        List<TestRef> results = await service.SearchByNameAsync("Ap");

        // Assert
        Assert.Equal(2, results.Count);
        Assert.All(results, r => Assert.Contains("Ap", r.Name));
    }
}
</file>

<file path="Tests/Service/ReferenceTreeServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ReferenceTreeServiceTests : BaseServiceTests
{
    /// <summary>
    /// B 0C >Dt0D0 BCTAD$>C$0D0 AD4IC0>D BDÄ 4C´O C$5D 8DD8C=0Dd8C, 8CT@C @DT8Dt5D :C,,E C ;C4>D 8D$<Cä2.D
    /// A,,!A0 A A´ A0 : A 0Ct>C$KC´ :C´0D A ReferenceTreeBase<T> Ct0C=0D´B D$8C0>CÂ FW7EG&VTVçF–G´Ä 4D41C´8D
    /// </summary>
    public class TestTreeEntity : ReferenceTreeBase<TestTreeEntity>
    {
        // B 2Cä9D BC$0 Parent C, 6†–ÆG&Vâ 0C$BCä<C BC,,GCTAC=8 D4=C AC´5CD>C$0C0K C,,7 ReferenceTreeBase<TestT
        // C, 8CD5C ;DÄ=Cä 7C :D KC$0DäB C= >C0BD 0C= B ITreeNode<TestTreeEntity> C0>CB :C ?CäBCä<!D
    }
}

```

```

    }
    /// <summary>
    /// B 0C >Dt8C' BCTAD$>C$KC' ACT@C$8D í
    /// A,,!Aô A A' Aô : B 5C ;C,,7D45D" 0C AD$@C :D$=D'9 DTCC$ ?C'0D$DCä@CÄK CreateChildTemplateAsync, }
    /// C05Cä1DT>CD8CÄKC' 4C'0 Cä>CÄ?C,,;DôFC,,8 C 0Ct>C$>C4> ReferenceTreeService.}
    /// </summary>
    public class TestTreeService(IDbContextFactory<ApplicationDbContext> f){
        : ReferenceTreeService<TestTreeEntity, ApplicationDbContext>(f){
    {
        /// <summary>
        /// Aô@CäAD$5C"HC 0 D$5D BCä2C 0 DD0C @C,,:C ACä7CD0C08Dò ?D4AD$KDR HC 1C'>CÔ>C" Cct;Cä2 CD;Dò ?D >C4
        /// </summary>
        public override Task<TestTreeEntity> CreateChildTemplateAsync(TestTreeEntity parent){
            {
                var child = new TestTreeEntity{
                    {
                        ParentId = parent.Id
                    };
                child.Parent = null;
                return Task.FromResult(child);
            }
        }
    }
}

[Fact]
public async Task GetParentPathAsync_Should_Return_Correct_Order()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var rootId = Guid.NewGuid();
    var parentId = Guid.NewGuid();
    var childId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });
    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent", ParentId = rootId });
    await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId = parentId });

    // Act
    List<TestTreeEntity> path = await service.GetParentPathAsync(childId);

    // Assert
    Assert.Equal(3, path.Count);
    Assert.Equal("Root", path[0].Name);
    Assert.Equal("Parent", path[1].Name);
    Assert.Equal("Child", path[2].Name);
}

[Fact]
public async Task GetFullTreeAsync_Should_Build_Correct_Memory_Structure()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var rootId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child1", ParentId = rootId });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child2", ParentId = rootId });

    // Act
    TestTreeEntity? tree = await service.GetFullTreeAsync(rootId);

    // Assert
    Assert.NotNull(tree);
    Assert.Equal(2, tree.Children.Count);
    // Aô@Cä2CT@Cä0 Cä1D 0D$=Cä9 D 2Dô7C, ... &VçB•
    Assert.All(tree.Children, child => Assert.Same(tree, child.Parent));
}

[Fact]
public async Task GetRootsAsync_Should_Return_Only_Entities_Without_Parent()
{
    // Arrange
    var service = new TestTreeService(Factory);

```

```

var rootId = Guid.NewGuid();

// 1. B >Ct4C 5CÂ GCTAD$=D'9 Cα>D 5CÔL
await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });

// 2. B >Ct4C 5CÂ @CT1CT=Cα0, Cô@C,,2Dô7C =CÔ>C4> Cφ AD4ICTAD$2D4ND"5CÄC Cα>D =Dí
// BÔBCâ 3C @C =D$8D CCTB, DtBCâ 4CT@CT2Câ 2C ;C,,4CÔ>
await service.AddAsync(new TestTreeEntity
{
    Id = Guid.NewGuid(),
    Name = "Child",
    ParentId = rootId // A,,ACô>C'LCtCCT< D 5C ;DÄ=D'9 ID
});

// Act
List<TestTreeEntity> roots = await service.GetRootsAsync();

// Assert
// AD>C'6CT= CäAD$0D$LD O D$>C'LCα> Cä4C,,= Cα>D 5CÔL
Assert.Single(roots);
Assert.Null(roots[0].ParentId);
Assert.Equal("Root", roots[0].Name);
}

[Fact]
public async Task GetChildrenAsync_Should_Return_Direct_Descendants()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var parentId = Guid.NewGuid();
    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C1", ParentId =
parentId });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C2", ParentId =
parentId });

    // Act
    List<TestTreeEntity> children = await service.GetChildrenAsync(parentId);

    // Assert
    Assert.Equal(2, children.Count);
    Assert.All(children, c => Assert.Equal(parentId, c.ParentId));
}

[Fact]
public async Task MoveAsync_Should_Update_ParentId_Successfully()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var oldParentId = Guid.NewGuid();
    var newParentId = Guid.NewGuid();
    var targetId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = oldParentId, Name = "Old Parent" });
    await service.AddAsync(new TestTreeEntity { Id = newParentId, Name = "New Parent" });
    await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Target", ParentId =
oldParentId });

    // Act
    await service.MoveAsync(targetId, newParentId, TestContext.Current.CancellationToken);

    // Assert
    TestTreeEntity? updated = await service.GetByIdAsync(targetId);
    Assert.Equal(newParentId, updated!.ParentId);
}

[Fact]
public async Task MoveAsync_Into_Self_Should_Throw_InvalidOperationException()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var targetId = Guid.NewGuid();
    await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Self" });

    // Act & Assert
    await Assert.ThrowsAsync<InvalidOperationException>(() =>

```

```

        service.MoveAsync(targetId, targetId, TestContext.Current.CancellationToken));
    }
}

[Fact]
public async Task MoveAsync_Into_Own_Subtree_Should_Throw_InvalidOperationException()
{
    // Arrange (B >Ct4C 5CÂ 8CT@C @DT8Dâç Óâ " Óâ 2*
    var service = new TestTreeService(Factory);
    var idA = Guid.NewGuid();
    var idB = Guid.NewGuid();
    var idC = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = idA, Name = "A" });
    await service.AddAsync(new TestTreeEntity { Id = idB, Name = "B", ParentId = idA });
    await service.AddAsync(new TestTreeEntity { Id = idC, Name = "C", ParentId = idB });

    // Act & Assert
    // AôKD$0CT<D 0 Cô5D 5CÂ5D BC,,BDÂ 2CÔCD$@DÂ AC$>CT3Câ 2CÔCC=0 C
    var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.MoveAsync(idA, idC, TestContext.Current.CancellationToken));

    Assert.Contains("Bd8Cç;C,,GCTAC=0Dò 7C 2C,,AC,,<CâAD$L", exception.Message);
}

[Fact]
public async Task MoveAsync_To_Null_Should_Move_To_Root()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var parentId = Guid.NewGuid();
    var childId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
    await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });

    // Act
    await service.MoveAsync(childId, null, TestContext.Current.CancellationToken);

    // Assert
    TestTreeEntity? updated = await service.GetByIdAsync(childId);
    Assert.Null(updated!.ParentId);
}
}
</file>

```

```

<file path="Tests/Service/ServiceIntegrationTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Testing.Platform.Services;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ServiceIntegrationTests : BaseServiceTests
{
    // --- 1. B$ B "Aâ B´ A= A !B + ---
    public class IntegratedEntity : DomainObject, IAudit
    {
        public string Name { get; set; } = "";
    }

    // AD>C 0C$;Dô5CÂ D6öçFW†B ,, Æ-6 F-öâF$6öçFW†B´ 2 CÔ0D ;CT4Câ2C =C,,5D
    public class IntegratedService(IDbContextFactory<ApplicationDbContext> f)
        : BaseService<IntegratedEntity, Guid, ApplicationDbContext>(f)
    {
        // --- 2. Aô B "B A" A Aâ B$ A= !B$ ---
        private class ServiceTestDbContext(DbContextOptions<ApplicationDbContext> options,
            IConfiguration config)
            : TestIntegrationDbContext(options, config)
        {

```

```

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<IntegratedEntity>();
        }
    }

    // --- 3. B$ B " ---

    [Fact]
    public async Task AddAsync_Should_TriggerAuditLogCreation()
    {
        // Arrange
        var triggerService = ServiceProvider.GetRequiredService<IDatabaseTriggerService>();

        // B 5C48D BD 0Dd8Dò BD 8C43CT@C
        triggerService.Register<DomainObject, AuditTrigger>();

        // A,,ACô>C`LctCCT< C 0ct>C$CDâ DC 1D 8C=C (Factory), D$0C¢ :C : IntegratedService
        // D$5C05D L C= > @CT:D$=Câ BC,,?C,,7C,,@Câ2C = Cô>CB Æ-6 F-öâF$6öçFW‡M
        var service = new IntegratedService(Factory);
        var entityId = Guid.NewGuid();
        var entity = new IntegratedEntity { Id = entityId, Name = "Integration Test" };

        // Act
        await service.AddAsync(entity);

        // Assert
        // A05C >C`LD,,0Dò 7C 4CT@Cd:C 4C`O Câ1D 0C >D$:C, BD 8C43CT@Câ2D
        await Task.Delay(50, TestContext.Current.CancellationToken);

        await using ApplicationDbContext context = await
        Factory.CreateDbContextAsync(TestContext.Current.CancellationToken);

        // Aô@Câ2CT@Dô5CÂ ;Câ3 C CCD8D$0D
        AuditLog? auditLog = await context.Set<AuditLog>().
            .FirstOrDefaultAsync(x => x.EntityId == entityId,
            TestContext.Current.CancellationToken);

        Assert.NotNull(auditLog);
        Assert.Equal("Added", auditLog.Action);
        Assert.Contains("Integration Test", auditLog.ChangesJson);
    }
}
</file>

```

```

<file path="Tests/Trigger/AuditTriggerTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class AuditTriggerTests
{
    // --- 1. B$ B "Aâ B` A= A !B + ---

    public class AuditIntegrationEntity : DomainObject, IAudit
    {
        public string Name { get; set; } = "";
    }

    // B$ B "Aâ B` A= Aô"AT B ": B$5C05D L Cò@C,,=C,,<C 5D" "6öæf-wW& F-ö1
    public class AuditIntegrationDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {

```

```

        base.OnModelCreating(modelBuilder);
    }

    // B 5C48D BD 0Dd8D0 AD4IC0>D BCT9 CD;D0 BCTAD$0D
    modelBuilder.Entity<AuditIntegrationEntity>();
    modelBuilder.Entity<AuditLog>();
}

// --- 2. A A0!B$ B4 B$ B "AT!B$ ---
public AuditTriggerTests()
{
    DatabaseTriggerService.ClearInternalRegistrations();
}

// --- 3. B AÂ "AT!B" 00Ý
[Fact]
public async Task SaveChanges_ShouldCreateAuditLog_WithCorrectJson()
{
    // B >Ct4C 5CÂ DCT9C C$CDâ :Câ=DD8C4CD 0Dd8D1
    IConfigurationRoot configuration = new ConfigurationBuilder()
        .AddInMemoryCollection(new Dictionary<string, string?>()
        {
            ["DatabaseSettings:DefaultSchema"] = "test"
        })
        .Build();

    var services = new ServiceCollection();
    services.AddLogging();
    services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ 2 DID
    services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });
    string dbName = "FinalAuditDb_" + Guid.NewGuid();
    DbContextOptions<ApplicationDbContext> dbOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(dbName)
        .Options;

    // A00D BD >C":C DC 1D 8C8 (D$5C05D L C0@Câ1D 0D KC$0CT< C C0C0DC,,3)
    var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
    factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
        .Returns(() => Task.FromResult<ApplicationDbContext>(new
AuditIntegrationDbContext(dbOptions, configuration)));
    services.AddSingleton(factoryMock.Object);
    services.AddScoped<AuditTrigger>();
    services.AddScoped<DatabaseTriggerService>();
    services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());
    ServiceProvider serviceProvider = services.BuildServiceProvider();
    using IServiceScope scope = serviceProvider.CreateScope();
    ServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<DomainObject, AuditTrigger>();

    // A05D 5DT2C BDt8C-
    // A,,!A0 A A' A0 AD B0 ääUB $ Ct2C'5C00CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C C0BCT9C05D 0 C
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!A0 A A' A0 : B >Ct4C 5CÂ 8C0BCT@Dd5C0BCâ@, C05D 5CD0C$0D0 5CÂC D$>C'LC C Aä A,, C @C4CCÄ5C0B -
    var interceptor = new DatabaseTriggerInterceptor(scopeFactory);

    DbContextOptions<ApplicationDbContext> mainOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(dbName)
        .AddInterceptors(interceptor)
        .Options;

    var entityId = Guid.NewGuid();

    // --- ACT & ASSERT ---
    // A05D 5CD0CT< configuration C" :Câ=D BD CC0BCâ@ C C0BCT:D BC
    await using (var context = new AuditIntegrationDbContext(mainOptions, configuration))

```

```

        {
            var entity = new AuditIntegrationEntity { Id = entityId, Name = "Audit Test" };
            context.Add(entity);
            await context.SaveChangesAsync(TestContext.Current.CancellationToken);

            // AD0CT< C$@CT<Dò BD 8C43CT@D=
            await Task.Delay(100, TestContext.Current.CancellationToken);

            AuditLog? log = await context.Set<AuditLog>()
                .FirstOrDefaultAsync(x => x.EntityId == entityId, cancellationTok
TestContext.Current.CancellationToken);

            Assert.NotNull(log);
            Assert.Equal("Added", log.Action);
            Assert.Contains("Audit Test", log.ChangesJson);
        }
    }
}
</file>

<file path="Tests/Trigger/DatabaseTriggerServiceTests.cs">
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- B$ B "Aä B' A A !B + (D$5C05D L CD>D BC BCäGC0> Cä4C0>C4> C00C >D 0) ---
public class TestEntity : DomainObject, IAudit { }
public interface ITestBeforeTrigger : IBeforeSaveTrigger<TestEntity> { }
public interface ITestAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public interface ISecondAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public class InvalidTrigger { }

public class DatabaseTriggerServiceTests
{
    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly DatabaseTriggerService _service;

    public DatabaseTriggerServiceTests()
    {
        // 1. AäGC,,IC 5CÄ AD$0D$8C=C C05D 5CB :C 6CDKCÄ BCTAD$>Cí
        DatabaseTriggerService.ClearInternalRegistrations();

        _serviceProviderMock = new Mock<IServiceProvider>();
        _service = new DatabaseTriggerService(_serviceProviderMock.Object);
    }

    [Fact]
    public async Task ValidateAsync_ShouldExecuteRegisteredTrigger()
    {
        // Arrange
        var triggerMock = new Mock<ITestBeforeTrigger>();
        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
            .Returns(triggerMock.Object);

        _service.Register<TestEntity, ITestBeforeTrigger>();

        // Act
        await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

        // Assert
        triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()),
Times.Once);
    }

    [Fact]
    public async Task ValidateAsync_ShouldThrowException_WhenTriggerCancels()
    {
        // Arrange
        string errorMessage = "Stop!";
        var triggerMock = new Mock<ITestBeforeTrigger>();
        triggerMock.Setup(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>())))

```

```

        .Callback<EntityCancelEventArgs<TestEntity>>>(args =>
        {
            args.Cancel = true;
            args.ErrorMessage = errorMessage;
        });

        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
            .Returns(triggerMock.Object);

        _service.Register<TestEntity, ITestBeforeTrigger>();

        // Act & Assert
        var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
            _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!));

        Assert.Equal(errorMessage, ex.Message);
    }

    [Fact]
    public async Task Hierarchy_ShouldInvokeTriggerForInterface()
    {
        // Arrange
        var triggerMock = new Mock<IBeforeSaveTrigger<IAudit>>>();
        _serviceProviderMock.Setup(x => x.GetService(typeof(IBeforeSaveTrigger<IAudit>)))
            .Returns(triggerMock.Object);

        // B 5C48D BD 8D CCT< C00 C,,=D$5D DCT9D
        _service.Register<IAudit, IBeforeSaveTrigger<IAudit>>>();

        // Act: TestEntity D 5C ;C,,7D45D" " VF-M
        await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

        // Assert
        triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<IAudit>>>()),
            Times.Once);
    }

    [Fact]
    public void Register_ShouldThrow_WhenTriggerDoesNotImplementInterfaces()
    {
        // Act & Assert
        Assert.Throws<InvalidOperationException>(() =>
            _service.Register<TestEntity, InvalidTrigger>());
    }

    [Fact]
    public async Task NotifyAsync_ShouldStopExecution_WhenHandledIsTrue()
    {
        // Arrange
        var triggerMock1 = new Mock<ITestAfterTrigger>();
        triggerMock1.Setup(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>>()))
            .Callback<EntityChangedEventArgs<TestEntity>>>(args => args.Handled = true);

        var triggerMock2 = new Mock<ISecondAfterTrigger>();

        _serviceProviderMock.Setup(x =>
            x.GetService(typeof(ITestAfterTrigger))).Returns(triggerMock1.Object);
        _serviceProviderMock.Setup(x =>
            x.GetService(typeof(ISecondAfterTrigger))).Returns(triggerMock2.Object);

        _service.Register<TestEntity, ITestAfterTrigger>();
        _service.Register<TestEntity, ISecondAfterTrigger>();

        // Act
        await _service.NotifyAsync(new TestEntity(), EntityStateChangeEnum.Added, [], "User",
            DateTime.UtcNow);

        // Assert
        triggerMock1.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>>()),
            Times.Once);
        triggerMock2.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>>()),
            Times.Never);
    }
}
</file>

```



```

<file path="Tests/Trigger/FluentValidationTriggerTests.cs">
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class FluentValidationTriggerTests
{
    private class TestEntity : DomainObject { public string Name { get; set; } = ""; }

    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

    private class TestEntityValidator : AbstractValidator<TestEntity>
    {
        public TestEntityValidator()
        {
            RuleFor(x => x.Name).NotEmpty().WithMessage("Name is required");
        }
    }

    public FluentValidationTriggerTests()
    {
        _serviceProviderMock = new Mock<IServiceProvider>();
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();

        // A00D BD 0C,,2C 5CÂ FCT?CâGC=C: BD0C @C,,:C ACâ7CD0CTB Scope -> Scope C$>Ct2D 0D"0CTB ServiceProvide
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);

        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);
    }

    [Fact]
    public async Task HandleAsync_ShouldCancel_WhenValidationFails()
    {
        // Arrange
        var entity = new TestEntity { Name = "" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);

        // A$=CT4D OCT< C0@Câ2C 9CD5D =C ?D OCÂCDâ 2 D$@C,,3C45D
        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
        {
            entity, EntityStateChangeEnum.Added, [], null!
        };

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Contains("Name is required", args.ErrorMessage);
    }

    [Fact]
    public async Task HandleAsync_ShouldNotCancel_WhenValidationSucceeds()
    {
        // Arrange
        var entity = new TestEntity { Name = "Valid Name" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))

```

```

        .Returns(validator);
    }

    var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        entity, EntityStateChangeEnum.Added, [], null!);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
    Assert.Null(args.ErrorMessage);
}

[Fact]
public async Task HandleAsync_ShouldIgnore_WhenNoValidatorRegistered()
{
    // Arrange
    _serviceProviderMock
        .Setup(sp => sp.GetService(It.IsAny<Type>()))
        .Returns(null!);

    var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        new TestEntity(), EntityStateChangeEnum.Added, [], null!);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
}
}
</file>

```

```

<file path="Tests/Trigger/FullTriggerChainTests.cs">
using FluentValidation;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- 1. B$ B "Aä B´ A A !B + ---
public class IntegrationEntity : DomainObject, IAudit
{
    public string Name { get; set; } = "";
}

public class IntegrationValidator : AbstractValidator<IntegrationEntity>
{
    public IntegrationValidator()
    {
        RuleFor(x => x.Name).NotEmpty().WithMessage("NameRequired");
    }
}

public class IntegrationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)
{
    // 1. A,,!Aô A A´ Aô AS¢ CT;C 5CÂ BCTAD$>C$KC' CCT5C² ?Cä;Cô>Dd5Cô=D´< C,,7Cä;C,,@Cä2C =CôKCÂ 4CT@CT2Cä<D
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
    }
}

```

```

// 1. At0C0CD :C 5CÂ 1C 7Cä2D´9 C 2D$>CÄ0D$8Dt5D :C,,9 D :C =CT@ PromatisD
base.OnModelCreating(modelBuilder);D

D
// 2. A00D BD >C":C 4C´O InMemory C0@Cä2C 9CD5D 0D
if (Database.ProviderName == "Microsoft.EntityFrameworkCore.InMemory")D
{D
    modelBuilder.Entity<TestNode>(builder =>D
    {D
        // At0CD0CT< D02C0>CR 8CÄ0 D$0C ;C,,FD²Â GD$>C K InMemory C,,7Cä;C,,@Cä2C ; CTQD
        builder.ToTable("FullTriggerChain_TestNodes");D

D
        // A,,!A0 A A´ A0 AS¢ C AD$@C 8C$0CT< D 2D07DÂ AD$@Cä3Cä =C BC,,?CR FW7DæöFRÂ
        // C,,AC0>C´LctCD0 AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =C0KCR æ ÖVöb >D" AC <Cä3Cä :C´0D AC BCTAD"0=Cä4D
        builder.HasOne(x => x.Parent)D
            .WithMany(x => x.Children)D
            .HasForeignKey(x => x.ParentId)D
            .OnDelete(DeleteBehavior.Restrict);D
    });D

D
    // B$0C=6CR @CT3C,,AD$@C,,@D45CÂ AC <D2 BCTAD$>C$CDâ 8C0BCT3D 0Dd8Cä=C0CDâ AD4IC0>D BDÍ
    modelBuilder.Entity<IntegrationEntity>();D
}D
}D
D
public class FullTriggerChainTestsD
{D
    [Fact]D
    public async Task SaveChanges_ShouldExecuteFullChain_AndCancelOnValidationError()D
    {D
        // --- ARRANGE ---D

D
        // 1. B >Ct4C 5CÂ DCT9C= >C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()D
            .AddInMemoryCollection(new Dictionary<string, string?>D
            {D
                ["DatabaseSettings:DefaultSchema"] = "test"D
            })D
            .Build();D

D
        var services = new ServiceCollection();D
        services.AddLogging();D
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ :Cä=DD8C2 2 DI0

D
        // 2. A,,=DD@C AD$@D4:D$CD =D´5 C00D BD >C":C•
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });D
        // AÄ>C¢ DC 1D 8C=8 D$5C05D L C$>Ct2D 0D´0CTB C= >C0BCT:D B D :Cä=DD8C4>CÍ
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();D
        services.AddSingleton(factoryMock.Object);D

D
        // 3. B 5C48D BD 0Dd8D0 ACT@C$8D >C" BD 8C43CT@Cä2D
        services.AddScoped<DatabaseTriggerService>();D
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());D
D
        // B 0CÄ8 D$@C,,3C45D K0
        services.AddScoped<FluentValidationTrigger>();D
        services.AddScoped<ReferenceTreeParentTrigger>();D
        services.AddScoped<AuditTrigger>();D

D
        // 4. At0C4;D4HC=8 CD;D0 8C0BCT@DD5C"ACä2 (DtBCä1D² FCT?CäGC=0 C05 D 2C ;C ADÄÄ 5D ;C, BD 8C43CT@ D$0
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);D
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);D

D
        // 5. B 5C48D BD 0Dd8D0 2C ;C,,4C BCä@C
        services.AddTransient<IValidator<IntegrationEntity>, IntegrationValidator>();D

D
        ServiceProvider serviceProvider = services.BuildServiceProvider();D

D
        using IServiceScope scope = serviceProvider.CreateScope();D
        IServiceProvider sp = scope.ServiceProvider;D
        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();D

D
        // A0@C,,2D07D´2C 5CÂ fÇVVÇEf Æ-F F-0äg&-vvw" :Cä 2D 5CÂ AD4IC0>D BD0< D wV-B0:C´Ndt>CÍ
        triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();D

```

```

    triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

    // A,,!A A A' A AD B ääUB ¢ Ct2C'5Cæ0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C@Cä2C 9CD5D 0D
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A00D BD >C":C >CöFC,,9 A D 8D ?D 0C$;CT=CÖKCÄ ?CT@CTEC$0D$GC,,:Cä< (Cö5D 5CD0CT< D$>C'LCæ> DD0C
    DbContextOptions<ApplicationDbContext> options = new
    DbContextOptionsBuilder<ApplicationDbContext>().
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!A A A' A : Ää4C,,= C @C4C
        .Options;

    // B >Ct4C 5CÄ :Cä=D$5CæAD"Ä ?CT@CT4C 2C 0 Cä?Dd8C, 8 Cæ>CÖDC,,3D
    await using var context = new IntegrationDbContext(options, configuration);
    await context.Database.EnsureCreatedAsync(TestContext.Current.CancellationToken);

    // B >Ct4C 5CÄ >C JCT:D"Ä :CäBCä@D'9 A C@Cä9CD5D" 2C ;C,,4C FC,,N (Name CöCD BCä9)D
    var entity = new IntegrationEntity { Name = "" };
    context.Add(entity);

    // --- ACT & ASSERT ---

    // Ää6C,,4C 5CÄ GD$> C,,=D$5D FCT?D$>D 2D'7Cä2CTB D$@C,,3C45D Ä BCäB C$Kct>C$5D" 2C ;C,,4C BCä@, C, 1D4
    var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    {
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);
    });

    // A0@Cä2CT@Dö5CÄ GD$> CD>D,,;C, 8CÄ5CÖ=Cä 4Cä =C HCT3Cä ACä>C ICT=C,,0 C,,7 IntegrationValidator
    Assert.Equal("NameRequired", exception.Message);
}
</file>

```

```

<file path="Tests/Trigger/ReferenceTreeParentTriggerTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class ReferenceTreeParentTriggerTests
{
    // --- 1. B$ B "Ää B' Aæ A !B + ---

    private class TestNode : ReferenceTreeBase<TestNode> { }

    // Aæ>CÖBCT:D B D$5Cö5D L C@C,,=C,,<C 5D" "6öæf-wW& F-öí
    private class TestDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<TestNode>();
        }
    }

    // --- 2. A AD Ää"Ä Aæ Ää B #Ad A Bò 00Ý

    private readonly IConfiguration _configuration;

    public ReferenceTreeParentTriggerTests()
    {
        // B >Ct4C 5CÄ DCT9Cæ>C$KC' :Cä=DD8C2 4C'0 D$5D BCä2D
        _configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })

```

```

        })
        .Build();
    }

    private async Task<TestDbContext> GetContextAsync()
    {
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;

        var context = new TestDbContext(options, _configuration);
        await context.Database.EnsureCreatedAsync();
        return context;
    }

    // --- 3. B$ B "B² 00Ý

    [Fact]
    public async Task Trigger_ShouldCancel_WhenObjectIsItsOwnParent()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        var nodeId = Guid.NewGuid();
        // Id C, &VçD-B ACä2Cö0CD0DäB
        var node = new TestNode { Id = nodeId, ParentId = nodeId, Name = "Self Parent" };

        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            node, EntityStateChangeEnum.Added, [], context);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Equal("Aä1D=5C=B C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", args.ErrorMessage);
    }

    [Fact]
    public async Task Trigger_ShouldCancel_WhenParentDoesNotExistInDatabase()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        var missingParentId = Guid.NewGuid();
        var node = new TestNode { Name = "Orphan Node", ParentId = missingParentId };

        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            node, EntityStateChangeEnum.Added, [], context);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Contains("C05 C00C"4CT=", args.ErrorMessage);
    }

    [Fact]
    public async Task Trigger_ShouldNotCancel_WhenParentExistsInDatabase()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        var parent = new TestNode { Name = "Valid Parent" };
        context.Add(parent);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        var child = new TestNode { Name = "Child Node", ParentId = parent.Id };
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            child, EntityStateChangeEnum.Added, [], context);
    }

```

```

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.False(args.Cancel);
        Assert.Null(args.ErrorMessage);
    }

    [Fact]
    public async Task Trigger_ShouldCancel_WhenParentIsSoftDeleted()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        var parent = new TestNode
        {
            Id = Guid.NewGuid(),
            Name = "Deleted Parent",
            DeletedAt = DateTime.UtcNow,
            DeletedBy = "Admin"
        };
        context.Add(parent);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        var child = new TestNode { Name = "Child of Dead", ParentId = parent.Id };
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            child, EntityStateChangeEnum.Added, [], context);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Equal("A05C'Lct0 C00Ct=C GC„BDÂ @Cä4C„BCT;CT< D44C ;CT=C0KC' >C JCT:D"â"Â &w2äw'&÷$ÖW76 vR"½
    }

    [Fact]
    public async Task Trigger_ShouldCancel_WhenDirectCyclicDependencyDetected()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        // 1. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ @Cä4C„BCT;DÄAC=8C' MC'5CÄ5C0B A
        var nodeA = new TestNode { Id = Guid.NewGuid(), Name = "Node A" };
        context.Add(nodeA);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // 2. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ 4CäGCT@C08C' MC'5CÄ5C0B B (B >CD8D$5C'L: A •
        var nodeB = new TestNode { Id = Guid.NewGuid(), Name = "Node B", ParentId = nodeA.Id };
        context.Add(nodeB);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // 3. A„<C„BC„@D45CÂ ?Cä?D'BC=C D 4CT;C BDÂ Cct5C² " @Cä4C„BCT;CT< CD;D0 Cct;C „ CTBC'0: A -> B ->
        nodeA.ParentId = nodeB.Id;

        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            nodeA, EntityStateChangeEnum.Modified, [], context);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Equal("Bd8C=C„GCTAC=0D0 7C 2C„AC„<CäAD$L: C05C'Lct0 C05D 5CÄ5D BC„BDÂ @Cä4C„BCT;DÄAC=8C' Cct5
    }

    [Fact]
    public async Task Trigger_ShouldCancel_WhenDeepCyclicDependencyDetected()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        // A0>D BD >C„< Dd5C0>Dt:D3¢ &ö÷B Óâ 6†-ÆB Óâ 7V$6†-ÆM
    }

```

```

var root = new TestNode { Id = Guid.NewGuid(), Name = "Root" };
context.Add(root);

var child = new TestNode { Id = Guid.NewGuid(), Name = "Child", ParentId = root.Id };
context.Add(child);

var subChild = new TestNode { Id = Guid.NewGuid(), Name = "SubChild", ParentId = child.Id };
context.Add(subChild);

await context.SaveChangesAsync(TestContext.Current.CancellationToken);

// A,,<C,,BC,,@D45CÂ ?Cä?D´BC¤C Cö5D 5CÄ5D BC,,BDÂ AC <D´9 C$5D ECö8C´ Cct5C² %&ö÷B" 2CöCD$@DÂ 5C4> C4;D4
// Bd5Cö>Dt:C ?D >C$5D :C, ?Cä9CD5D" 2C$5D E: SubChild (C" AB´ Óâ 6†-ÆB „2 A ) -> Root (Cö5D 5CÄ5D
root.ParentId = subChild.Id;

var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(&
    root, EntityStateChangeEnum.Modified, [], context);

// Act
await trigger.HandleAsync(args);

// Assert
Assert.True(args.Cancel);
Assert.Equal("Bd8C¤;C,,GCTAC¤0Dò 7C 2C,,AC,,<CäAD$L: Cö5C´Lct0 Cö5D 5CÄ5D BC,,BDÂ @Cä4C,,BCT;DÄAC¤8C´ Cct5
}
}
</file>

<file path="Tests/Trigger/TreeTriggerChainTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- B4 A,, A BÄ B´ B #B" Aä!B$ AD Bò At A´/Bd A, "AT!B$ A" òòÝ
public class SoftDeleteNode : ReferenceTreeBase<SoftDeleteNode> { }
public class SelfParentNode : ReferenceTreeBase<SelfParentNode> { }

// Aä1D"8C´ :Cä=D$5C¤AD" 4C´O C,,=D$5C4@C FC,,>Cö=D´E D$5D BCä2 (D$5Cö5D L D "6öæf-wW& F-öâ•
public class TreeTestDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<SoftDeleteNode>();
        modelBuilder.Entity<SelfParentNode>();
    }
}

public class TreeTriggerChainTests
{
    private (ServiceProvider Provider, IConfiguration Config) CreateProviderAndConfig()
    {
        // B >Ct4C 5CÂ DCT9C¤>C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;Dö5CÂ :Cä=DD8C=

        // 1. AäACö>C$=D´5 D 5D 2C,,AD½

```

```

        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());
        // 2. B A4 B "B B #AT A$!AR "B A4 AT B%
        services.AddScoped<FluentValidationTrigger>();
        services.AddScoped<AuditTrigger>();
        services.AddScoped<ReferenceTreeParentTrigger>();

        // 3. At0C$8D 8CÄ>D BC, BD 8C43CT@Cä2D
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });

        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
        services.AddSingleton(factoryMock.Object);

        // 4. At0C4;D4HC8 CD;Dò 8CÔBCT@DD5C"ACä2D
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);

        return (services.BuildServiceProvider(), configuration);
    }

    [Fact]
    public async Task SaveChanges_ShouldCancel_WhenParentIsSoftDeleted_ThroughChain()
    {
        // --- ARRANGE ---
        (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
        using IServiceScope scope = serviceProvider.CreateScope();
        IServiceProvider sp = scope.ServiceProvider;

        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
        triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

        // A,,!Aô A A' Aô AD Bò ääUB ¢ Ct2C'5C80CT< DD0C @C,,D2 66÷ R 8Cr BCTAD$>C$>C4> Cò@Cä2C 9CD5D 0D
        var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

        // A,,!Aô A A' Aô : Aô5D 5CD0CT< C" 8CÔBCT@Dd5CôBCä@ D$>C'LC8> Cä4C,,= C @C4CCÄ5CÔB – scopeFactory
        DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
        .Options;

        // Aô5D 5CD0CT< Cä?Dd8C, 8 C8>CÔDC,,3D
        await using var context = new TreeTestDbContext(options, configuration);

        // 1. B >Ct4C 5CÄ $CCD0C'5CÔ=Cä3Cä" @Cä4C,,BCT;Dý
        var deadParent = new SoftDeleteNode
        {
            Id = Guid.NewGuid(),
            Name = "Dead Parent",
            DeletedAt = DateTime.UtcNow
        };
        context.Add(deadParent);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // 2. B 5C 5CÔ>C-
        var child = new SoftDeleteNode
        {
            Name = "Child",
            ParentId = deadParent.Id
        };
        context.Add(child);

        // --- ACT & ASSERT ---
        await Assert.ThrowsAsync<OperationCanceledException>(async () =>
        {
            await context.SaveChangesAsync(TestContext.Current.CancellationToken);
        });
    }

    [Fact]
    public async Task SaveChanges_ShouldCancel_WhenSelfParenting_ThroughChain()
    {
        // --- ARRANGE ---
        (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();

```



```

using IServiceScope scope = serviceProvider.CreateScope();
IServiceProvider sp = scope.ServiceProvider;

var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

// A,,!Aô A A´ Aô AD Bô ääUB ¢ Ct2C´5Cð0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> Cô@Cä2C 9CD5D 0D
var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

// A,,!Aô A A´ Aô : Aô5D 5CD0CT< C" 8C0BCT@Dd5C0BCä@ D$>C´LCð> Cä4C,,= C @C4CCÄ5C0B – scopeFactory
DbContextOptionsBuilder<ApplicationDbContext>(options)
    .UseInMemoryDatabase(Guid.NewGuid().ToString())
    .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
    .Options;

await using var context = new TreeTestDbContext(options, configuration);

var node = new SelfParentNode { Id = Guid.NewGuid(), Name = "Self" };
node.ParentId = node.Id;
context.Add(node);

// --- ACT & ASSERT ---
var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    await context.SaveChangesAsync(TestContext.Current.CancellationToken));

Assert.Equal("Aä1Dð5CðB C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", exception.Message);
}
</file>

```

```

<file path="Tests/Validator/DomainObjectValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Validator;

public class DomainObjectValidatorTests
{
    // 1. B >Ct4C 5CÄ BCTAD$>C$KC´ >C JCT:D" 8 CT3Cä 2C ;C,,4C BCä@
    private class TestEntity : DomainObject { }

    private class TestEntityValidator : DomainObjectValidator<TestEntity> { }

    private readonly TestEntityValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenIdIsEmpty()
    {
        // Arrange
        var entity = new TestEntity { Id = Guid.Empty };

        // Act
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.Id)
            .WithErrorMessage("A,,4CT=D$8DD8Cð0D$>D >C JCT:D$0 C05 CÄ>Cd5D" 1D´BDÂ ?D4AD$KCÄâ""½);
    }

    [Fact]
    public void Validator_ShouldFail_WhenCreatedAtIsInFuture()
    {
        // Arrange
        var entity = new TestEntity
        {
            CreatedAt = DateTime.UtcNow.AddMinutes(5)
        };

        // Act
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.CreatedAt)
            .WithErrorMessage("AD0D$0 D >Ct4C =C,,0 C05 CÄ>Cd5D" 1D´BDÂ 2 C CCDCD"5CÄâ""½);
    }
}

```

```

    }
}
[Fact]
public void Validator_ShouldPass_WhenObjectIsValid()
{
    // Arrange
    var entity = new TestEntity(); // Id C, 7&V FVD B 7C ?Cä;CÖDäBD O C" :Cä=D BD CC=BCä@CR 1C 7Cä2D´E C
    // Act
    TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);
    // Assert
    result.ShouldNotHaveAnyValidationErrors();
}
}
</file>

<file path="Tests/Validator/ReferenceBaseValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Validator;
public class ReferenceBaseValidatorTests
{
    // B$5D BCä2C O D CD"=CäAD$L C, 5E 2C ;C,,4C BCä@D
    private class TestReference : ReferenceBase { }
    private class TestReferenceValidator : ReferenceBaseValidator<TestReference> { }

    private readonly TestReferenceValidator _validator = new();

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData("A")] // AÄ5CÖLD,,5 2 D 8CÄ2Cä;Cä2D
    public void Name_ShouldHaveErrors_WhenInvalid(string? invalidName)
    {
        var model = new TestReference { Name = invalidName! };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name);
    }

    [Fact]
    public void Name_ShouldHaveError_WhenTooLong()
    {
        var model = new TestReference { Name = new string('A', 251) };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name)
            .WithErrorMessage("Aö@CT2D´HCT=C <C :D 8CÄ0C´LCÖ0Dö 4C´8CÖ0 CÖ0C,,<CT=Cä2C =C,,O (250 D 8CÄ2Cä;C
    }

    [Theory]
    [InlineData("VALID_CODE_123")]
    [InlineData("REF1")]
    [InlineData("")] // B$5Cö5D L DÖBCä ?D >C"4CTB D4ACö5D,,=Cí
    [InlineData(null)] // A, MD$> D$>Cd5D
    public void Code_ShouldBeValid_WhenCorrectFormat(string? code)
    {
        var model = new TestReference { Code = code };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldNotHaveValidationErrorFor(x => x.Code);
    }

    [Theory]
    [InlineData("C>CEö=C ö@D4AD :Cä<")]
    [InlineData("lower_case")]
    [InlineData("CODE-1")] // B$8D 5 CtöCö@CTICT=Cí
    public void Code_ShouldHaveError_WhenInvalidFormat(string invalidCode)
    {
        var model = new TestReference { Code = invalidCode };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);
    }
}

```

```

        result.ShouldHaveValidationErrorFor(x => x.Code);
    }

    [Fact]
    public async Task ValidateValue_ShouldReturnErrors_OnlyForSpecificProperty()
    {
        // Arrange
        var model = new TestReference
        {
            Name = "", // AäHC,,1C=0
            Code = "INVALID-CODE" // AäHC,,1C=0
        };

        // Act
        // Aô@Cä2CT@Dô5CÂ @C 1CäBD2 2C HCT3Cä 4CT;CT3C BC f Æ-F FUF ÇVR 4C´O Cö>C´O Name
        IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
        nameof(TestReference.Name));
        IEnumerable<string> codeErrors = await _validator.ValidateValue(model,
        nameof(TestReference.Code));

        // Assert
        Assert.Contains(nameErrors, e => e.Contains("Aô0C,,<CT=Cä2C =C,,5 Cä1Dô7C BCT;DÄ=Cä"´´"½
        Assert.Contains(codeErrors, e => e.Contains("A=CB <Cä6CTB D >CD5D 6C BDÄ BCä;DÄ:Cä ;C BC,,=C,,FD2"´´½

    }

    [Theory]
    [InlineData(" Name")]
    [InlineData("Name ")]
    public void Name_ShouldHaveError_WhenHasLeadingOrTrailingSpaces(string invalidName)
    {
        var model = new TestReference { Name = invalidName };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name)
            .WithErrorMessage("Aô0C,,<CT=Cä2C =C,,5 Cô5 CÄ>Cd5D" =C GC,,=C BDÄADò 8C´8 Ct0C=0CôGC,,2C BDÄADò ?D >

    }

    [Fact]
    public async Task ValidateValue_ShouldIgnoreOtherPropertiesErrors()
    {
        // Arrange: Cä1C ?Cä;Dò =CT2C ;C,,4CôK
        var model = new TestReference { Name = "A", Code = "low_case" };

        // Act: C$0C´8CD8D CCT< B$ A´,A= Name
        IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
        nameof(TestReference.Name));

        // Assert
        Assert.Single(nameErrors); // B$>C´LC= CäHC,,1C=0 CD;C,,=D² 8CÄ5Cô8
        Assert.DoesNotContain(nameErrors, e => e.Contains("A=CB"´´² òò D,,8C >C¢ :Cä4C 1D´BDÄ =CR 4Cä;Cd=Ci

    }
}
</file>

```

```

<file path="Tests/Validator/ReferenceTreeBaseValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Validator;
public class ReferenceTreeBaseValidatorTests
{
    // B$5D BCä2C 0 D CD"=CäAD$L CD;Dò ?D >C$5D :C, 0C AD$@C :D$=Cä3Cä :C´0D AC
    private class TestNode : ReferenceTreeBase<TestNode> { }

    // B$5D BCä2D´9 C$0C´8CD0D$>D
    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenParentIdIsEqualToId()
    {
        // Arrange
        var nodeId = Guid.NewGuid();
    }
}

```

```

var node = new TestNode()
{
    Id = nodeId,
    ParentId = nodeId, // AäHC,,1Cð0: ID D >C$?C 4C 5D" A ParentId
    Name = "Self-referencing node"
};

// Act
TestValidationResult<TestNode>? result = _validator.TestValidate(node);

// Assert
result.ShouldHaveValidationErrorFor(x => x.ParentId)
    .WithErrorMessage("Aä1Dð5CðB C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
}

[Fact]
public void Validator_ShouldFail_WhenParentIdIsEmptyGuid()
{
    // Arrange
    var node = new TestNode()
    {
        ParentId = Guid.Empty, // AäHC,,1Cð0 C0> C$BCä@Cä<D2 ?D 0C$8C´C
        Name = "Empty ParentId"
    };

    // Act
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);

    // Assert
    result.ShouldHaveValidationErrorFor(x => x.ParentId)
        .WithErrorMessage("B >CD8D$5C´LD :C,,9 C,,4CT=D$8DD8Cð0D$>D =CR <Cä6CTB C KD$L C0CD BD´< GUID");
}

[Fact]
public void Validator_ShouldPass_WhenParentIdIsDifferent()
{
    // Arrange
    var node = new TestNode()
    {
        Id = Guid.NewGuid(),
        ParentId = Guid.NewGuid(),
        Name = "Valid child node"
    };

    // Act
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);

    // Assert
    result.ShouldNotHaveValidationErrorFor(x => x.ParentId);
}

[Fact]
public void Validator_ShouldPass_WhenParentIdIsNull()
{
    // Arrange (Að>D 5C0L CD5D 5C$0)
    var node = new TestNode()
    {
        Id = Guid.NewGuid(),
        ParentId = null,
        Name = "Root node"
    };

    // Act
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);

    // Assert
    result.ShouldNotHaveValidationErrorFor(x => x.ParentId);
}
}
</file>
</files>

```